ҚАЗАҚСТАН РЕСПУБЛИКАСЫНЫҢ ҒЫЛЫМ ЖӘНЕ ЖОҒАРЫ БІЛІМ МИНИСТРЛІГІ

Ш. ЕСЕНОВ атындағы КАСПИЙ ТЕХНОЛОГИЯЛАР ЖӘНЕ ИНЖИНИРИНГ УНИВЕРСИТЕТІ

«Педагогика» факультеті

6В01509 – Информатика (IP)

білім беру бағдарламасының

«Объектлі-бағдарланған программалау» пәні бойынша

ДӘРІСТЕР ЖИНАҒЫ

АҚТАУ 2025

Дәріс №1

Дәріс тақырыбы:Алгоритмдеу негіздері

Дәріс мақсаты: Алгоритмдеу негізгі ұғымдарымен танысу. Алгоритмнің қасиеттерін, берілу жолдарын қарастыру

Кілт сөздер: Алгоритм, Алгоритмнің негізгі ұғымдары, Алгоритмнің қасиеттері:Дискреттілік, түсініктілік, детерминділік, нәтижелік, жұптылық (аяқталушылық), жалпылық, орындаушы, команда, әрекеттер тізбегі, қадам, блок-схема элементтері: бастау, процесс, шешім, енгізу/шығару, байланыс

Дәріс жоспары:

1. **Негізгі ұғымдар мен түсініктер**
2. **Алгоритм қасиеттері**
3. **Алгоритмдерді бейнелеу жолдары**

1. Негізгі ұғымдар мен түсініктер

ЭЕМ-ді пайдалану істерін қарастырмас бұрын оның жұмысымен тығыз байланысты алгоритм, программа ұғымдарын білуіміз қажет. Әрбір ЭЕМ алдын ала берілген алгоритммен, яғни жоспармен жұмыс істейді. Алгоритмді заңдылық, реттел- ген амалдар жиыны, кезекпен орындалатын операциялар тізімі деп ұғынған жөн. Бұл ұғым қазіргі кезде кеңінен қолданылып жүр. Оның көптеген анықтамалары да бар. Соның бірін келтіре кетейік.

Алгоритм – *берілген есептің шығару жолын реттелген амалдар тізбегі түріне келтіру*. Кез келген есепті қарапайым амалдарды тізбектей орындау арқылы шығаруға болады. Алгоритмді ЭЕМ-де орындау үшін оны программа түрінде жазып шығу керек. Сонымен, алгоритм оны атқарушы ЭЕМ-ге жұмыс тәртібін түсіндіретін ережелер мен нұсқаулар тізбегінен тұрады. Алгори- тмді атқарушының рөлін негізінен адам немесе автоматтандырылған аспап, яғни ЭЕМ, робот, т.б. атқарады. Мысалы, $y=(ax+b)(cx-d)$ функциясын есептеу төменгі іс-әрекеттерден тұрады:

1) a-ны x-ке көбейту, оны R_1 деп белгілеу;
2) оған b-ны қосу, нәтижесін R_2 деп белгілеу;
3) c-ны x-ке көбейту, оны R_3 деп белгілеу;
4) одан d-ны алу, оны R_4 деп белгілеу;
5) R_2-ні R_4-ке көбейту, оны y деп белгілеу.

Алгоритмнің орындалу кезінде оны орындаушыға келесі жолы қандай нұсқау бойынша орындалатыны белгілі болуы қажет. Ал орындаушының жүзеге асыра алатын командалар жиыны – командалар жүйесін құрайды.

Алгоритм мен программаға байланысты ЭЕМ-нің мынадай жұмыс ерекшеліктері болады:

1) есепті шығару жолы алгоритм түрінде өрнектелуі қажет;
2) алгоритм программаға айналдырылуы тиіс;
3) программа машина жадына енгізіліп, ретімен орындалуы керек. Алгоритм күнделікті тұрмыста да кеңінен қолданылады. Мысалы, студент болу үшін алгоритмнің мынадай қадамдарын орындау керек.

1. Орта мектепті бітіріп, аттестат алу.
2. Керекті құжаттарды аттестаттың түпнұсқасымен бірге белгілі бір оқу орнына өткізу.
3. Конкурстан өту.

Бұл көрсетілген пункттердің орнын ауыстыруға болмайды. Олар көрсетілген ретпен кезектесіп орындалуы тиіс. Сонда ғана керекті нәтижеге (студент болу) қолымыз жетеді.

Алгоритм информатиканың іргелі ұғымдарының бірі. Квадрат теңдеудің түбірін табу ережесі, үшбұрыштың ауданын есептеу жолдары алгоритмдердің мысалдары болып табылады.

Алгоритмдеу – *есепті шығару алгоритмін құрастыру процесі*, мұның нәтижесінде мәліметтерді өңдеу процесінің кезең- дері айқындалады да, кезеңдер мазмұны формальды (жасанды) түрде жазылып, солардың орындалу реттілігі анықталады.

Алгоритмдік тіл – алгоритмдерді жазуға арналған символдар мен сол символдардан тұратын конструкцияларды құрас- тыру және түсіндіру ережелерінің жиыны.

Программалау тілі ЭЕМ-дерде программаларды орындау ісін атқарады.

Программа – *алгоритмді машинаға түсінікті нұсқаулар тізімі ретінде жазу.* Программада берілген мәліметтердің сипаттамаларымен бірге оларды өңдейтін командалар болады. Осы командалар тізбегі орындалу барысында есептің нәтижесі шығады. Командалар қандай мәліметтер қандай операцияларға қатынасатынын, олар қандай реттілікпен орындалатынын және нәтиженің қандай түрде шығарылатынын көрсетеді. Бұлар опера- торлар арқылы жүзеге асырылады.

Әрбір ЭЕМ алдын ала жазылған программамен істейді. Процессор программаның құрамындағы командаларды кезекпен орындап отырады. Командалар тізбегін программа деп қарастыруға болады. Команда бір ғана қарапайым амалды орындау үшін берілген бұйрық ретінде беріледі. Командалар: арифметикалық немесе логикалық амал; ақпаратты тасымалдау командасы; берілген сандарды салыстыру командасы; келесі командаларға көшу тәртібін орындау, т.с.с.

Алгоритм және программа ұғымдары ұқсас екені көрініп тұр, алгоритм есептің шығару жолын қарапайым әрекеттер тізбегімен өрнектесе, программа сол өрнекті машинаға түсінікті тілмен жазып береді. Сондықтан программа жазу тәсілдері солардың қандай ережені пайдаланып, қандай компьютерлерге арналып жазылғанына байланысты алгоритмдік тілдер немесе программалау тілдері

ұғымына келіп тіреледі.

Мәліметтер – белгілі бір процесс көмегімен тасымалдап, өңдеуге болатын, формальды түрде бейнеленген фактілер мен идеялар.

Оператор – операциялар мен мәндерді көрсететін немесе солардың элементтерінің қай жерде орналасқанын білдіретін символдар жиыны. Мысалы:

A:= B+C; {A, B, C – айнымалы;} K:= 2; IF T<0 THEN ...

Айнымалы – программа орындалуы барысында өз мәнін өзгерте алатын объект.

Айнымалы қасиеттері:

1. айнымалы белгілі бір мәнге ие болғанша, анықталмаған болып саналады. Оған мән беру мынадай тәсілдермен орындалады:
   - сырттан енгізу арқылы;
   - тұрақты мәнді (константаны) меншіктеу арқылы;
   - бұрын анықталған айнымалының мәнін беру арқылы;
2. кез келген сәтте айнымалының белгілі бір мәні болады не- месе ол анықталмаған болып есептеледі;
3. айнымалыға соңғы берілген мән оның алдыңғы мәнін жойып (өшіріп) жібереді. Айнымалыны таңдау (оқу) және оны пайдалану айнымалының мәнін өзгертпейді.

Бұл пәннің заттық негізі болып (ЭЕМ-де шығару мақсатында) алгоритмдер мен программаларды құрастыру тәсілдері мен құралдары саналады. Программалар құру үшін программалау жүйелері пайдаланылады.

Программалау жүйесі – программалауды автоматтандыру құралдары. Олар программалау тілінен, осы тілдің трансляторы-

нан, құжаттамаларынан және де программаларды дайындау, әрі орындау құралдарынан тұрады.

Транслятор – бір тілді екінші тілге аудару программасы. Ол интерпретатор және компилятор сияқты екі топқа бөлінеді.

Интерпретатор – бұл командаларды аударып, оларды бірден орындауға арналған трансляторлық программа.

Компилятор – бұл алгоритмдік тілдің конструкцияларын толығымен машиналық кодқа түрлендіретін программа. Есептің нәтижесін алу түшін машиналық кодты орындау керек.

2. Алгоритм қасиеттері

Алгоритм ұғымының мәнін ашатын негізгі қасиеттерінен немесе оған қойылатын талаптардан қысқаша мағлұматтар кел- тірейік. ЭЕМ-де орындалуға тиіс алгоритмдерге мынадай талап- тар қойылады:

1) ол анық әрі дәл өрнектелуі тиіс – детерминділік (анықтылық, бір мәнділік) қасиеті, яғни алгоритмді басқаша түсінуге жол бермей, тек қана көрсетілген әрекеттерді айқын түрде орындауға арналған нұсқаулар дәлдігі;

2) алгоритм шектелген уақыттан соң нәтиже беруі тиіс – нәтижелілік қасиеті, мұнда белгілі бір әрекеттер саны атқарылған соң, процестің қажетті нәтижесін алып, оны аяқтау мүмкіндігі- нің болуы немесе есептеу процесін ары қарай жалғастыруға болмайтындығы жайлы мәлімет алуымыз қажет;

3) бір тектес есептерге жалпы бір ғана алгоритм қолданылуы тиіс – жалпылық қасиеті, ол алгоритмнің осы сияқты көптеген басқа да есептерге қолданылу мүмкіндігінің болуын көрсетеді;

4) алгоритмді кішкене бөліктерге бөлу мүмкіндігі болуы қажет – модульдік (дискреттілік, бөліктік) қасиет – есептеу процесін жекеленген қарапайым операцияларға бөлу мүмкін- дігінің болуы, яғни күрделі есепті атқарылуына күдік келтіруге болмайтын шағын бөліктерге жіктеу орындалуы тиіс.

Біріншіден, алгоритм анық, әрі *дәл өрнектелуі* қажет. Онда қандай қадамдар көрсетілсе, тек соны ғана орындау керек. Есеп шығару жолына керектің бәрі біржақты анықталуы және орындаушыға түсінікті, әрі нақты болуы тиіс.

Екіншіден, алго- ритм *нәтижелі болуы* керек. Әрекеттердің шектелген санынан кейін белгілі бір уақыт ішінде қорытынды нәтиже алуымыз қажет. Әрбір алгоритм біршама бастапқы мәліметтердің болуын талап етеді және іздеген нәтижені алуға жеткізеді. Мысалы, сандарды қосу алгоритмі үшін бастапқы мәліметтерге қосылғыштар мәні жатады, ал нәтижесі қосынды болады.

Үшіншіден, алгоритмнің *жалпылық қасиеті* болады, яғни бастапқы мәліметтер мәнінің бір жиыны бір ғана нәтиже береді. Егер берілген мәліметтер өзгерсе, нәтиже де өзгереді. Басқаша айтқанда, бір алгоритм бір типтес есептердің әр түрлі алғашқы мәліметтері үшін әр түрлі нәтижелер беруі тиіс. Мысалы, квадрат теңдеуді шешу алгоритмі кез келген *a, b, c* мәндері үшін оның түбірін дұрыс табуы керек.

Төртіншіден, алгоритмнің *үзік-үзік модульдерге бөліну* қасиеті болуы тиіс, яғни үлкен алгоритмді бірнеше кішкене алгорит- мдерге жіктеуге әрқашанда мүмкіншілік болуы керек. Сондықтан алгоритмді екі-үш бөлікке бөліп, оларды өзінше құра алатын дәрежеде жұмыс істелуі қажет. Олар тек бірінің қорытындысын келесі жолы керекті мәлімет ретінде қолдануы тиіс.

3. Алгоритмдерді бейнелеу жолдары

Алгоритмдерді ЭЕМ-де орындау үшін оларды алдын ала жазып алу керек, яғни ол белгілі бір заңдылықпен өрнектелуі тиіс. Жалпы алгоритмді жазып өрнектеу, яғни оларды бейнелеу түрлеріне төмендегі тәсілдер жатады:

- табиғи тіл сөздері арқылы;

- формулалық-сөздік тәсіл арқылы;
- графикалық түрде бейнелейтін блок-схемалар арқылы;
- псевдокодтар арқылы;
- құрылымдық диаграммалар арқылы;
- программалау тілі арқылы.

Алгоритмдерді **табиғи тіл сөздері арқылы** бейнелеуде – есептеу кезеңдері мазмұны кез келген түрде табиғи тілде жазылады.

Осы тәсілмен келесі мысалдың алгоритмін жазып шығайық. Сандар жиымы (массиві) берілген делік. Осы жиым сандарының көрсетілген аралықта, яғни интервалда толығынан жататынын/ жатпайтынын тексеру керек. Интервал өзінің шекаралық А және В мәндерімен берілген.

1. Бірінші санды аламыз.

2. Осы сан интервалға кіретінін салыстыру жолымен тексереміз; егер жауабы «Иә» болса, онда **3-пунктке көшу**, әйтпесе (жау- абы – «Жоқ» болса) – **6-пунктке көшу**.

3. Жиымның барлық элементтері қарастырылды ма? Егер жауабы «Иә» болса, онда **5-пунктке көшу**, жауабы «Жоқ» болса,

– **4-пунктке көшу**.

4. Келесі элементті қарастырамыз. **2-пунктке көшу**.

5. Мынадай хабарлама шығару: барлық элементтер осы интервалға кіреді. **7-пунктке көшу**.

6. Мынадай хабарлама шығару: элементтер интервалға толы- ғынан кірмейді.

7. Соңы.

Бұл тәсілде көрнекілік жоқ, яғни толық формальдау мүмкін- дігі жоқ. Жалпы алгоритмді табиғи тілде өрнектеу ЭЕМ-дерде қолданылмайды, өйткені онда дәлдік, нақтылық болмайды.

Алгоритмдерді **формулалық-сөздік тәсіл арқылы** бейнеленуі – тапсырманың математикалық символдар мен өрнектер- дің және сөздердің араласуымен берілуі болып табылады.

Мысалы, үшбұрыш ауданын оның үш қабырғасының ұзын- дығы арқылы есептеу алгоритмін құру керек болсын делік.

1. – үшбұрыштың жарты периметрін есептеу

$$p=(a+b+c)/2$$

2. – үшбұрыштың ауданын есептеу

3.
$$S = \sqrt{p(p-a)(p-b)p}$$

4. – нәтиже ретінде *S* мәнін шығарып, алгоритм жұмысын аяқтау. Бұл тәсілді пайдаланғанда, алгоритмді кез келген деңгейде айқындап көрсетуге болады, бірақ формальды түрде анық бейнелеу қиын.

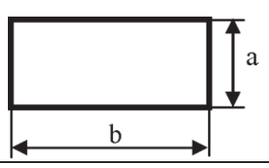
Алгоритмді **графикалық түрде блок-схемалар арқылы** көрсету – оның логикалық құрылымын графикалық түрде бей- нелеу болып саналады. Бұл – алгоритмдерді өрнектеудің ең көп тараған түрі. Мұнда мәліметтерді өңдеудің әрбір кезеңі атқарыла- тын операцияға сәйкес әр түрлі геометриялық фигуралар (блок- тар) түрінде көрсетіледі.

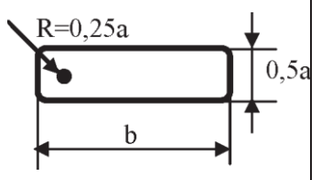
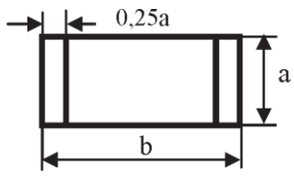
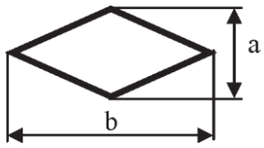
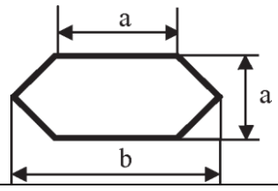
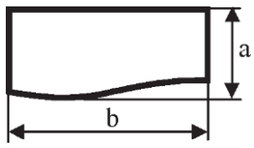
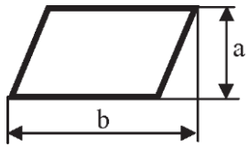
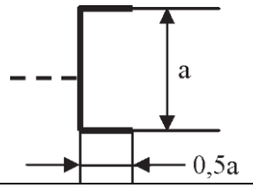
Әр блоктың ішіне орындалатын іс-әрекеттің (амалдың) мазмұны жазылады. Символдардың (блоктардың) бір кіру және бір шығу сызықтары болуға тиіс.

Графикалық жолмен алгоритмдерді жазу үшін мемлекеттік стандарт белгіленген, онда кез келген амал белгілі бір геомет- риялық фигурамен өрнектеледі. Ол фигуралар немесе блоктар амалдар символы деп те аталады. Блоктар бағытталған сызық- тармен байланысып, бірінен соң бірі орналасады. Жиі қолданы- латын амалдар, яғни мәліметтерді ЭЕМ-ге енгізу, формуламен есептеу, шарттардың орындалуын тексеру, нәтижені қағазға басу символдары 1-кестеде көрсетілген. Осы суреттегі көрсетілген блоктардан (символдардан) алгоритм схемалары құрастырыла- ды. Алгоритмдер схемасымен ақпаратты өңдеудің әрбір сатысы немесе орындалатын операциялар реті анықталады. Кейде алго- ритмдер схемасын оның блок-схемасы деп те атайды.

1- кесте

Алгоритмдерді бейнелеу блоктары

Іс-әрекеттің аты	**Блоктың пішімі**	**Атқаратын жұмысы**
Процесс		Математикалық өрнектерді есептеу

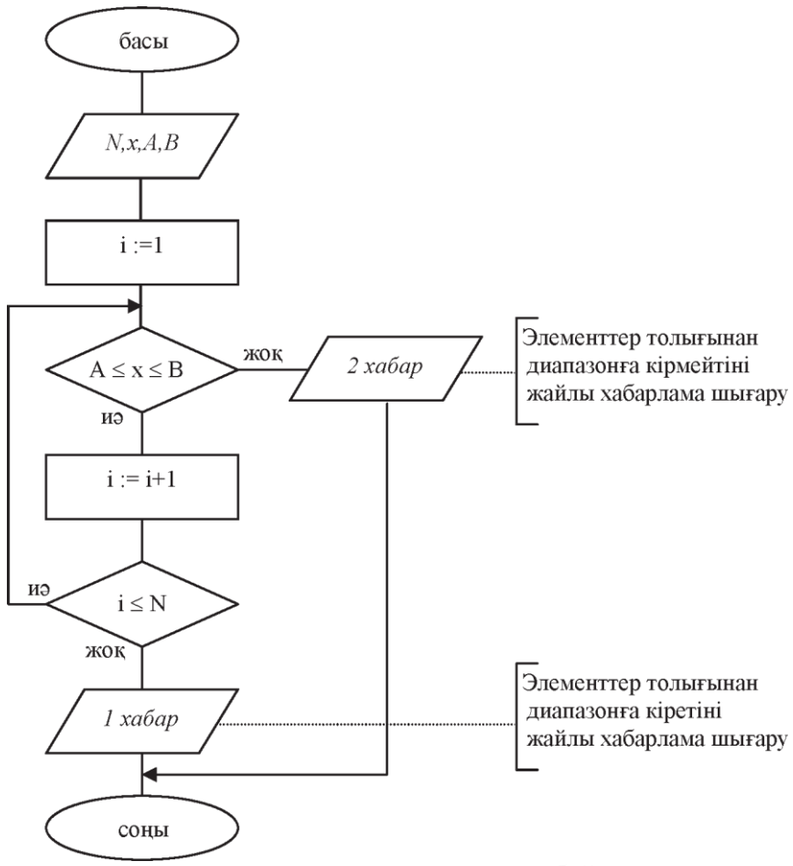
Басы – соңы		Алгоритм дерді бастау, аяқтау
Алдын ала анықталған процесс (подпрограмма)		Қосалқы программаларға кіру және шығу
Шешім		Есеп шығару жолын таңдау
Модификация		Цикл басы
Құжат		Нәтижені баспаға (қағазға) шығару
Енгізу-шығару		Мәліметтерді енгізу және шығару
Түсініктеме		Схеманы, формула-ларды түсіндіру

Сонымен алгоритм блоктармен немесе геометриялық көпбұ- рыштар түріндегі фигуралармен өрнектеледі.

Блок-схемалар дәстүрлі (кәдімгі) және құрылымды болып екіге бөлінеді.

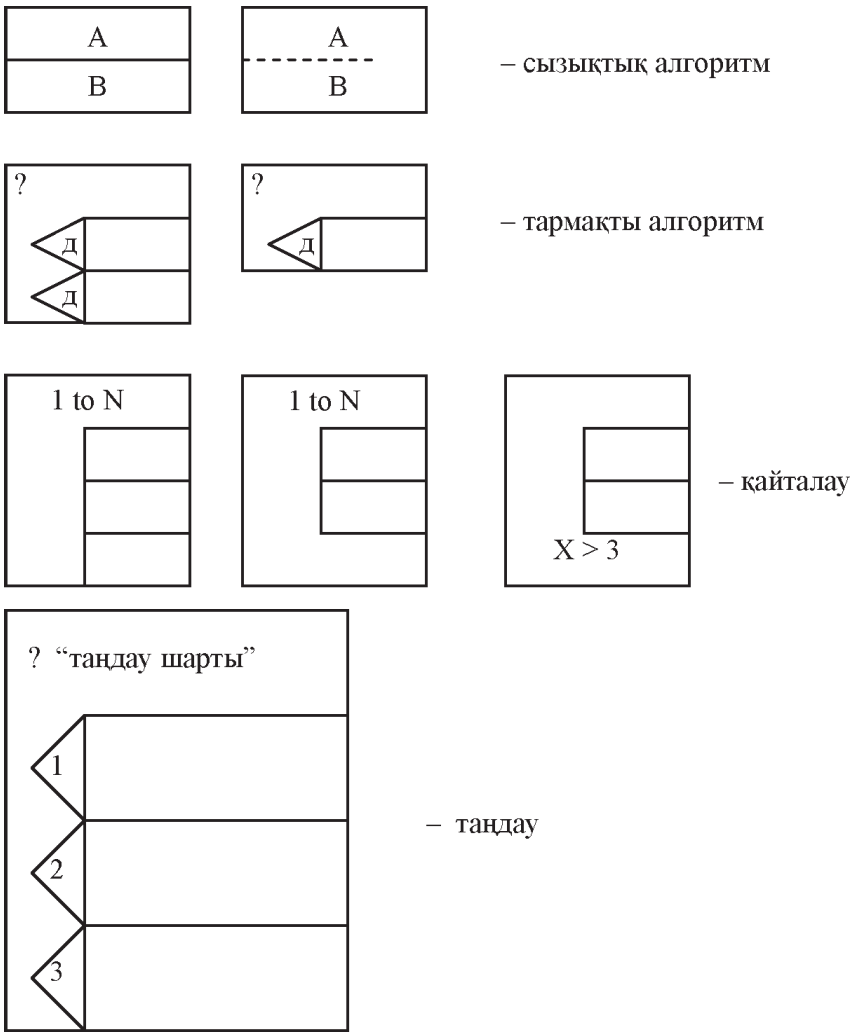
Табиғи тіл сөздері арқылы көрсетілген алгоритмдегі мысал- ды блок-схемалар арқылы бейнелеу төмендегі суретте көрсетіл- ген. Онда берілген жиым сандарының көрсетілген аралықта,

яғни интервалда толығынан жататынын/жатпайтынын тексеру керек болатын. Интервал өзінің шекаралық А және В мәндерімен берілген.



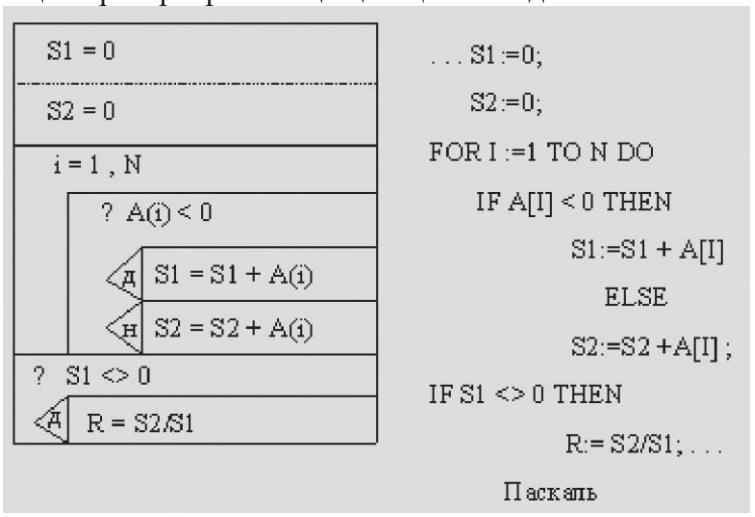
Құрылымдық диаграммалар – модульдер арасындағы бай- ланысты көрсететін, мәліметтер құрылымын, программаларды және мәліметтерді өңдеу жүйелерін бейнелейтін құрылымдық блок-схемалар ретінде пайдаланылады. Құрылымдық диаграмма- лардың бірнеше түрлері болады: Насси-Шнейдерман диаграмма- сы, Варнье диаграммасы, Джексон диаграммасы, МЭСИД және т.б. диаграммалары.

МЭСИД диаграммасының негізгі элементтері



МЭСИД диаграммасын пайдалануға бір мысал келтірейік. Оң және теріс сандардан тұратын бір өлшемді жиым беріл-

ген. Осы жиымдағы оң элементтердің қосындысын оның теріс элементтерінің қосындысына бөлгендегі бөліндіні анықтау қа- жет. Төменде осы есептің диаграммасы мен соған сәйкес Паскаль тілінің операторлары оның оң жақ бөлігінде



келтірілген.

Программалау тілдері – программаларды ЭЕМ-де тікелей орындауға арналған алгоритмдерді жазу тәсілі. Программа – алгоритмнің ЭЕМ-ге түсінікті түрде жазылуы.

Әрбір машинаның өз тілі (машиналық тіл) болады және ол тек осы тілдегі программаларды, яғни командалар тізбегін орын- дай алады. Программаларды машиналық тілде жазу өте күрделі, әрі адамды шаршататын жұмыс болып табылады. Программа- лаушылардың жұмыс өнімділігін арттыру мақсатында жасанды тілдер, яғни программалау тілдері қолданылады. Мұндайда жа- санды тілде жазылған программа машиналық тілге аударылуы тиіс. Аудару жұмысын, яғни программаны бір тілден екінші тілге түрлендіру ісін транслятор атқарады. Жиі қолданылатын, тікелей интерпретация жасайтын транслятор түріне Бейсик тілінің транс- ляторы жатады, ол командаларды аударады да, оны бірден орын- дайды. Мұндай транслятор жұмысының қорытындысы қажетті нәтижелерді алу болып табылады.

Алгоритмдік, яғни программалау тілдері есептерді шығару жолын баяндау-өрнектеу үлгісі, белгілі бір проблеманы шешу негізінде орындалатын әрекеттерге басшылық, ой еңбегін үнем- деуге мүмкіндік беретін әдіс, есеп шешімін табуды автомат- тандыруға қажетті іс-әрекет, жаңа проблеманы шешу кезінде қолданылатын тәсілдер, күрделі процестерді өрнектеу және мате- матикалық дәлдікпен анық етіп жазу құралы бола алады.

Дәріс №2

Дәріс тақырыбы: Python бағдарламалау тілі туралы жалпы деректер. Python бағдарламалау тілінің шығу тарихы.

Дәрістің мақсаты: *Python бағдарламалау тілі туралы жалпы деректер беру,* тіл мүмкіндіктерін қарастыру.

Кілт сөздер: Python, Жоғары деңгейлі тіл, Интерпретацияланатын тіл, Кроссплатформалық, Ашық бастапқы код (open source), Қарапайым синтаксис

Дәріс жоспары:

1. Python бағдарламалау тілінің шығу тарихы.
2. Python программалау ортасын қолдану
3. Python тілінің мүмкіндіктері.

1. Python бағдарламалау тілінің шығу тарихы.

Python – жоғары деңгейлі және әмбебап бағдарламалау тілі болып табы- лады. Ол қарапайым және қуатты объектілі-бағытталған бағдарламалау тілі.

Python бағдарламалау тілі 1980 жылы ойластырылған және оны құру Ни- дерландыда математика және информатика орталығында Гидо ван Россумның көмегімен 1989 жылдың желтоқсанынан басталды. Python қазіргі уақытта белсенді түрде жетілдірілуде. Оның жаңа нұсқалары жиі шығады. Ресми сайты http://python.org.

Python бағдарламалау тілінің артықшылықтары:

- Python тілі интернетте тегін таратылады;
- Кітапханасы өте бай, мүмкіндіктері мол;
- Web қосымшаларын жасайды;
- Мәліметтер базасымен жақсы жұмыс істей алады;
- Үстелдегі дербес компьютерлерде жеңіл программаланады;
- Мәліметтерді талдау ісінде де қолданылады;
- Бейнелер мен суреттерді жақсы өңдейді;
- Объектіге бағытталған программалауды сүйемелдейді;
- Интерактивті түрде жұмыс істеуге де ыңғайлы болып табылады.

Оның ерекше айырмашылығы-командаларды блогқа топтастыру үшін Python кілт сөздерді немесе басқа таңбаларды емес, шегіністерді қолданады. Python-ның басқа программалау тілдерінен айырмашылығы: белгілі бір құрылым жоқ, яғни программалық кодты жазу кезінде айнымалыларды немесе константтарды сипаттаудың белгілі бір ережелерін сақтау, операторларды жазу ережелерін сақтау қажет емес. Тіпті (.py) форматында сақталған бос файл орындалады және пайдаланушы қате туралы хабарлама алмайды.



Ойындар, жасанды интеллект

pandas

Ғылыми есептеулерді программалау

Жүйелік бағдарламалау

Графикалық интерфейс

Веб сценарийлер

Жылдам прототиптеу

Деректер қорының қосымшалары

Crython
Jython
IronPython
компоненттерінің интеграциясы

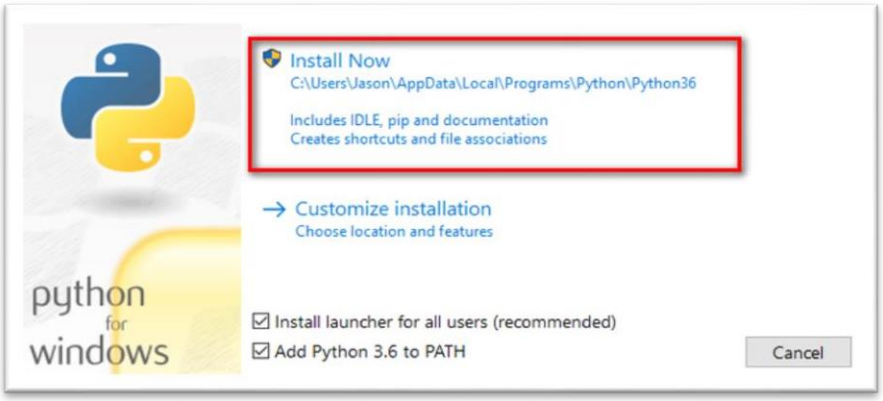
Python программалау тілі белсенді қолданылатын аймақтар

2. Python программалау ортасын қолдану

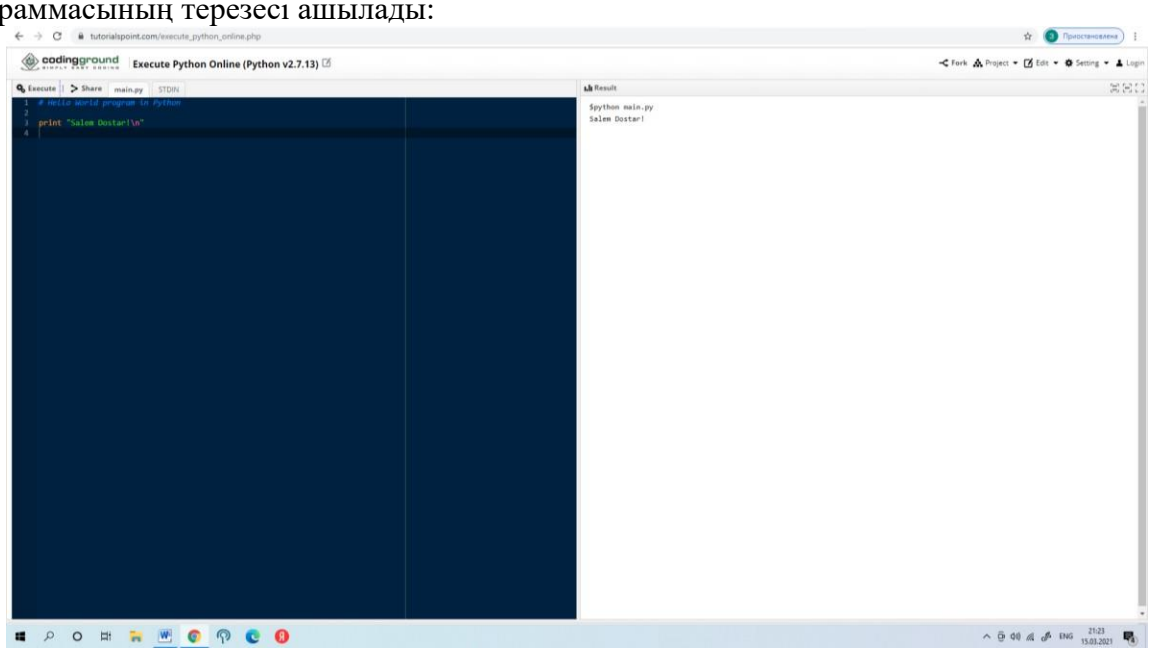
Python программалау тілін қолданбас бұрын, оны жүктеп, компьютерге орнату қажет. Python программасы тегін және оңай орнатылады. Windows, Mac, Linux және Ubuntu операциялық жүйелерінде жұмыс жасайды.

Windows-тегі Python орнату алгоритмі:

1. Python-ның веб-сайтына кіріңіз (http://www.python.org)
2. Windows-қа арналған Python-ның үшінші нұсқасынан бастап, тізімнің жоғары бөлігінде болатын соңғы нұсқасына басыңыз.
3. Орнату автоматты түрде жүктеледі, одан соң жүктеу файлын ашып,экранның төменгі жағындағы хабарламаларға «қанатша» белгілерді қойып, "Install Now" батырмасын басыңыз.

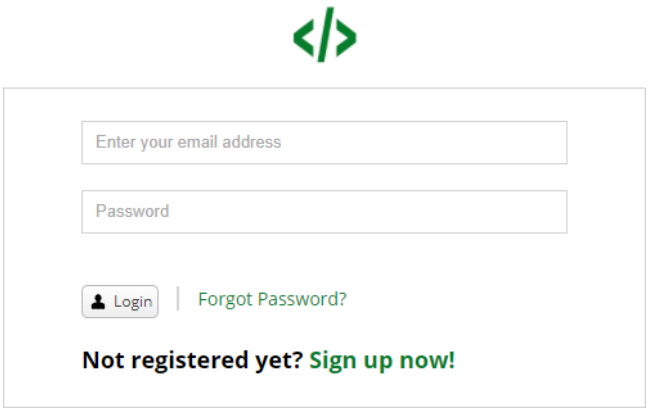


Компьютерге орнату уақыты мен жадын үнемдеу үшін Python программасының онлайн нұсқасын қолдануға болады: https://www.tutorialspoint.com/execute_python_online.php. Python программасының терезесі ашылады:



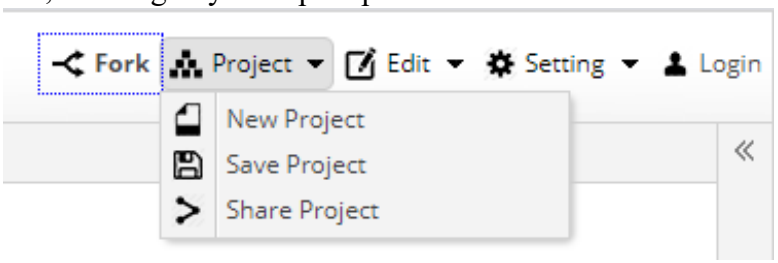
Терезенің сол жағында программа кодын теріп, «Execute» батырмасын таңдағаннан кейін терезенің оң жағындағы бөлігінде программаның нәтижесін көруге болады.

«Share» қойындысын пайдаланып программалық кодтың сілтемесімен бөлісуге болады.



«Execute» батырмасын басқаннан кейін терезенің оң жағында $python main.py жазуы пайда болады, содан кейін егер «print» операторын қолданған болсаң, жауапты көре аласың. Жұмыс аймағы «main.py» қосымшасын басқан кезде пайда болады, бұнда «main.py» (main) – файл атауы, (py) – типі Python болатын файлдың кеңейтілімі. Файл атауын өзгертуге болады. «STDIN» қосымшасын тәжірибелі пайдаланушылар пайдаланады және ол файлдармен жұмыс істеу үшін қажет.

Терезенің жоғарғы оң жағында мәзір бар, онда: «Share» бөлісу командасымен бірдей «ForK» және «Project», «Edit», «Setting» пункттері бар.



Python жұмыс ортасын қолдану

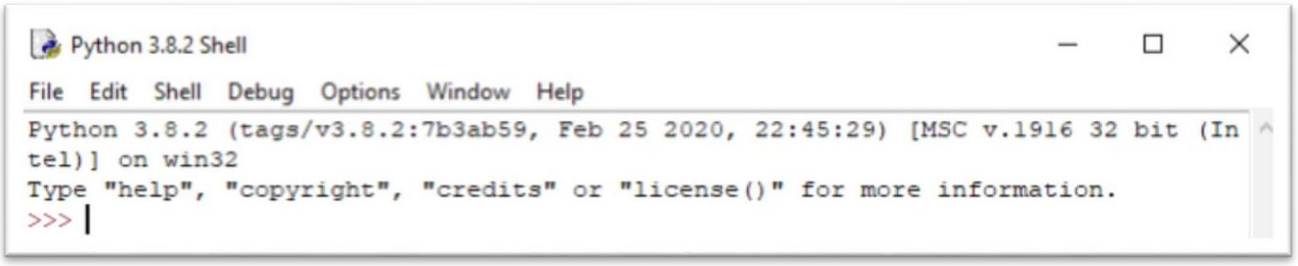
Интеграцияланған өңдеу ортасы (ағылш. Integrated DevelopmentEnvironment-IDE) - программалық қамтамасыз етуді әзірлеу үшін программистер пайдаланатын программалық құралдар жүйесі.

Python тіліне арналған көптеген сапалы IDE-лер бар(IDLE, CMD, PyCharm, Jupyter NotebooK).

IDLE (Integrated Development and Learning Environment) — бұл TKinter кітапханасының көмегімен құрылған Python тілін дамыту мен оқытудың интеграцияланған ортасы, көптеген платформаларда, соның ішінде Windows, Mac OS, Unix тәрізді операциялық жүйелерде қолданыла алады.

Tkinter кітапханасы Python-да стандартты модуль ретінде орнатылған, сондықтан оны пайдалану үшін бізге ешнәрсе орнатудың қажеті жоқ. TKinter - өте қуатты кітапхана. Егер сіз Python-ді бұрыннан орнатқан болсаңыз, сіз Python-мен бірге келетін интеграцияланған IDE-ді қолдана аласыз, бұл IDETKinter көмегімен жазылады.

Python IDLE- Python кодын жазуға арналған арнайы орта, **интерактивті жұмыс терезесі** Python Shell немесе мұны **терминал** деп атайды.

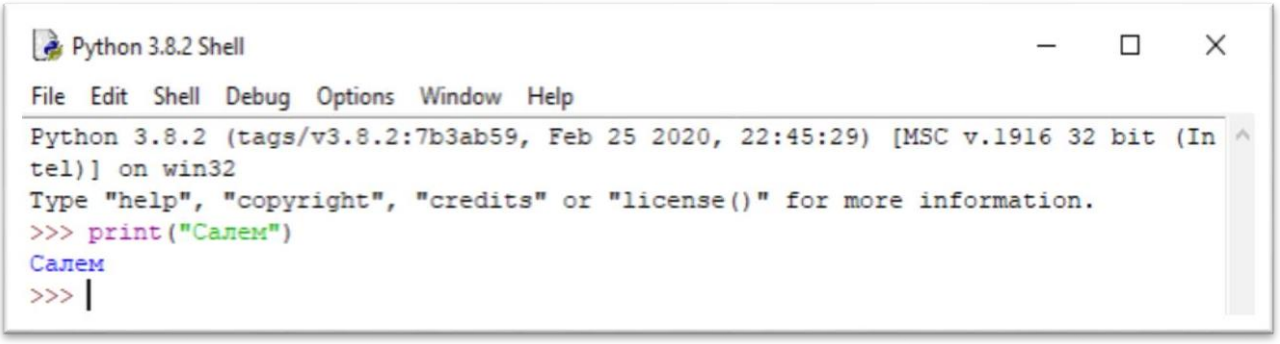


Python Shell терезесіндегі мәліметтерге тоқталатын болсақ, экранның ең жоғарыдағы бөлігіндегі мәлімет қай нұсқадағы интерпретатор екенін көрсетеді. Одан кейін сіздің компьютер туралы қосымша мәлімет береді.

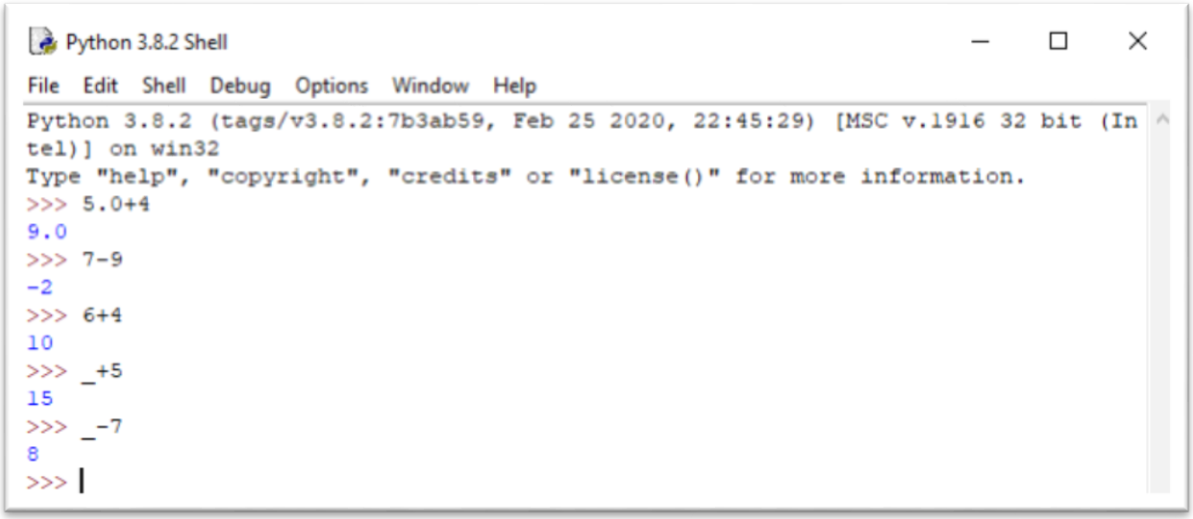
>>> -бір нәрсе енгізуімізді сұрайтын символ.

Python Shell терезесіндегі мәліметтер түстерінің де өзіндік мақсаты бар, Python командалары -күлгін түсті, мәтіндер-жасыл, нәтиже-көк түспен ерекшеленеді.

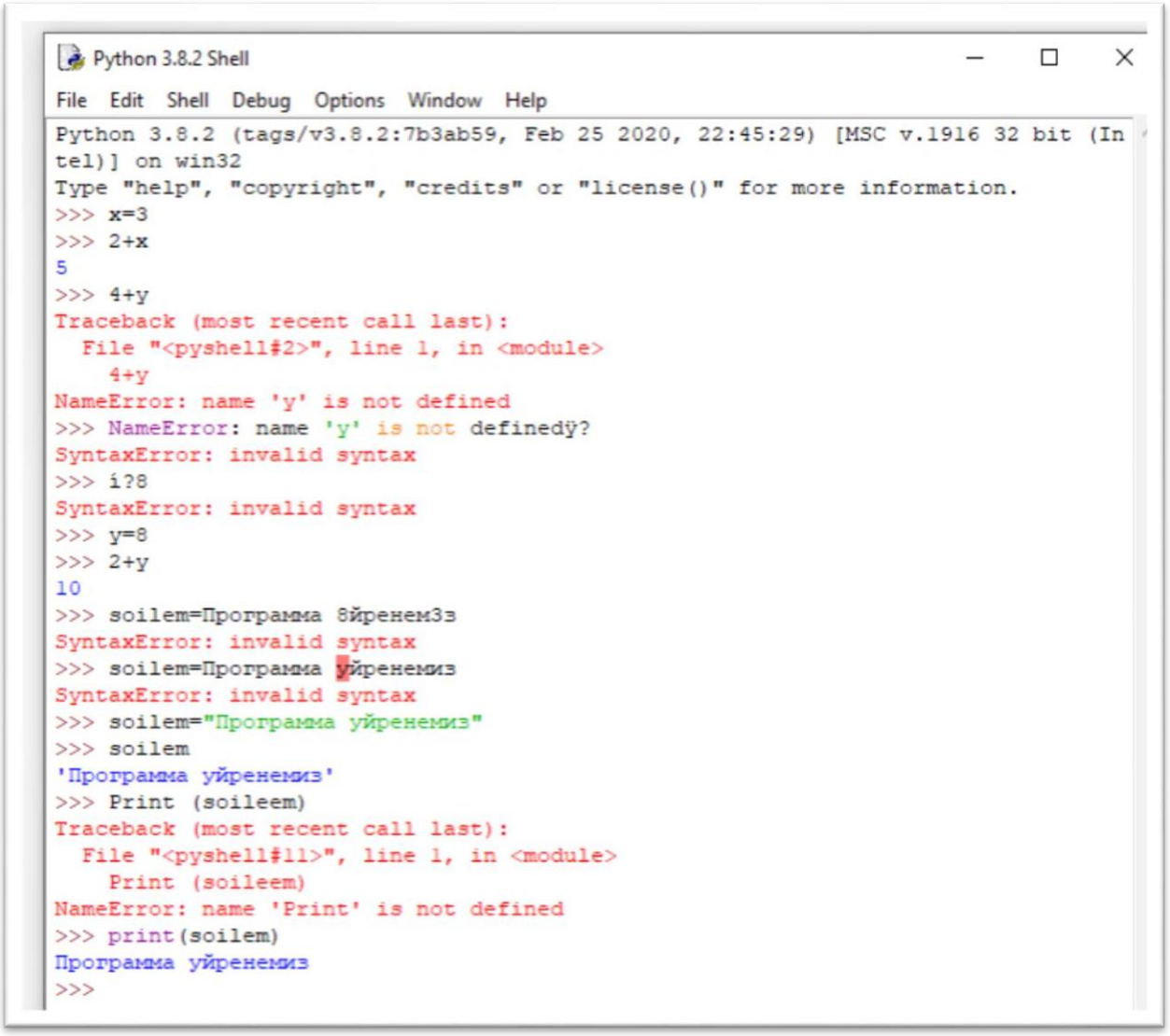
Python программасын кәдімгі интерактивті калькулятор ретінде қарастыруға болады.



Alt+p және **Alt-n** пернелері арқылы алдында жазылған командаларды көріп, қажеттісін таңдап алуға болады. IDLE интерактивті режимінде математикалық өрнектердің мәндерін табатын болсақ, өрнектерді жазып болғаннан кейін экрандағы нәтижелерді енгізу және шығару үшін Enter пернесін басамыз. Мысалдағы өрнек жазылуындағы төменгі сызба(нижний пробел) есептеуде алынған соңғы нәтижені қолданады дегенді білдіреді, яғни 10+5 және 15-7 өрнегімен бірдей.



Егер қандай да бір себептермен команданы енгізу кезінде қателік кеткен болса, онда қате туралы хабарлама шығады. Мысалы:



name 'y' is not defined - у аты анықталмаған деген қате беріп тұр, бұл жерде у-ке мән берілмеген.
SyntaxError: invalid syntax - SyntaxError: синтаксистік қате деп беріп тұр, сондықтан программа жазғанда индикатор ағылшын тілінде тұру керек.
NameError: name 'Print' is not defined - NameError: 'Print' атауы анықталмаған деп көрсетіп тұр, себебі soileem деген айнымалының мәні жоқ, енді қатесіз қайта жазсақ print (soilem), онда дұрыс нәтиже береді

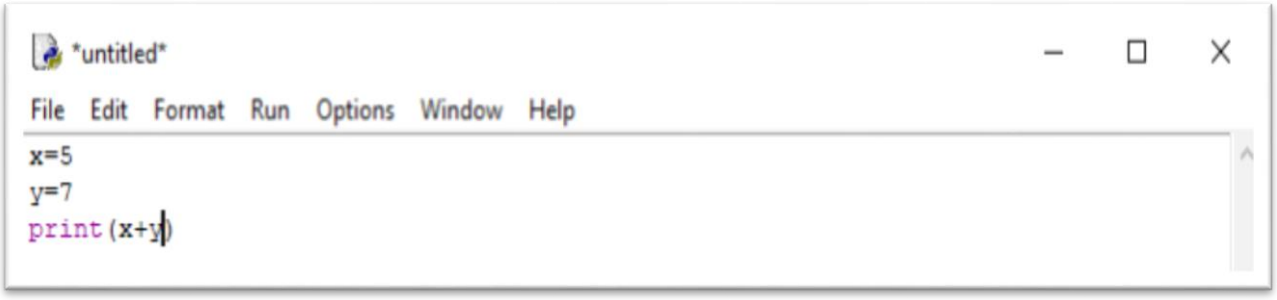
Жалпы программалау кезінде жіберілетін қателерді келесі топтарға бөледі: *Синтаксистік қателер* программа мәтінін теру кезінде операторлардың және программалау тілінің ережелерінің қате жазылуы.

Логикалық қателер есеп алгоритмінің дұрыс құрылмауынан болады.

Динамикалық қателер программаның орындалуы кезінде пайда болып, оның орындалу тәртібінің бұзылуына немесе нәтижесіз тоқтап қалуына әкелетін қателер.

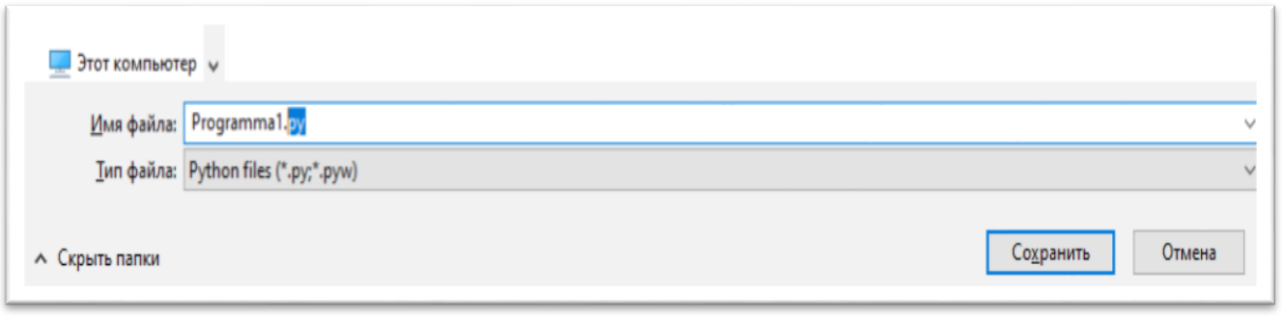
Интерактивті режимде интерпретатормен тікелей жұмыс атқарылады. **Скриптік** (Script) **режимде** программаны (кодты) файлға (Script) сақтап (кеңеймесі ry деп сақталады), интерпретаторды скриптік режимде орындаймыз. Меню жолынан **File → New File** пернесін бассақ, программамен жұмыс істеу

(құру, түзету, сақтау,орындау) терезесі, яғни мәтіндік редактор ашылады.

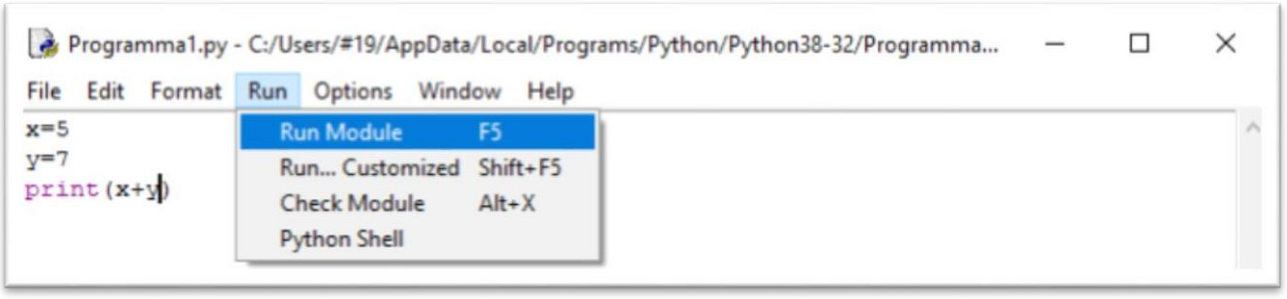


Бұл мәтіндік редактор Python программасының редакторы болғандықтан Python командаларын таниды. Программаны толық жазып шығамыз. Әр команданы өз алдына бір жолға жазамыз.

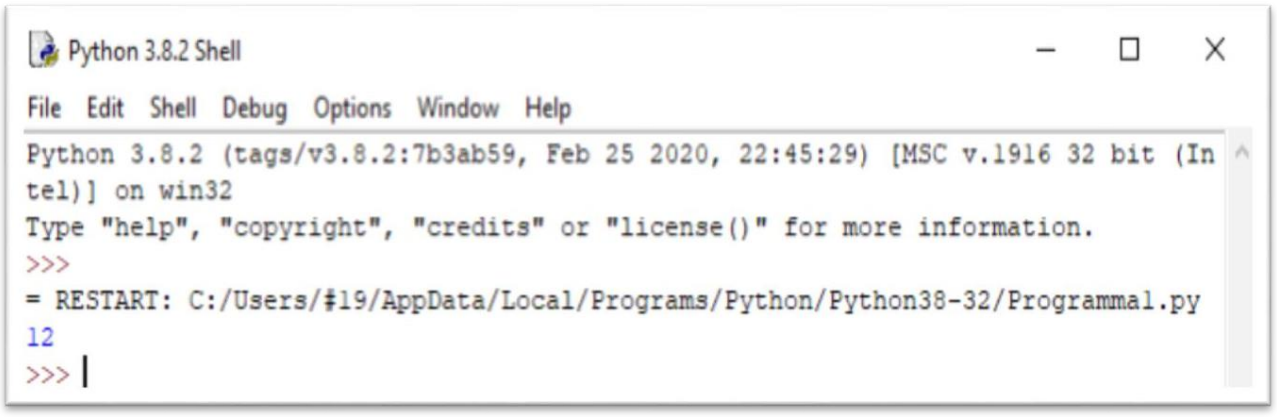
Программаны **File → Save as** командасын орындау арқылы қажетті каталогқа сақтау керек. Программа аты ***файл_аты.py*** Мұндағы .py-файлдың кеңейтілімі, Python программасы екенін білдіреді. Мысалы: Programma.py



Сақтағаннан кейін программаны орындауға жібереміз. Программаны орындау үшін менюде **Run → Run Module** немесе **F5** басу керек.



Нәтиже Python терминалының экранынан көрінеді. Экрандағы 12 саны біздіңпрограммамыздың нәтижесі:



Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Python қандай бағдарламалау тіліне жатады?
2. Python тілінің негізгі ерекшеліктері қандай?
3. Python тілінің артықшылықтарын атаңыз.
4. Python қай салаларда қолданылады?
5. Python интерпретацияланатын тіл екенін қалай түсінесіз?
6. Python тілінің синтаксисінің қарапайымдылығы нені білдіреді?
7. Python-ның кроссплатформалық қасиеті дегеніміз не?
8. Python тілінде модульдік жүйенің рөлі қандай?

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс №3

Дәріс тақырыбы: Айнымалыны анықтау. Мәліметтер түрлері.

Дәрістің мақсаты: Python тілінің мүмкіндіктері. Мәліметтер типтерімен жұмыс жасау.

Кілт сөздер: `int` – бүтін сандар, `float` – нақты сандар, `complex` – комплекс сандар, `str` – жолдар (мәтіндер), `bool` – логикалық тип (`True`, `False`), `list` – тізімдер, `tuple` – кортеждер, `set` – жиындар, `dict` – сөздіктер

Дәріс жоспары:

1. Айнымалы. Мәліметтер түрлері
2. Input() функциясы және print() функциясының мүмкіндіктері
3. Деректердің типін өзгерту.

1. Айнымалы-бұрын берілген атауы арқылы қол жеткізуге болатын әртүрлі типтегі деректерді сақтайтын жад ұяшығы. Айнымалы мәнді пайдалану кезінде программа барысында ұяшық мәні өзгеруі мүмкін, ал тұрақты мәнді пайдалану кезінде ұяшық мәні өзгермейді. Айнымалылар программадағы ақпараттың орналасу орнын анықтау үшін қолданылады.

Айнымалыны құру үшін, ең алдымен оның атын жазу керек. Одан кейін, тең белгісін қойып, айнымалыға мән беру қажет. Айнымалының мәні ретінде сандар, әріптер, сөздер және т.б. объектілер бола алады, айнымалы = өрнек. Python бағдарламасында деректер мен айнымалылар арасындағы байланыс

=белгісімен орнатылады. Мұндай операция **меншіктеу** деп аталады. Өрнектің мәні есептелінеді де, көрсетілген айнымалыға меншіктеледі. Python-да айнымалы түрі тек

бағдарламаны орындау кезінде анықталады.

Алгоритмдік тілдерде өрнектер бір жолға ғана жазылады. Амалдар математикалық ережелер бойынша орындалады.

Егер айнымалының мәні "None" сөзі болса, айнымалы бос болып есептелінеді. Айнымалының аты әріптен басталады және тек қана сандардан, әріптерден және сызық таңбаларынан тұра алады. Айнымалының аты сандардан бастала алмайды және де деректердің аты программалау тілінде қолданылатын қызметтік сөздермен бірдей болмауы керек:

and	as	assert	break	class	continue
def	del	elif	else	except	
finally	for	from	global	if	import
in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with
yield	True	False	None		

Дұрыс айнымалыларға мысал келтірсек: a = 7, num = 10, string = "hello", boolean = True, mans_name = "Alex".

Бұрыс айнымалыларға мысал келтірсек: 22num = 3, $money = 100, def = 20. Айнымалыға дұрыс емес ат қойып, бағдарламаны жүзеге асырсақ, қате шығып, бағдарлама жұмыс істемей қалады.

Айнымалының мәнін экранға шығару үшін, print(айнымалының аты) командасын теру қажет.

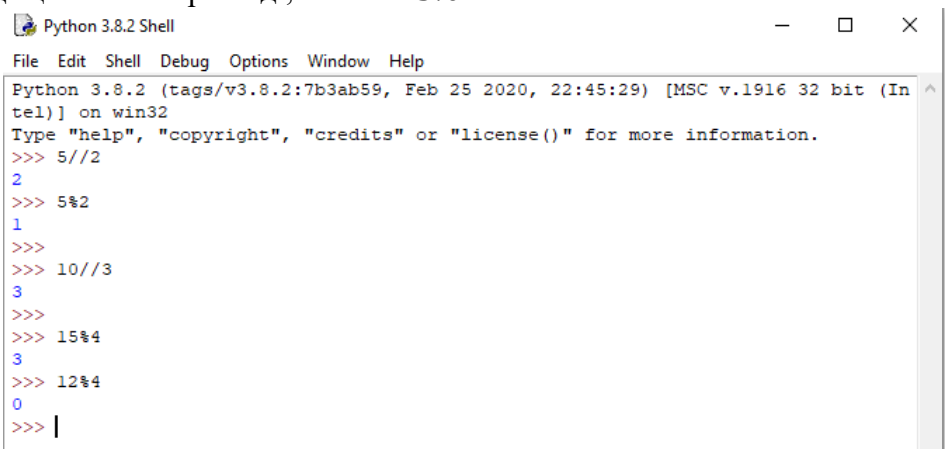
Бірнеше айнымалыға бірден, бір мәнді меншіктеуге болады. v=y=z=2*p+aКейбір меншіктеу амалдарын қысқартып жазуға болады.

оператор	мысал	орындалуы	амалдар
+=	x+=y	x=x+y	қосу
-=	x-=y	x=x-y	азайту
*=	x*=y	x=x*y	көбейту
/=	x/=y	x=x/y	бөлу
//=	x//=y	x=x//y	бүтін бөлу
%=	x%=y	x=x%y	қалдықпен бөлу
**=	x**=y	x=x**y	дәрежелеу

Мысалы: Санның бүтін бөлігі мен қалдық бөлігін табу.

//-бүтін бөлігі, мысалы 5//2=2.

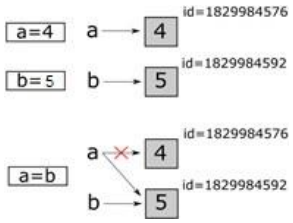
%-қалдық бөлігін көрсетеді, мысалы 5%2=1



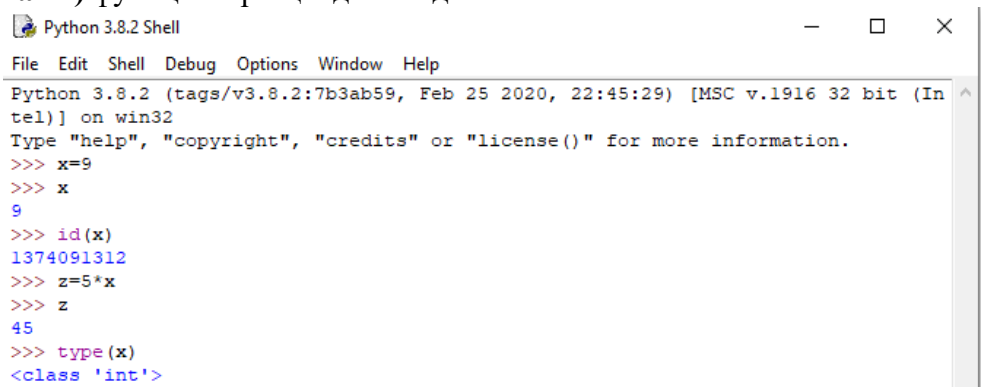
Анықталған деректің төрт қасиеті болады: **идентификаторы, адресі, түрі және мәні.**

Идентификатор дегеніміз осы деректің сандық номері. Бұл номер қайталанбайды.

Деректің идентификаторын білу үшін **id (дерек аты)**, ал түрін білу үшін



type (дерек аты) функциялары қолданылады.



Программада айнымалылардың қандай типі қолданылатынын білу өте маңызды. Дегенмен Python-ның басқа программалау тілдерінен айырмашылығы бар. Python айнымалыларды сипаттауды қажет етпейді. Python программалау тілінде қолданылатын айнымалылардың негізгі типтерін қарастырайық.

1. Сандардың үш түрі бар: бүтін, нақты (бөлшек) және күрделі. Бірақ қарапайым программаларды құрастыру кезінде әдетте бүтін және нақты сандарды қолданады.

2. Жолдар– бір немесе екі тырнақша ішіне жазылған таңбалар жиынтығы.

Мысалы, a='Hello, World!' және a = ' 'Hello, World!' ' – бірдей жазбалар. Ұзын мәтіндерді жазу кезінде үштік апострофтарды немесе тырнақшаларды қолдануға болады.

3. Логикалық тип (bool). Логикалық типтегі айнымалылардың екі мәні бар. Олар: Ақиқат (True), Жалған (False).

Есіңде сақта! «True», «False» бас әріппен жазылуы керек, әйтпесе программа қате деп қабылдайды.

Python бағдарламалау ортасында біз деректердің үш түрін қолданамыз:

- **бүтін сандар** (int) - оң және теріс бүтін сандар, сондай-ақ 0 (мысалы, 5, 798, -56, 0).
- **өзгермелі нүкте сандары** (float) – Бөлшек сандар (мысалы, 2.56, - 4.899765, 0.00564). Ескерту: санның бүтін және бөлшек бөлігі нүктемен ажыратылады.
- **жолдар** (string) - тырнақшаға алынған символдар жинағы, (мысалы, "bala", "Atyn Kim?", 'oKushy', '7698'). Ескерту: Python тырнақшалары бір немесе екеу болуы мүмкін.

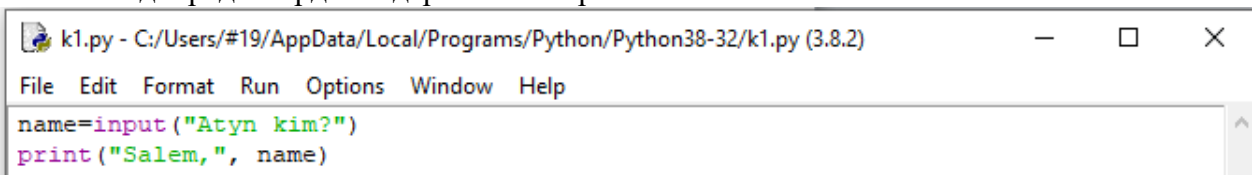
2. Input() функциясы және print() функциясының мүмкіндіктері

Программа пайдаланушыдан әртүрлі ақпаратты сұрай алады. Пайдаланушы пернетақта арқылы ақпаратты енгізеді, программа бұл ақпараттышығару кезінде қолдана алады.

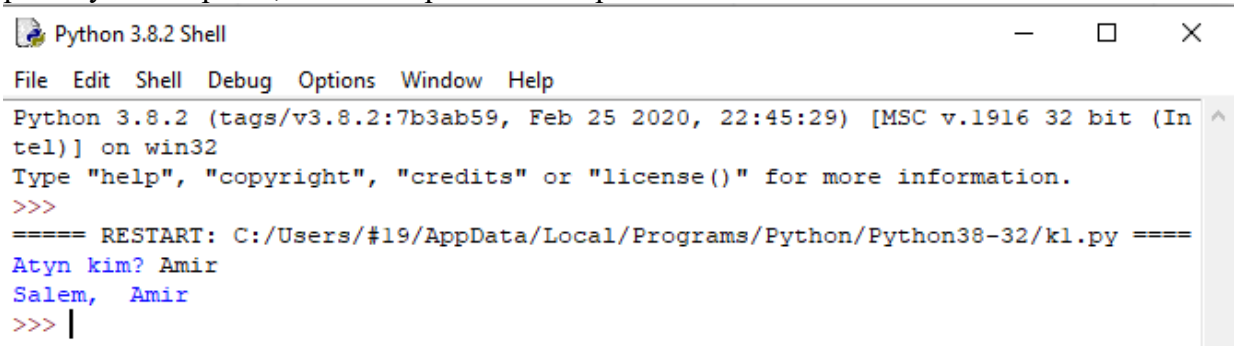
Программаға ақпарат енгізу үшін, **input()** функциясы қолданылады. Осы функция белгілі бір айнымалының ішінде болуы қажет.

Мысалы, name=input("Atyn Kim?"). Яғни бұл жағдайда, программа сізден тырнақшадағы сөздермен ақпарат сұрайды. Осыдан кейін, енгізілген ақпаратты экранға шығаруға болады. Бұл үшін, мысалға, print("Salem,", name) деп жазуға болады, бұл жағдайда программа “Salem” сөзімен бірге сіз енгізген есімді экранға шығарады. Python тілінде **print()** функциясы арқылы экранға енгізілген объектілер шығарылады.

Мәтіндік редакторда бағдарламаны теріміз.



Орындауға жібереміз, нәтиже терминалда көрінеді.



3. Деректердің типін өзгерту.

Деректердің бір түрін екіншісіне түрлендіруге мүмкіндік беретін арнайы функциялар (арнайы операторлар) қолданылады. Сонымен, **int ()** функциясы оған берілген жолды (немесе өзгермелі нүкте санын) **бүтін санға** түрлендіреді, **str()** функциясы оған берілген аргументті **жолға**, **float() - бөлшек санға** түрлендіреді. Мысалы:

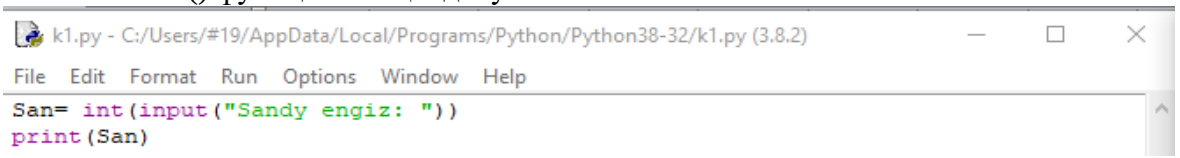
Өрнек	Орындалу нәтижесі
int ("67")	67
int (5.02)	5
int ("comp597")	қате
str (67)	'67'
str (5.02)	'5.02'
float (67)	67.0
float ("67")	67.0

Осыған қоса, егер сіз пайдаланушыдан сандарды сұрағыңыз келсе, input() функциясымен бірге **int()** немесе **float()** функцияларын қолдану керек. Жай ғанаinput() функциясын тергенде де, сіз сандарды енгізе аласыз, бірақ int() немесе float() функцияларымен бірге сіз сәйкесінше тек қана бүтін немесе бөлшек сандарды енгізе аласыз. Яғни сандарды енгізу кезінде бірден сәйкес түрге ауыстыруға болады.

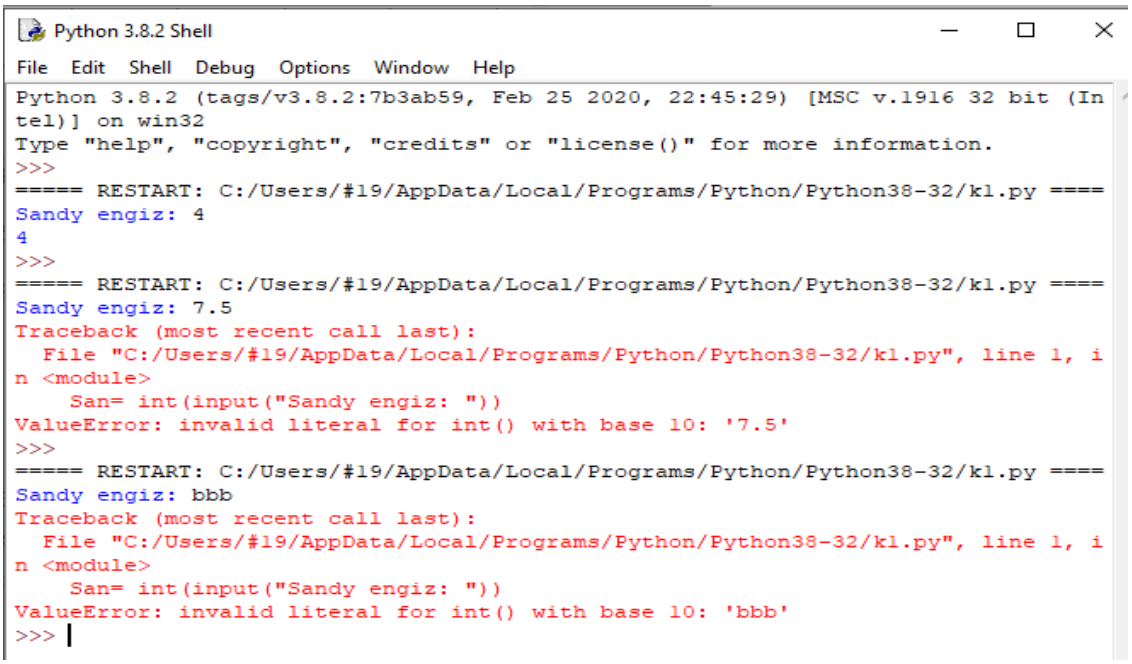
n = int(input('n='))

x = float(input('x='))

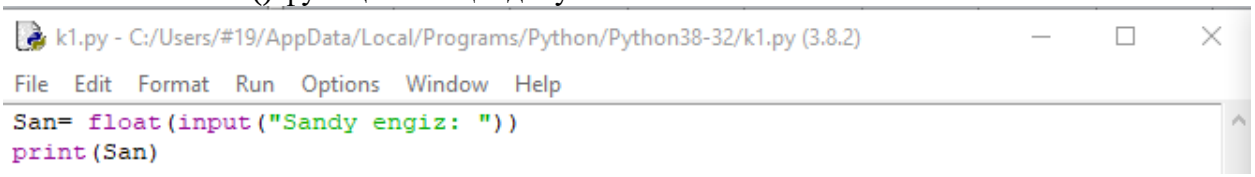
Мысалы: int() функциясын қолдану



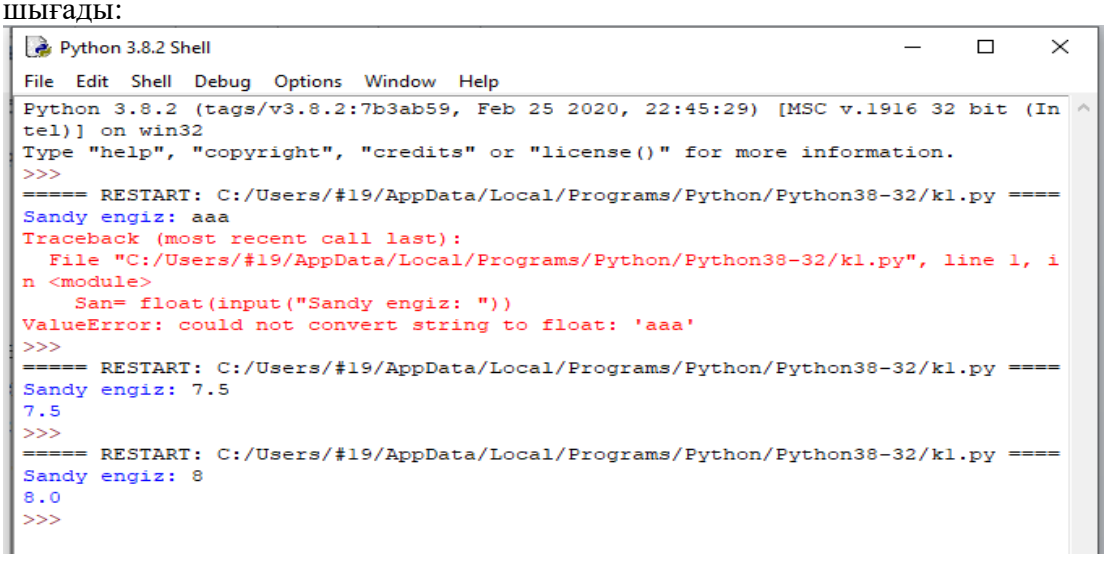
Егер бүтін сандарды енгізсек нәтиже дұрыс шығады, ал біз санның орнына жолды, бөлшек сандарды енгізсек қате шығады:



Мысалы: float() функциясын қолдану



Егер біз санның орнына жолды енгізсек қате шығады, бөлшек сандардыенгізсек нәтиже дұрыс шығады:



4. Математикалық операциялар

Python-да әртүрлі математикалық операциялар бар. Python-дағы барлық математикалық амалдар математикалық заңдарға толық бағынады. Мысалы, жақшалар амалдың қай бөлігін бірінші кезекте орындау керектігін көрсету үшін қолданылады. Python әрқашан ең алдымен жақшаның ішіндегі амалдарды орындайды, тек одан соң келесі амалдарға көшеді.

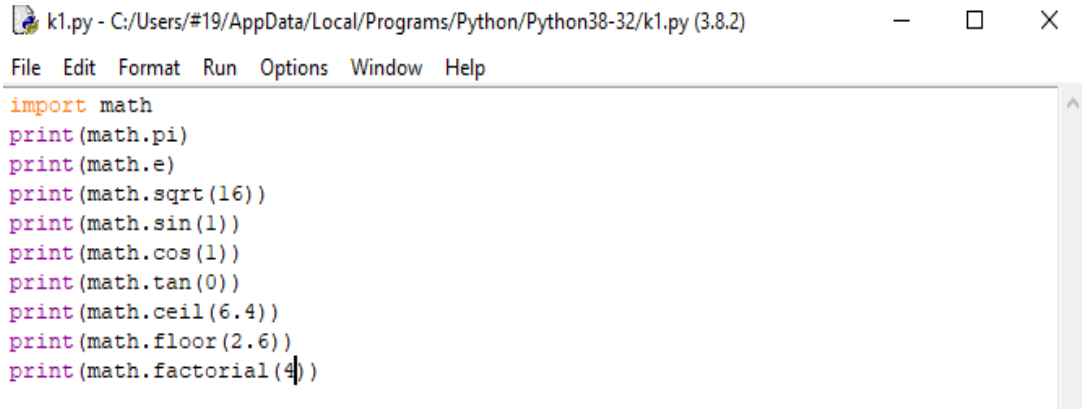
Мысалы, print((5 + 5) * 2), print(3 + (4 * 5)), print(10 / (6 - 1)). Осыған қоса, біз сандық мән берілген айнымалылармен математикалық операциялардыорындай аламыз.

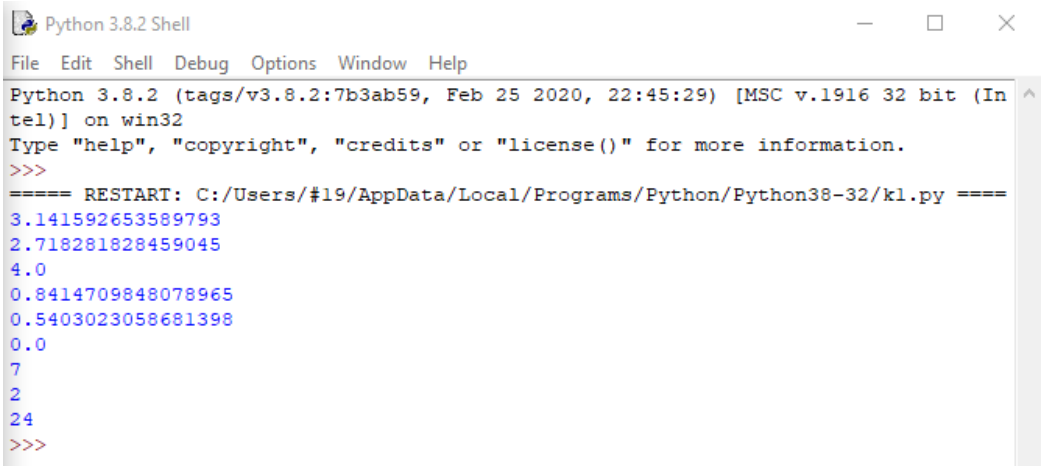
Python-да математикалық модуль **“math”** деп аталады және оны программаға қосу үшін, **import math к**омандасын теру қажет.

Арифметикалық амалдармен функциялар	Сипаттама
x + y	қосу
x - y	азайту
x * y	көбейту
x / y	бөлу
x // y	бүтін сандық бөлу
x % y	бөлінді қалдығы
x ** y	дәрежеге шығару
-x	таңба ауыстыру
abs(x)	санның абсолюттік шамасы (модулі)
divmod(x, y)	x-ті y-ке жеке бөлінудің қалдығы
pow(x, y)	санды дәрежеге шығару
math.ceil(X)	ең жақын үлкен санға дейін дөңгелектеу
round(x, y)	санды дөңгелектеу, y-үтірден кейінгі таңбалар саны

math – математикалық функциялар модулі	
math.fabs(x)	модуль x
math.log(x, [негізі])	*xсанының көрсетілген негіз бойынша логарифмі*, егер негізі жазылмаса натурал логарифм есептелінеді
math.log10(x)	x аргументінінің ондық логарифмі
math.pow(x, y)	x^y
math.sqrt(x)	x түбірі
math.acos(x)	arccos x (рад)
math.asin(x)	arcsin x
math.atan(x)	arctg x
math.cos(x)	cosx (x рад)
math.sin(x)	sinx
math.tan(x)	tgx
math.exp(x)	e^x
math.pi	pi = 3,1415926...
math.e	e = 2,71828
math.ceil(x)	ең жақын үлкен санға дейін дөңгелектеу
math.floor(x)	ең жақын кіші санға дейін дөңгелектеу
math.factorial(x)	санның факториалын анықтау
math.trunc(x)	санның бөлшек бөлігін тастайды
math.modf(x)	бөлшек және бүтін бөліктерді анықтайды
math.fmodf(x,y)	x санын y-ке бөлгендегі қалдық

Модуль бойынша кейбір мысалдарды қарастырайық:





Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Айнымалы дегеніміз не?
2. Python тілінде айнымалыны қалай жариялайды?
3. Айнымалы атын жазуда қандай ережелер бар?
4. Python-дағы негізгі мәліметтер типтерін атаңыз.
5. `list` пен `tuple` айырмашылығы неде?
6. `set` мәліметтер типінің ерекшелігі қандай?
7. Сөздік (`dict`) құрылымы қалай жасалады?
8. Айнымалы мәнін қалай қайта тағайындауға болады?
9. `bool` типінің мүмкін мәндері қандай?
10. Комплекс сандар Python-да қалай жазылады?
11. Енгізілген деректі бүтін санға (`int`) немесе нақты санға (`float`) қалай түрлендіруге болады?
12. `print()` функциясының негізгі қызметі қандай?
13. `print()` функциясында бірнеше мәнді бір уақытта қалай шығарамыз?
14. `print()` функциясында `sep` параметрі не үшін қолданылады?
15. `print()` функциясында `end` параметрі не үшін қажет?
16. `print()` арқылы мәтін мен айнымалыны бірге қалай шығара

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с.

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс №4

Дәріс тақырыбы:Жолдар және оларға қолданылатын амалдар.

Дәріс мақсаты: Python-да мәтінмен жұмыс істеу үшін деректердің арнайы **str** жол түрін меңгеру.

Кілт сөздер: `str` – жол типі,`len()` – жол ұзындығын табу,`+` – жолдарды біріктіру (конкатенация),`*` – жолды бірнеше рет қайталау,`in, not in` – жол ішінде символдың бар-жоғын тексеру,`[]` – индекс арқылы жол элементіне қатынау,`[:]` – жолды кесіп алу (слайс),Жол әдістері (methods),`.upper()` – барлық әріптерді бас әріпке айналдырады,`.lower()` – барлық әріптерді кіші әріпке айналдырады,`.capitalize()` – бірінші әріпті бас әріпке айналдырады. `title()` – әр сөздің бірінші әрпін бас әріп қылады,`strip()` – артық бос орындарды алып тастайды

Дәріс жоспары:

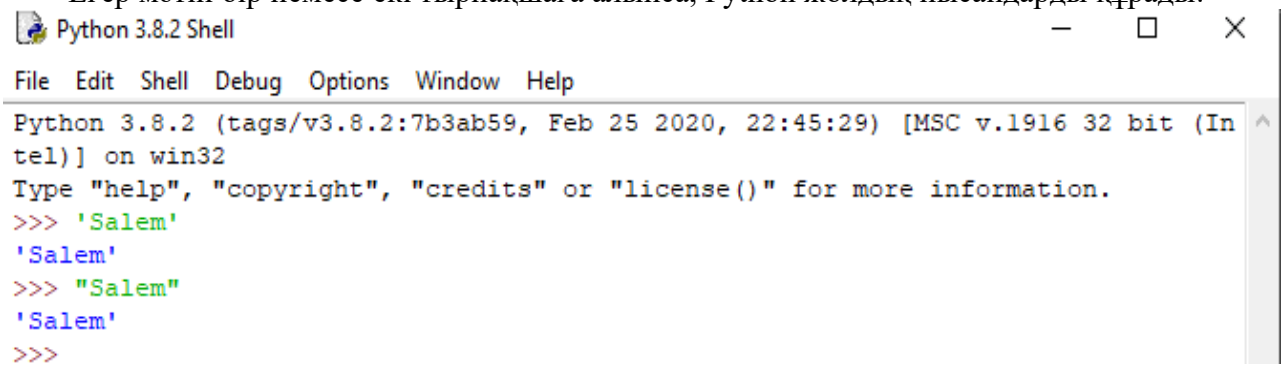
1. Жолдар және оларға қолданылатын амалдар
2. Жолдарды форматтау әдісі
3. Сандарды форматтап шығару

1. **Жолдар және оларға қолданылатын амалдар**

Python мәтіндерді өңдеу, мәтінді іздеу, мәтіннің жеке бөліктерін ауыстыру және т.б. үшін жиі қолданылады. Python-да мәтінмен жұмыс істеу үшін деректердің арнайы **str** жол түрі бар.

Жол-бұл таңбалар тізбегі болып табылатын күрделі мәліметтер түрі.

Егер мәтін бір немесе екі тырнақшаға алынса, Python жолдық нысандарды құрады:



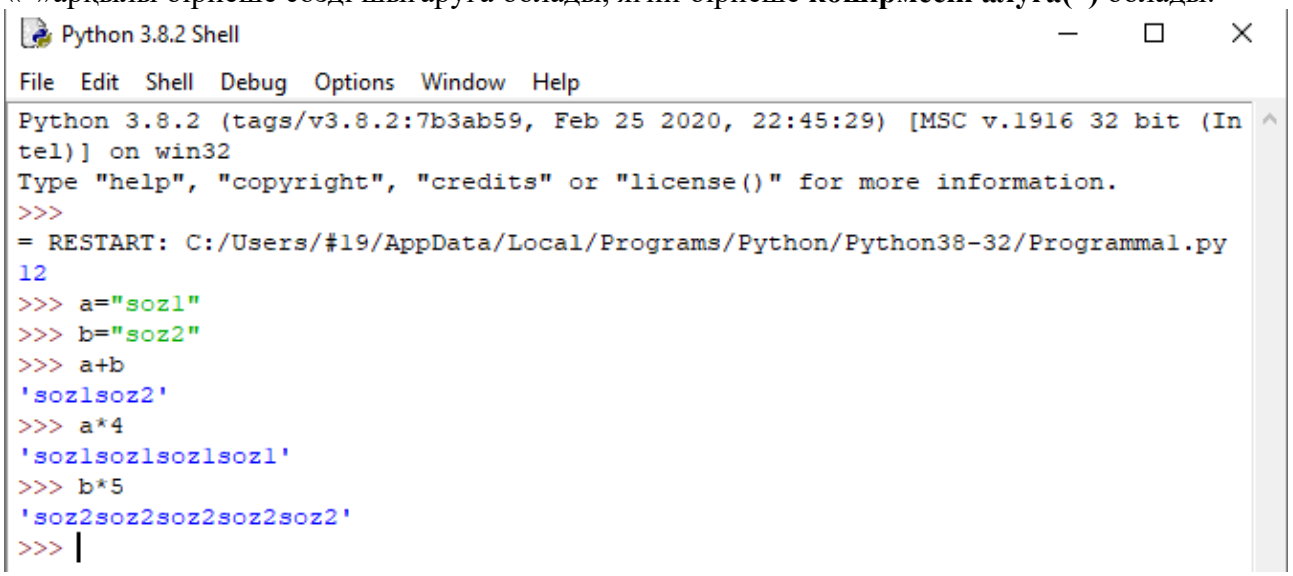
Тырнақшасыз жазылса Python мәтінді айнымалы деп санайды және оның мазмұнын экранға шығаруға тырысады.

Python-да жолдармен жұмыс істеу үшін көптеген кіріктірілген функциялар бар, мысалы, len. Бұл функция оған аргумент ретінде берілетін жолдың ұзындығын анықтайды:

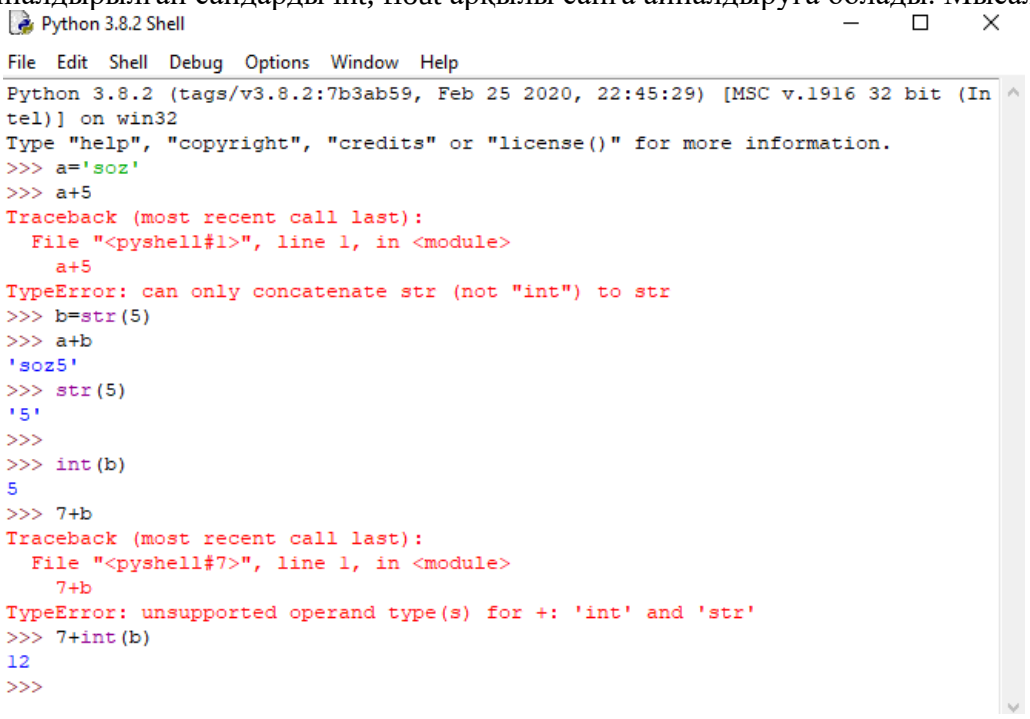
Мәтінді айнымалыларды бір-біріне қоса аламыз. Бұл түсінік – **конкатенация(+)** деп

```
>>> len('Salem!')
6
```

аталады. Төмендегі мысалға назар аударсақ «+» арқылы сөздерді біріктіруге болады. «*»арқылы бірнеше сөзді шығаруға болады, яғни бірнеше **көшірмесін алуға(*)** болады.



str функциясының көмегімен санды мәтінді жолға айналдыруға болады немесе мәтін типіне айналдырылған сандарды int, float арқылы санға айналдыруға болады. Мысалы:



Мысалда жол мен санды қосу амалы орындалмайды, қате шығады, себебі бірдей типті емес, сондықтан бірдей типке айналдырып алып, амалды орындау керек.

Тізбекте символдардың реті маңызды, жолдағы әрбір таңбаның ерекшереттік нөмірі - индекс болады. Жолдағы белгілі бір таңбаға сілтеме жасап және оны индекстеу операторының көмегімен шығарып алуға болады, бұл жерде таңбаның реттік номері тік жақша ішінде көрсетіледі. Тізбектегі таңбалардың реттік нөмірі 0-ден бастап саналады, бірінші элементті шығару үшін [0] деп жазу керек. Сонымен бірге таңбалардың реттік нөмірін соңынан бастап анықтауға да болады. Бұл кезде санау -1-ден(ең соңғы таңбадан) басталады.

Мысалы:

```
>>> Apta='duisenbi, seisenbi, sarsenbi'[1]
>>> 'duisenbi, seisenbi, sarsenbi'[1]
'u'
>>> 'duisenbi, seisenbi, sarsenbi'[3]
's'
>>> 'duisenbi, seisenbi, sarsenbi'[7]
'i'
>>> Apta='duisenbi, seisenbi, sarsenbi'
>>> Apta[3]
's'
>>> Apta[8]
','
>>> Apta[10]
's'

>>> Apta[-1]
'i'
>>> Apta[-3]
'n'
```

Жолдың өзімен емес, оларға сілтеме жасайтын айнымалылармен жұмыс істеу ыңғайлы. Индекстеу өрнегінің нәтижесін басқа айнымалыға тағайындауға болады. Жоғарыда келтірілген мысалда жолды Apta айнымалысына меншіктедік.

Жолдан бір таңбаны емес, бірнеше таңбаны яғни бөлігін шығарып алуға болады, Жолдың бөлігін алу операторы келесідей көрінеді: [X: Y). X - бөліктің басталу индексі, ал Y – соңы, бұл жерде Y саны бөлікке енбейді. Егер бірінші индекс жоқ болса, онда бөлік жол басынан

```
>>> Apta='duisenbi, seisenbi, sarsenbi'
>>> Apta[0:7]
'duisenb'
>>> Apta='duisenbi, seisenbi, sarsenbi'
>>> Apta[0:8]
'duisenbi'
>>> Apta[:16]
'duisenbi, seisen'
>>> Apta[20:]
'sarsenbi'
```

екінші индекске дейін алынады; егерекінші индекс болмаса, онда бөлік бірінші индекстен бастап жолдың соңына дейін алынады. Мысалы:

Сонымен қатар, таңбаларды қатармен емес, белгілі бір таңбалар санынан кейін шығаруға болады. Бұл жағдайда индекстеу операторы келесідей көрінеді: [X: Y: Z); мұндағы Z - элементтер таңдалатын қадам. Мысалы:

```
>>> Apta='duisenbi, seisenbi, sarsenbi'
>>> len(Apta)
28
>>> Apta[::5]
'dnsnsn'
>>> Apta[0:18:2]
'dieb,sieb'
```

Len(Apta) функциясы арқылы жолдың ұзындығы анықталған. Apta[::5] мысалында жолдың таңбаларын 5-қадам сайын шығарады, Apta[0:18:2] бұл жерде 1-ші таңбадан 18-ші таңбаға дейін 2 қадаммен шығарады.

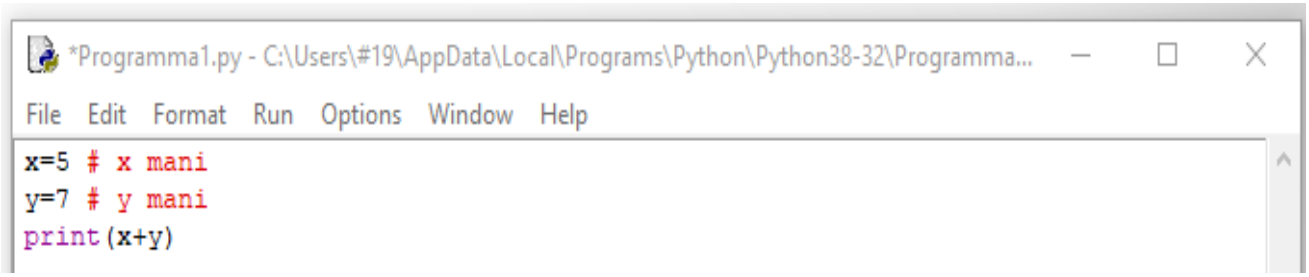
Комментарий(түсіндірме)

Программалауда жазылған код жайында түсініктемелер жазу өте маңызды.Себебі жұмыс кезінде басқа программистер сіздің кодыңыздың мағынасын түсіну керек. Яғни түсініктемелер арқылы сіз кез келген заттарды жаза аласыз және олар кодқа әсер етпейді.

Python-да бір жолды түсініктемені құру үшін (тек бір жолға жазылатын қысқа

түсініктемелер үшін), # символын қолдану қажет. Комментарийлер бірнеше жолдан тұрса әрбір жолдың алдына # белгісі қойылады.

Мысалы:



Тағы біз көп жолды түсініктемелерді құра аламыз. Егер сіз ұзақ түсініктеме жазғыңыз келсе, бұл түрді қолдану керек. Көп жолды түсініктеме құру үшін, сіз алты тырнақша (жұп немесе дара) қолдану керексіз. Мысалы:

```
'''
Salem!
Kalyn kalai?
Programma  kiyn ba?
'''
```

2. Жолдарды форматтау әдісі

Жолдарды форматтау-бұл жолдарды сізге қажет формаға келтіру. Осындайқажеттіліктерді қанағаттандыру үшін бізге **format ()** әдісі қажет.

Format () әдісі аргументтердің кез келген санын қабылдайды және олардыаталған жолдағы көрсетілген орындарға ауыстырады:

```
>>> s='lll_{0}_333_{1}_555_{2}_777
>>> s.format('222','444','666')
'lll_222_333_444_555_666_777'
```

Осы мысал қалай жұмыс істейтінін қарастырайық:

- s шаблон ретінде жұмыс істейді;
- фигуралық жақша, біз сізге кірістіру керек жерлерді белгіледік;
- фигуралық жақшалардағы сандар format() әдісінің позициялық аргументтерінің индекстері болып табылады.

Осы мысалды format() әдісінің позициялық аргументтерінің индекстеріналмастырып, қайта қарастырайық:

```
>>> s='lll_{0}_333_{0}_555_{0}_777'
>>> s.format('222','444','666')
'lll_222_333_222_555_222_777'
```

```
>>> s='lll_{2}_333_{1}_555_{0}_777'
>>> s.format('222','444','666')
'lll_666_333_444_555_222_777'
```

Фигуралық жақшалардағы format() әдісінің позициялық аргументтеріниңиндекстерінің өзгеруіне байланысты нәтижелер алатындығымызды көруге болады.

Позициялық аргументтерден басқа, бұл әдіс атаулы аргументтерді қабылдай алады:

Жалпы жағдайда аргументтер *атаулы* және *позициялық* болуы мүмкін:

```
>>> s = '{a}____{b}_____{c}'
>>> s.format(a = ':Python', b= '1234', c = 'Programma   ')
':Python____1234_____Programma   '
>>> s = '{c}____{a}_____{b}'
>>> s.format(a = ':Python', b= '1234', c = 'Programma   ')
'Programma   ____:Python_____1234'
>>> s = '{x}; {0}; {y}; {1}'
>>> s.format('A', 'B', x = 1, y = 2)
'1; A; 2; B'
```

Мұнда екі мәселеге назар аудару керек: біріншісі – позициялық аргументтер, екіншісі-аргументтер кез-келген түрдегі мәліметтер болуы мүмкін. Мысалы:

Сонымен бірге, біз тізімдер мен сөздіктерді алмастырып қана қоймай, оларда орналасқан

```
>>> s = '{int}; {float}; {complex}'
>>> s.format(int = 2, float = 2e-5, complex = 2+0.2j)
'2; 2e-05; (2+0.2j)'
>>> a = list(range(3))
>>> b = dict([[1,'a'],[2,'b']])
>>> c = set('aabbcc')
>>> s = '{list}; {dict}; {set}'
>>> s.format(list = a, dict = b, set = c)
"[0, 1, 2]; {1: 'a', 2: 'b'}; {'c', 'a', 'b'}"
```

элементтерге де қол жеткізе аламыз. Мысалдар:

```
>>> s = 'sym: {0}; num: {1}'
>>> s.format('XYZ', [34, 45, 56])
'sym: XYZ; num: [34, 45, 56]'
>>> s = 'sym: {0[2]}; num: {1[1]}'
>>> s.format('XYZ', [34, 45, 56])
'sym: Z; num: 45'
```

```
>>> s = 'sym: {0[0]}; num: {1[2]}'
>>> s.format('XYZ', [34, 45, 56])
'sym: X; num: 56'
```

3.Сандарды форматтап шығару

Форматсыз шығару:

```
a = 5450000.0
print(a)
b = a/12.0
print(b)
```

Нәтижесі:

```
5450000.0
454166.6666666667
```

Форматпен шығару, '2f' – формат спецификаторы : бөлшегі 2 цифр болады:

```
a = 54321.6789
print(format(a,'.2f'))
print(format(a,'.1f'))
```

Нәтижесі:

```
54321.68
54321.7
```

Форматпен шығару, 'e' – формат спецификаторы : экспоненциалды сан.

```
>>> print(format(54321.6789,'e'))
5.432168e+04
>>> print(format(54321.6789,'.2E'))
5.43E+04
```

Форматпен валютаны шығару, бүтін бөліктің әрбір 3 цифрын үтірменбөліп жазу:

```
at = 3500.0   # айлық табыс
jt = at*12    # жылдық табыс
print('Сіздің жылдық табысыңыз $',format(jt,',.2f'),sep='')
```

Нәтиже:

```
Сіздің жылдық табысыңыз $42,000.00
```

Форматпен шығару, '10.2f' – формат спецификаторы : ені – 10, бөлшегі 2цифр болады:

```
>>> a = 54321.6789
>>> print('a санының мәні = ', format(a,'10.2f'))
a санының мәні =    54321.68
```

Формат арқылы бүтін санның енін (14) беріп, әрбір 3 цифрды үтірменбөліп жазу:
Сан өрісі енін көрсету:

```
>>> print(format(123456789,'14,d'))
   123,456,789
```

```
>>> print('Сан = ', format(25345.6789,'12,.2f'))
Сан =     25,345.68
>>> print('Сан = ', format(25345.6789,'12.2f'))
Сан =      25345.68
```

Бүтін сан енін көрсету:

• format функциясын бүтін сандарға да қолдана аламыз. Мұнда екі түрлімүмкіндік бар:

• тип көрсеткіші ретінде d символы қолданылады:

• дәлдік көрсетілмейді.

Төменде 123456 саны форматсыз шығарылған:

```
>>> print(format(123456, 'd'))
123456
```

Келесі жолда 123456 үтір арқылы ажыратылып жазылады:Төменде

```
>>> print(format(123456, ',d'))
123,456
```

санның ені 10 орынмен берілген:

>>> print(format(123456, '10,d')) Нәтижесі:123,456

```
>>> print(format(123456, '10,d'))
   123,456
```

Пайыз (%) таңбасын шығару

Пайыз таңбасын шығару үшін санның типін көрсететін f символы орнына % таңбасын қою керек:

```
>>> print(format(0.5,'%'))
50.000000%
```

Егер бүтін сан түрінде бергіміз келсе, дәлдігін 0 етіп береміз:

```
>>> print(format(0.5,'.0%'))
50%
```

Python тілінде мәтінді жолдар мен символдар дара немесе қос тырнақшалар арқылы жазылады. Ал сандарды шығару үшін, print() функциясында тырнақшаларды қолданбай, сандарды теру қажет. Мысалы, print(150) командасын терсек, экранға 150 саны шығарылады. Одан басқа, print() функциясы арқылы арифметикалық операцияларды жүзеге асыруға болады.

print() функциясының қосымша мүмкіндіктеріне көшейік. Ақпаратты тасымалдаудың тағы екі әдісін қарастырайық:

```
>>> x = 'Amir'
>>> y = 'zasy'
>>> z = 2
>>> print(x, y, z, sep='-')
Amir-zasy-2
```

“sep=” командасы арқылы біз әр сөзді немесе санды әртүрлі символдарменбөле аламыз. Жоғарыда келтірілген мысалда біз айнымалылар мәндерін бір- бірінен дефис арқылы бөлектедік. Бұл командаға қатысты тағы мысал қарастырайық:

```
>>> print(x, y, z, sep="\n")
Amir
zasy
2
```

Бұл мысалда біз **“\n”** командасын қолдандық. Бұл команда әр мәнді жаңа жолға тасымалдайды, яғни әр мән жаңа жолдан шығарылады.

Тағы Python-да **“\t”** (tab) командасы бар. Бұл команда әр мәнді бір-бірінен үлкен бос орындармен бөледі.

Енді, шығаруды тоқтату әдісін қарастырайық. Python-да **“end=”** командасы арқылы әртүрлі символдарды нәтиже ретінде шығаруға болады. Мысалы, print(“Salem”, ‘.’) командасында нүкте жол ретінде берілген. Ал біз print(“Salem”, end=’.’) деп жазсақ, нүкте нәтиже ретінде шығады. Тағы мысал қарастырайық:

>>> print("Ne zanalyK?", end=' ')

Ne zanalyK? - сөйлемнің соңында бос орын шығады, бірақ ол көрінбейді.

>>> print("Amir", end="\n\n\n\n") Бұл мысалда “Amir” атынан кейін төрт жаңа жол шығарылады.

```
>>> print('Salem', end='.')
Salem.
>>> print("Ne zanalyk?", end=' ')
Ne zanalyk?
>>> print("Amir", end="\n\n\n\n")
Amir


>>>
```

Бұл мысалда біз **“\n”** командасын қолдандық. Бұл команда әр мәнді жаңажолға шығарады.

```
>>> print("Amir", end="\nZaraisyn\nDurys\nOte zaksy\n5")
Amir
Zaraisyn
Durys
Ote zaksy
5
>>>
```

Практикалық жұмыс

№1. Тіктөртбұрыштың ұзындығы мен ені берілген. Периметрі мен ауданын табу программасын жаз.

Программасы:

```
a=int(input("a="))
b=int(input("b="))
s=a*b
p=2*(a+b)
print("s=",s,"p=",p)
```

Нәтижесі:

```
a=5
b=9
s= 45 p= 28
```

№2. Герон формуласы бойынша үшбұрыштың ауданын табу программасын жаз.

1-әдіс. Математикалық модульді қолданамыз. Ол үшін: Import math командасын пайдаланамыз.

Программасы:

```
import math # математикалық модуль қосамыз
a=int(input("a="))
b=int(input("b="))
c=int(input("c="))
p=(a+b+c)/2
s=math.sqrt(p*(p-a)*(p-b)*(p-c))
print("s=",s)
```

Нәтижесі:

```
a=8
b=9
c=10
s= 34.197039345533994
```

2-әдіс. Pow() функциясын қолдануға болады. Квадрат түбірді ½ дәреже деп қарастырамыз. Мысалы:Pow(4,2) –нің мәні 16.

Программасы:

```
import math #Математикалық модуль қосамыз
a=int(input("a="))
b=int(input("b="))
c=int(input("c="))
p=(a+b+c)/2
s=pow((p*(p-a)*(p-b)*(p-c)),1/2)
print("s=",s)
```

Нәтижесі:

```
a=5
b=6
c=7
s= 14.696938456699069
```

3-әдіс. Қарапайым дәрежелеуді қолдана қойсақта болады. Квадрат түбірді ½ немес 0.5 дәрежеге шығаруға болады. Python-да дәрежелеу үшін ** қолданылады, мысалы 3**2 –нің нәтижесі 9.

Программасы:

```
a=int(input("a="))
b=int(input("b="))
c=int(input("c="))
p=(a+b+c)/2
s=((p*(p-a)*(p-b)*(p-c))**0.5)
print("s=",s)
```

Нәтижесі:

```
a=5
b=6
c=7
s= 14.696938456699069
```

№3. Үшбұрыштың катеттері белгілі, гипотенузаның ұзындығын табу программасын жаз.

Катет және гипотенуза-бұл тікбұрышты үшбұрыштың қабырғалары. Катеттердің ұзындығы белгілі болса, гипотенуза ұзындығы Пифагор теоремасы бойынша табылады: $a^2+в^2=c^2$.

Программасы:

```
import math
a=float(input('a='))
b=float(input('b='))
c=math.sqrt(a**2+b**2)
print("c=%.2f"%c)
```

Нәтижесі:

```
a=4
b=5
c=6.40
```

№4. Сандық айнымалылардың мәндерін алмастыру

Пайдаланушы екі санды енгізеді. Біреуі бір айнымалыға, екіншісі екіншісіне тағайындалады. Айнымалылардың мәндері бірінші мән екінші, ал екінші мәні бірінші болып өзгеруі керек.

Айнымалылардың мәндерін алмастырудың екі әдісі бар. Біріншісі кез- келген деректер типіне сәйкес келеді, ал екіншісі тек сандық деректер типіне қолданылады:

1-ші әдіс. "Буферлік" айнымалы арқылы,

2-ші әдіс. Арифметикалық амалдар арқылы.1-әдіс.

Программасы:

```
a = int(input())
b = int(input())
print('a=%d; b=%d' % (a,b))
c = a
a = b
b = c
print('a=%d; b=%d' % (a,b))
```

Нәтижесі:

```
a=7; b=8
a=8; b=7
```

2-әдіс. Программасы:

```
a = int(input())
b = int(input())
print('a=%d; b=%d' % (a,b))
a = a + b
b = a - b
a = a - b
print('a=%d; b=%d' % (a,b))
```

Нәтижесі:

```
5
6
a=5; b=6
a=6; b=5
```

3-әдіс. Python үшін мәселені шешудің үшінші әдісі бар - кортеж арқылы. Мысалы, a, b = b, a. Осы әрекеттің нәтижесінде b мәні a-мен, a мәні b-мен байланысты болады.

Программасы:

```
a = int(input())
b = int(input())
print('a=%d; b=%d' % (a,b))
a,b = b,a
print('a=%d; b=%d' % (a,b))
```

Нәтижесі:

```
6
9
a=6; b=9
a=9; b=6
```

№5. Шеңбердің периметрі мен ауданын табу.
Шеңбердің периметрі P = 2πr, ауданы: S = πr^2.
Программасы:

```
import math
print("Шеңбердің радиусын енгіз: ")
r = int(input("r="))
print("P=%.2f; S=%.2f" % (2*math.pi*r, math.pi*r**2))
```

Нәтижесі:

```
Шеңбердің радиусын енгіз:
r=5
P=31.42; S=78.54
```

№6. Үш таңбалы санның цифрларының қосындысы мен көбейтіндісінтабу.
Программасы:

```
n = input("Үш таңбалы санды енгіз: ")
n = int(n)
d1 = n % 10
d2 = n % 100 // 10
d3 = n // 100
print("Санның цифрларының қосындысы:", d1 + d2 + d3)
print("Санның цифрларының көбейтіндісі:", d1 * d2 * d3)
```

Нәтижесі:

```
Үш таңбалы санды енгіз: 258
Санның цифрларының қосындысы: 15
Санның цифрларының көбейтіндісі: 80
```

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. .upper() – барлық әріптерді бас әріпке айналдырады
2. .lower() – барлық әріптерді кіші әріпке айналдырады
3. .capitalize() – бірінші әріпті бас әріпке айналдырады
4. .title() – әр сөздің бірінші әрпін бас әріп қылады
5. .strip() – артық бос орындарды алып тастайды
6. .replace(old, new) – символдарды/жолды ауыстыру
7. .find(sub) – ішкі жолдың индексін табу
8. .count(sub) – ішкі жолдың қайталану санын табу
9. .split() – жолды бөліктерге бөлу (тізім ретінде)
10. .join(list) – тізім элементтерін бір жолға біріктіру

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс № 5

Дәріс тақырыбы:Логикалық өрнектер. IF шартты операторы

Дәрістің мақсаты*: Python бағдарламалау* тілінде логикалық өрнектермен танысу. IF шартты операторының құрылымына мысалдар қарастыру.

Кілт сөздер: bool – логикалық мәліметтер типі,True, False – логикалық мәндер, Салыстыру операторлары, and – ЖӘНЕ, or – НЕМЕСЕ,not – ЕМЕС

Дәріс жоспары:

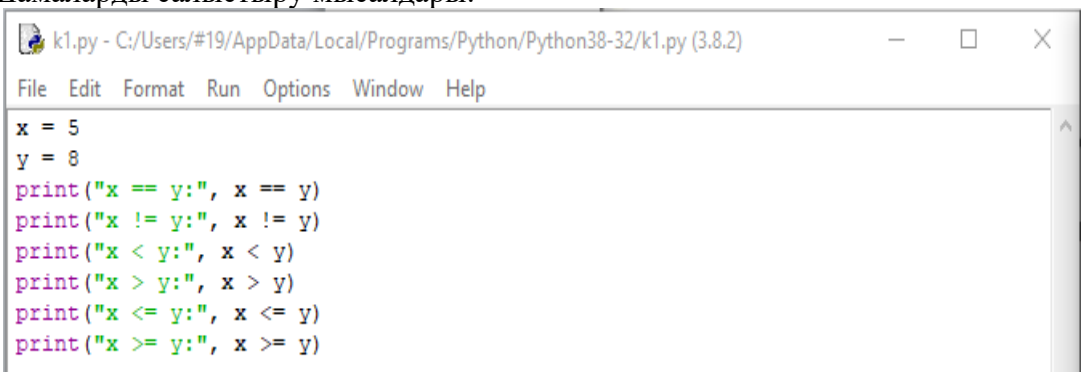
1. Логикалық операторлар
2. if шартты операторы

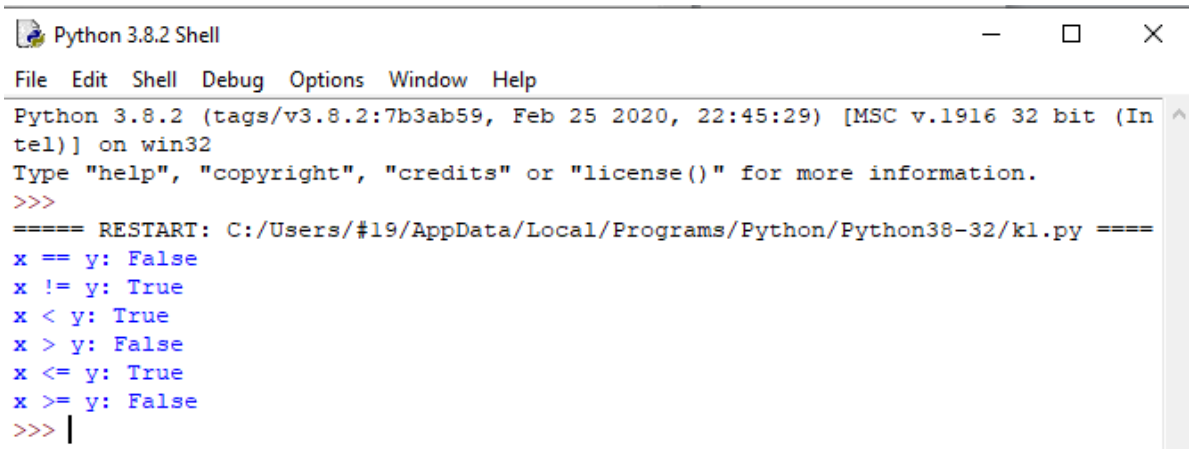
1. **Логикалық операторлар** арқылы біз айнымалыларды, сандарды, жолдарды салыстыра аламыз. Нәтижесі не ақиқат (True) не жалған (False) болады. Бұл мәндер салыстыру және логикалық амалдарының нәтижесі болады.

Салыстыру амалдары

амал	мағынасы
==	тең
!=	тең емес
<	үлкен
>	кіші
<=	үлкен немесе тең
>=	кіші немесе тең

Шамаларды салыстыру мысалдары:





Логикалық амалдар.

Python-да and, or, not үш логикалық оператор бар.

амал	мағынасы
and	Логикалық көбейту (және)
or	Логикалық қосу (немесе)
not	Логикалық теріске шығару (жоқ)

== амалы нәтижесінің кестесі

x	==	y	нәтиже
True	==	True	True

True	==	False	False
False	==	True	False
False	==	False	True

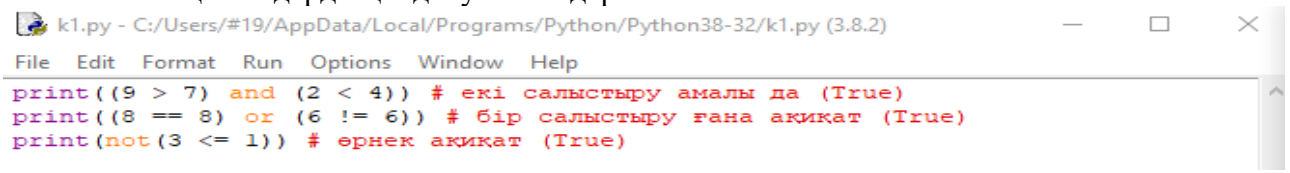
and амалы нәтижесінің кестесі

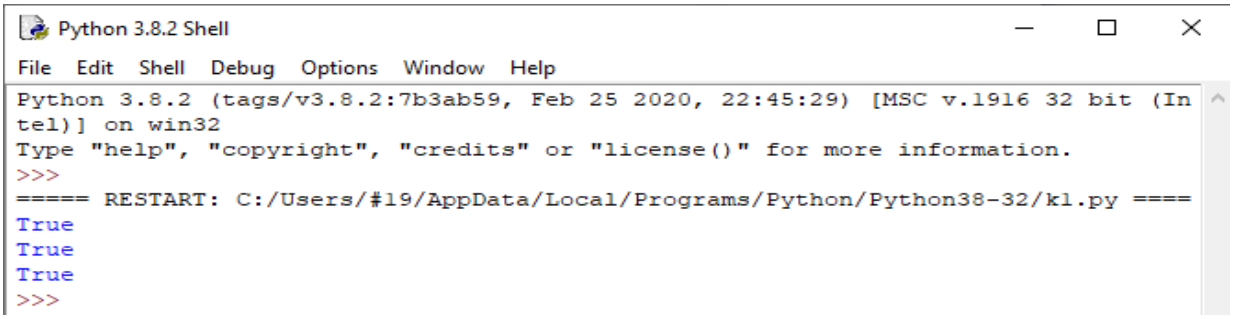
x	and	y	нәтиже
True	and	True	True
True	and	False	False
False	and	True	False
False	and	False	False

or амалы нәтижесінің кестесі

x	or	y	нәтиже
True	or	True	True
True	or	False	True
False	or	True	True
False	or	False	False

Логикалық амалдарды қолдану мысалдары:





2. if шартты операторы

Python-да тәуелді түрде программаның орындалуы үшін және белгілі біршарттарды орнату үшін **тармақталу** қолданылады.

Шартты оператордың жалпы жазылу түрі мынадай:if
логикалық_өрнек:
оператор
[elif логикалық_өрнек: Оператор][elif
логикалық_өрнек: Оператор]
.....
[else:
оператор]

Егер оператордың **if** немесе **elif** бөліктеріндегі шарт орындалса, онда сол бөліктегі оператор орындалады да, басқару келесі операторға беріледі. Ол бөліктен төмен орналасқан шарттар тексерілмейді. Егер **if** немесе **elif** бөліктеріндегі шарттардың ешқайсысы орындалмаса, онда соңғы **else** бөлігіндегі оператор орындалады. **if** операторының **elif** және **else** бөліктері болмауы да мүмкін, **elif** бөліктері бірнешеу болуы да мүмкін.

Практикалық жұмыс

№1. Берілген a, b – екі санның үлкенін табу программасын жаз.

Программасы:

```
a = int(input('a='))
b = int(input('b='))
max=a
if  max<b:
    max=b
print('max =', max)
```

Нәтижесі:

```
a=18
b=10
max = 18
```

№2. Қолыңызда бар ақшамен заттың құнына қарай зат алу программасынПрограммасы:

```
tovar1=int(input("1-зат куны="))
tovar2=int(input("2-зат куны="))
aksha=int(input("Мендеги акша="))
if tovar1+ tovar2 > aksha :
    print("акша жетпейді")
else:
    print("толенді")
```

Нәтижесі:

```
1-зат куны=2000
2-зат куны=3000
Мендеги акша=8000
толенді
```

№3. Координатасы (x,y) нүкте центрі координата басында жататынрадиусы r шеңберде жата ма, жатпай ма анықтайтын программа құр.

Егер координаттық жазықтықта нүктені таңдасаңыз, онда оның координаттарының проекциялары X және y осьтеріндегі тікбұрышты үшбұрыштың катеттері болып табылатынын көруге болады. Ал бұл тікбұрышты үшбұрыштың гипотенузасы координаттың басынан нүктеге дейінгі қашықтықты көрсетеді. Осылайша, егер гипотенузаның ұзындығы шеңбердің

радиусынан аз болса, онда нүкте шеңберге тиесілі болады; әйтпесе ол оның шегінен тыс болады.

Гипотенузаның ұзындығын Пифагор теоремасы бойынша есептеуге болады: гипотенузаның квадраты катеттерінің квадраттарының қосындысына тең.

Программасы:

```
import math

print("Нукте координаталары мен радиусты енгиз")
x = float(input("x = "))
y = float(input("y = "))
r = float(input("R = "))

gipotenuza = math.sqrt(x**2 + y**2)

if gipotenuza <= r:
        print("Нукте шенберде жатады")
else:
        print("Нукте шенберде жатпайды")
```

Нәтижесі:

```
Нукте координаталары мен радиусты енгиз
x = 5
y = 7
R = 8
Нукте шенберде жатпайды
```

№4. Берілген сан x>0 болса оң сан, болмаса теріс сан екенін хабарлайтынпрограмма жаз.

Программасы:

```
x=int(input("x="))
if x > 0 :
    print("он сан")
else:
    print("терiс сан")
```

Нәтижесі:

```
x=-4
терiс сан
```

№5. Үшбұрыштың үш қабырғасының мәніне қарай үшбұрыш бола ма, болмай ма анықтайтын программа жаз. «Үшбұрыштың екі қабырғасының қосындысы үшінші қабырғасынан артық болады» қасиетіне сүйенеміз.

Программасы:

```
print("ушбурыш кабырғаларын енгиз:")
a = float(input("a = "))
b = float(input("b = "))
c = float(input("c = "))

if a + b > c and a + c > b and b + c > a:
        print("Ушбурыш болады")
else:
        print("Ншбурыш болмайды")
```

Нәтижесі:

```
ушбурыш кабырғаларын енгиз:
a = 7
b = 8
c = 9
Ушбурыш болады
```

№6. Берілген a, b, c үш санның үлкенін табу программасын жаз.(Қабаттасқан шартты операторлар жазылады.)1-әдіс.

Программасы:

```
a = int(input('a='))
b = int(input('b='))
c = int(input('c='))

max = a
if max < b:
    max = b
if max < c:
    max = c

print('max=',max)
```

Нәтижесі:

```
a=8
b=9
c=10
max= 10
```

2-әдіс.

Программасы:

```
a = int(input('a='))
b = int(input('b='))
c = int(input('c='))

if a > b:
    if a > c:
        print('max=',a)
    else:
        print('max=',c)
else:
    if b > c:
        print('max=',b)
    else:
        print('max=',c)
```

Нәтижесі:

```
a=20
b=30
c=40
max= 40
```

3-әдіс.Логикалық and (және) амалын пайдалана отырып, бұл есепті мынадай түрде де жаза аламыз.

Программасы:

```
a = int(input('a='))
b = int(input('b='))
c = int(input('c='))

if (a>b) and (a>c): print('max =', a)

elif (b>c) and (b>a): print('max =', b)
else:

    print('max =', c)
```

Нәтижесі:

```
a=18
b=25
c=6
max = 25
```

№7. Үш санның ішінен ортаңғы мәнді табу(біреуінен үлкен, ал біреуіненкіші сан).

Программасы:

```
print('Үш санды енгіз: ')
a = int(input())
b = int(input())
c = int(input())

if b < a < c or c < a < b:
    print('Ортаңғы мән:', a)
elif a < b < c or c < b < a:
    print('Ортаңғы мән:', b)
else:
    print('Ортаңғы мән:', c)
```

Нәтижесі:

```
Үш санды енгіз:
5
9
7
Ортаңғы мән: 7
```

жаз.

№8. Енгізілген санның неше орынды сан екенін анықтайтын программаПрограммасы:

```
a=int(input("a="))
if a < 10 :
     print(" 1- орынды сан")
elif a < 100 :
     print(" 2- орынды сан")
elif a < 1000 :
     print(" 3- орынды сан")
elif a < 10000 :
     print(" 4- орынды сан")
else:
          print(" өте үлкен сан")
```

Нәтижесі:

4ас.

```
a=9
 1- орынды сан
>>>
===== RESTART:
a=45
 2- орынды сан
>>>
===== RESTART:
a=214
 3- орынды сан
>>>
===== RESTART:
a=2547
 4- орынды сан
>>>
===== RESTART:
a=456871
 өте үлкен сан
```

№9. Квадрат теңдеуді шешу программасын жаз. $ax^2 + bx + c = 0$, $D = b^2$ -Программасы:

```
import math

print("квадрат тендеудiй коэффициенттерiн енгiз:")
a = float(input("a = "))
b = float(input("b = "))
c = float(input("c = "))

discr = b ** 2 - 4 * a * c
print("Дискриминант D = %.2f" % discr)

if discr > 0:
    x1 = (-b + math.sqrt(discr)) / (2 * a)
    x2 = (-b - math.sqrt(discr)) / (2 * a)
    print("x1 = %.2f \nx2 = %.2f" % (x1, x2))
elif discr == 0:
    x = -b / (2 * a)
    print("x = %.2f" % x)
else:
    print("Тубiр жок")
```

Нәтижесі:

```
квадрат тендеудің коэффициенттерін енгіз:
a = 2
b = 10
c = 3
Дискриминант D = 76.00
x1 = -0.32
x2 = -4.68
```

№10. Берілген сан қандай сан: бір орынды немесе екі орынды, оң немесетеріс екендігін анықтайтын программа жаз

Программасы:

```
n = int(input("n = "))

if n == 0:
        print("Ноль - бір орынды сан")
else:
        if n > 0:
                print("он сан", end=' ')
        else:
                print("терiс сан", end=' ')
        if abs(n) < 10:
                print("бір орынды сан")
        elif 10 <= abs(n) < 100:
                print("екі орынды сан")
        else:
                print("уш не одан да коп орынды сан")
```

Нәтижесі:

```
n = -8
терiс сан бір орынды сан
>>>
= RESTART: C:/Users/ฐ19/AppData/Local/P
н.py
n = 226
он сан уш не одан да коп орынды сан
>>>
= RESTART: C:/Users/ฐ19/AppData/Local/P
н.py
n = -124
терiс сан уш не одан да коп орынды сан
```

№11. Координатасы берілген нүктенің қай ширекте жататынын анықтайтын программа құр.

1-әдіс.Программасы:

```
print("Нуктенин координатасын енгиз:")
x = float(input("x = "))
y = float(input("y = "))

if x > 0 and y > 0:
        print("Нукте I ширекте жатады")
elif x < 0 and y > 0:
        print("Нукте II ширекте жатады")
elif x < 0 and y < 0:
        print("Нукте III ширекте жатады")
elif x > 0 and y < 0:
        print("Нукте IV ширекте жатады")
elif x == 0 and y == 0:
        print("Нукте  координат басы болады")
elif x == 0:
        print("Нукте x осинде жатады")
elif y == 0:
        print("Нукте y осинде жатады")
```

Нәтижесі:

```
Нуктенин координатасын енгиз:
x = -6
y = -2
Нукте III ширекте жатады
>>>
= RESTART: C:/Users/#19/AppDa
y
Нуктенин координатасын енгиз:
x = 6
y = -7
Нукте IV ширекте жатады
>>>
= RESTART: C:/Users/#19/AppDa
y
Нуктенин координатасын енгиз:
x = 0
y = 0
Нукте  координат басы болады
```

2-әдіс. Программасы:

```
print("Нуктенин координатасын енгиз:")
x = float(input("x = "))
y = float(input("y = "))
if x == 0 and y == 0:
        print("Нукте  координат басы болады")
elif x == 0:
        print("Нукте x осинде жатады")
elif y == 0:
        print("Нукте y осинде жатады")
elif x > 0:
        if y > 0:
                print("Нукте I ширекте жатады")
        elif y < 0:
                print("Нукте IV ширекте жатады")
elif x < 0:
        if y > 0:
                print("Нукте II ширекте жатады")
        elif y < 0:
                print("Нукте III ширекте жатады")
```

Нәтижесі:

```
Нуктенин координатасын енгиз:
x = 5
y = 9
Нукте I ширекте жатады
```

№12. Функция мәнін тап.

, егер x>0
егер x=0
егер x<0

Программада '\ **n**' және '\ **t**' арнайы таңбалары қолданылады. Біріншісі жаңа жолға ауысуды білдіреді, ал екіншісі - табуляция (шегініс-отступ) және " \ " таңбасы программа жазуда ұзын жолдың жалғасын екінші жолға түсіруге мүмкіндік береді.

Программасы:

```
print("y = f(x):\n\t| y = x - 0.5, егер x > 0\n\t| \
y = 0, егер x = 0\n\t| y = |x|, егер x < 0")

x = float(input("x = "))

if x > 0:
        y = x - 0.5
elif x == 0:
        y = 0
else:
        y = abs(x)

print('y =', y)
```

Нәтижесі:

```
y = f(x):
        | y = x - 0.5, егер x > 0
        | y = 0, егер x = 0
        | y = |x|, егер x < 0
x = 8
y = 7.5
```

```
import math

print("квадрат тендеудің коэффициенттерін енгіз:")
a = float(input("a = "))
b = float(input("b = "))
c = float(input("c = "))

discr = b ** 2 - 4 * a * c
print("Дискриминант D = %.2f" % discr)

if discr > 0:
    x1 = (-b + math.sqrt(discr)) / (2 * a)
    x2 = (-b - math.sqrt(discr)) / (2 * a)
    print("x1 = %.2f \nx2 = %.2f" % (x1, x2))
elif discr == 0:
    x = -b / (2 * a)
    print("x = %.2f" % x)
else:
    print("Тубір жок")
```

№13. Функция мәнін тап.

$$y=\begin{cases} ax+\dfrac{b}{x+1} & егер\ \ x=3 \\ \arcsin\dfrac{1}{bx+2} & егер\ x=5 \\ e^{\sqrt{a|x|+b}} & егер\ x=0 \end{cases}$$

Программасы:

```
import math

a=float(input("a="))
b=float(input("b="))
x=float(input("x="))
if x==3:
     y=a*x+b/(x+1)
     print("y=",y)
elif x==3:
      y=math.asin(1/(b*x+2))
      print("y=",y)
elif x==0:
     y=math.exp(math.sqrt(a*math.fabs(x)+b))
     print("y=",y)

else:
            print("Жауабы жоқ")
```

Нәтижесі:

```
a=2
b=2
x=3
y= 6.5
```

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. `if` – шарт тексеру операторы
2. `elif` – қосымша шарт

3. `else` – шарт орындалмаса
4. Шартты блок (бірнеше команда орындалуы)
5. Ішкі шарт (шарт ішінде шарт – nested if)
6. Логикалық өрнектерді біріктіру (`if x > 0 and x < 10:`)
7. Шартты операторлардың орындалу реті

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс № 6

Дәріс тақырыбы: Қайталану алгоритмдері. While циклі. For циклі.

Дәріс мақсаты: Python-да қолданылатын циклдің түлерін қарастыру. for **және** while циклдарының мүмкіндіктерін қарастыру.Цикл түрлеріне есептер шығару.

Кілт сөздер: `or` ,`while` , `break` ,`continue` ,`else` ,Итерация ,Range (`range(start, stop, step)`)

Күрделі циклдар – цикл ішінде цикл (nested loop), Қайталау саны – цикл қанша рет орындалатынын анықтайтын мәнШарт (condition) – `while` циклының орындалу шарты

Дәріс жоспары:

1. Қайталану алгоритмдері
2. Евклид алгоритмі.
3. Қайталану операторы while. continue операторы, break операторы
4. Практикалық жұмыс

1. **Қайталану алгоритмдері.** Көптеген алгоритмдерде белгілі бір әрекеттер тізбегі бірнеше рет қайталанып орындалып отырады. Математикада, экономикада көптеген есептерді шығару кезеңінде де бір теңдеуді пайдаланып, ондағы айнымалының өзгеруіне байланысты оны бірнеше рет қайталап есептеуге тура келетін сәттер де жиі кездеседі. Осындай есептеу процесінің бөліктерінің қайталап орындалуын ***цикл*** деп атайды, ал қайталанатын бөлігі бар алгоритмдер тобы циклдік алгоритмдерге жатады. Қоршаған ортада көптеген процестер жүйелі түрде ретімен бірнеше рет қайталанып отырады. Мысалы , апта күндерінің ауысуы: дүйсенбі, сейсенбі, сәрсенбі, бейсенбі, жұма, сенбі, жексенбі немесе ауа-райының маусымдық ауысуы.

Қайталау санының алдын ала белгілі және белгісіз болуына байланысты цикл екі түрге бөлінеді. Қайталау санының алдын ала белгілі циклі **арифметикалық цикл** деп, орындалу саны белгісіз цикл **командалық(интерациялық) цикл** деп аталады. **Арифметикалық цикл** ең қарапайым цикл болып табылады және ол практикада жиі қолданылады.

Сонымен, Python-да циклдің екі түрі бар: **for** және **while** қолданылады. Яғни, сізге

командалар блогының неше рет қайталану қажеттілігі белгілі болса, “for” циклін, нақты саны белгілі емес болса, **“while” циклін** қолдануға болады.

2. Евклид алгоритмі.

Берілген ***m*** және ***n*** натурал сандарының (бүтін оң сандар) ең үлкен ортақ бөлгішін табу керек. Бұл алгоритм егер ***m > n*** болса, онда осы екі санның ең үлкен ортақ бөлгіші ***m-n*** және ***n*** сандарының ең үлкен ортақ бөлгішімен бірдей болады деген қасиетке сүйене отырып жазылған:

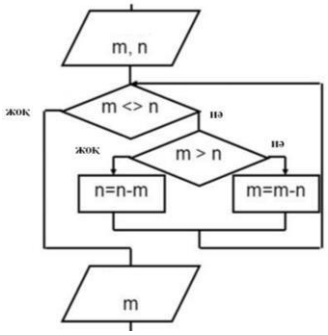
1) егер ***m=n*** болса, онда олардың кез келгенін ең үлкен ортақ бөлгіш ретінде қабылдап, алгоритмді атқаруды аяқтау керек, яғни 5-қадамға өту қажет, ал егер олар тең болмаса, онда 2-қадамға өтеміз;

2) сандардың үлкенін анықтаймыз;

3) оны үлкен сан мен кіші санның айырмасымен алмастырамыз;

4) табылған сандардың ең үлкен ортақ бөлгішін әрі қарай іздеу керек, яғни қайтадан 1-қадамға өтеміз;

5) есептің жауабын жазу, алгоритмнің соңы.



Алгоритмнің схемасынан көрініп тұрғандай, алгоритмнің бір бөлігі берілген сандарға байланысты бірнеше рет қайталануы мүмкін.

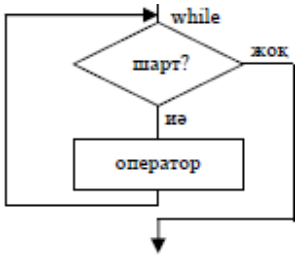
3. Қайталану операторы while.

Алдын ала берілген шарт бойынша қайталауды жүзеге асыратын **while** операторы.

While қайталану_шарты: оператор

Қайталану шартында жазылған логикалық өрнектің мәні **True** болған жағдайларда осы **оператор** орындалады. Егер логикалық өрнектің мәні **False** болғанда **while** операторының орындалуы тоқтатылады.

while операторының орындалу схемасы келесі суретте көрсетілген.



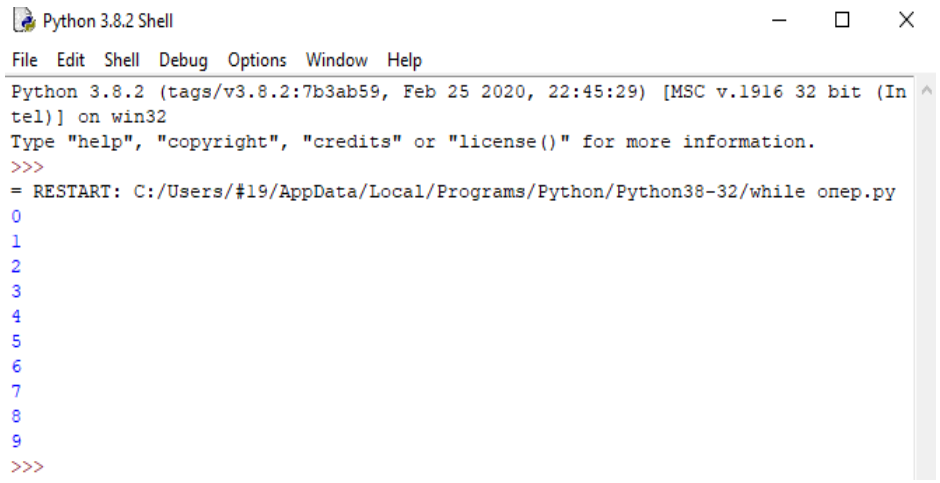
while- алдын ала берілген шарт бойынша қайталауды жүзеге асырады Мысалы: Сандарды тізбектей шығару десек, ол шексіз. Сондықтан шарт береміз. i<10 десек 10-ға дейінгі санды тізбектей шығарады. Санау 0-ден басталады, сондықтан нәтижеде 0-9 аралығындағы сандар тізбегі шығады, цикл 10 рет орындалады.

i<=10 десек 0-10аралығындағы сандар шығады, бұл жерде цикл 11 реторындалады т.с.с.

Программасы:

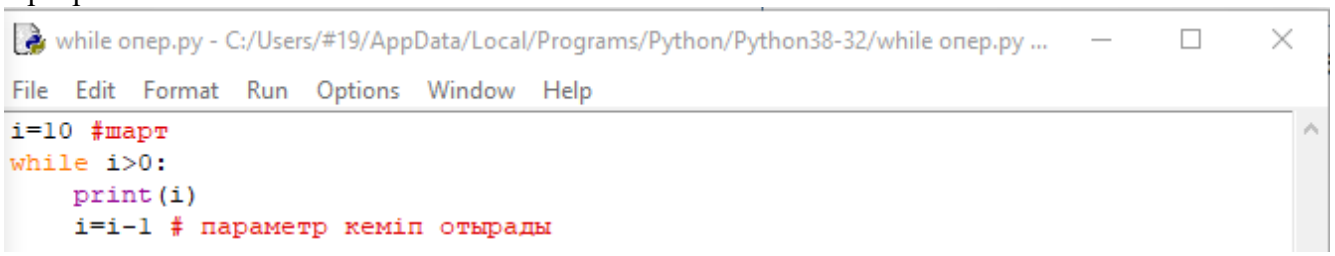


Нәтижесі:

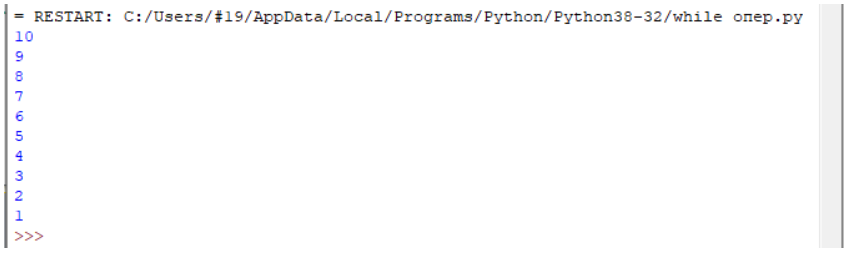


Мысалы: 10-ға дейінгі санды кері реттілікпен шығару керек болса, шартөзгереді және онда цикл параметрін 1-ге азайтып отырамыз.

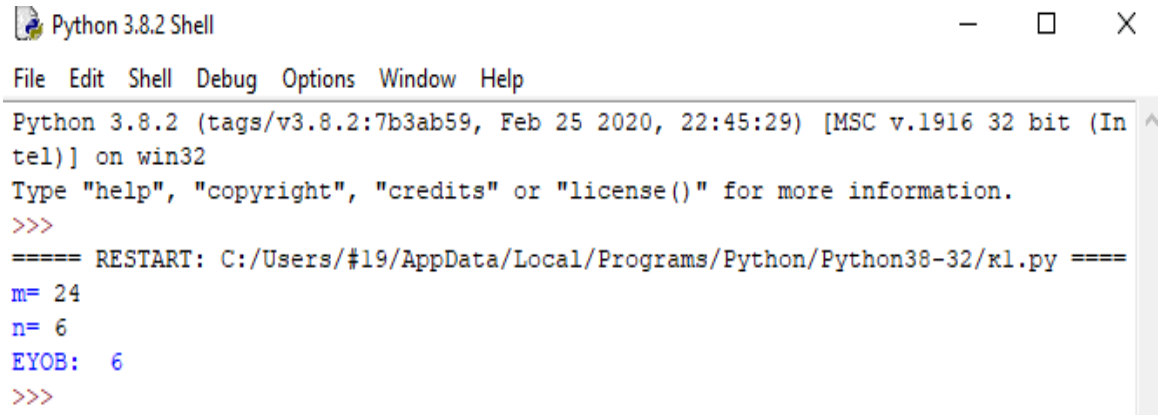
Программасы:

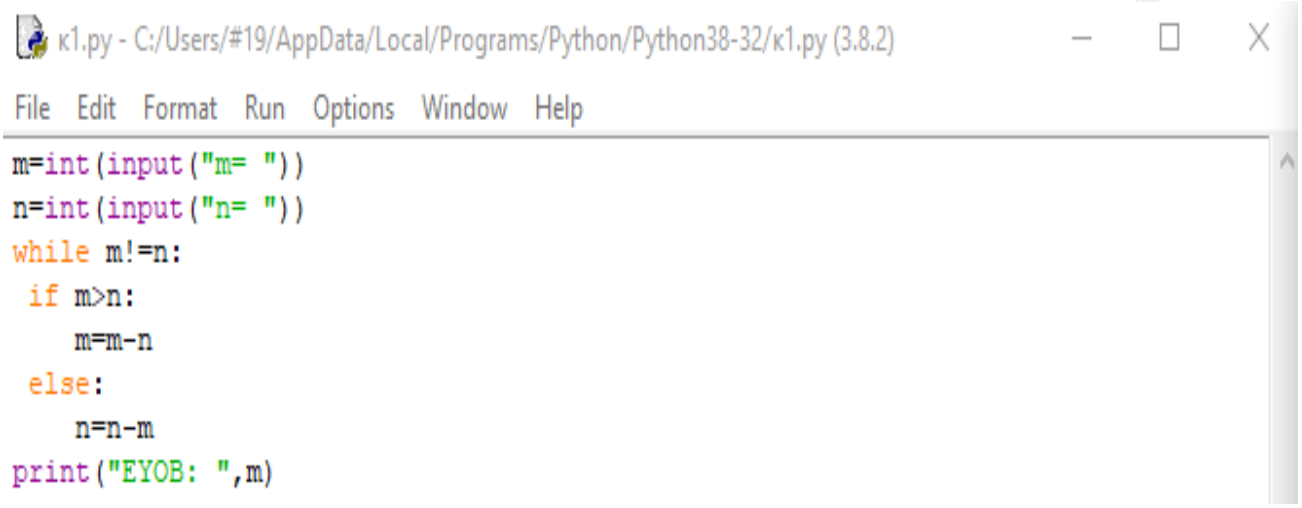


Нәтижесі:



Евклид алгоритмінің программасы:



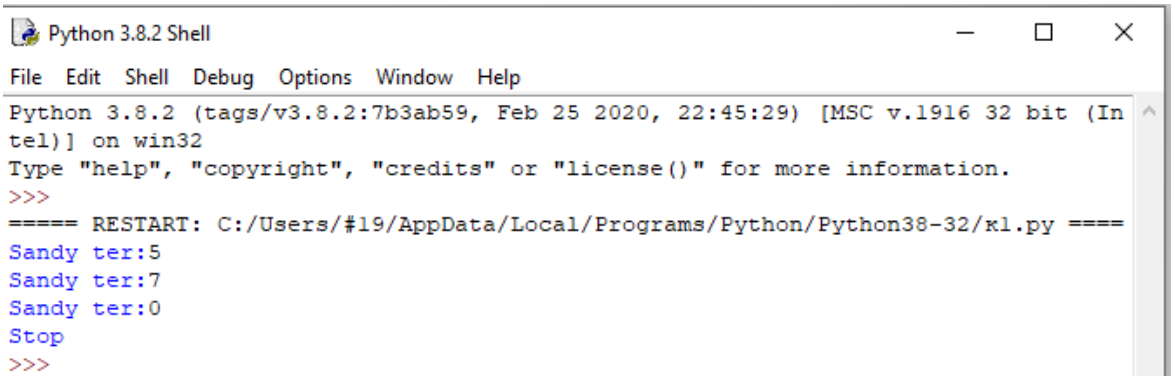


break операторы

Кей кезде цикл жұмысы кезінде белгілі бір жағдайларға байланысты онытоқтатуға тура келеді. Осындай жағдайда **break** операторы қолданылады.

Мысалы: Шексіз циклді қолданамыз. Цикл қанша сан терсеңіз де жүребереді, тек 0-тергенде тоқтайды.





4. continue операторы.

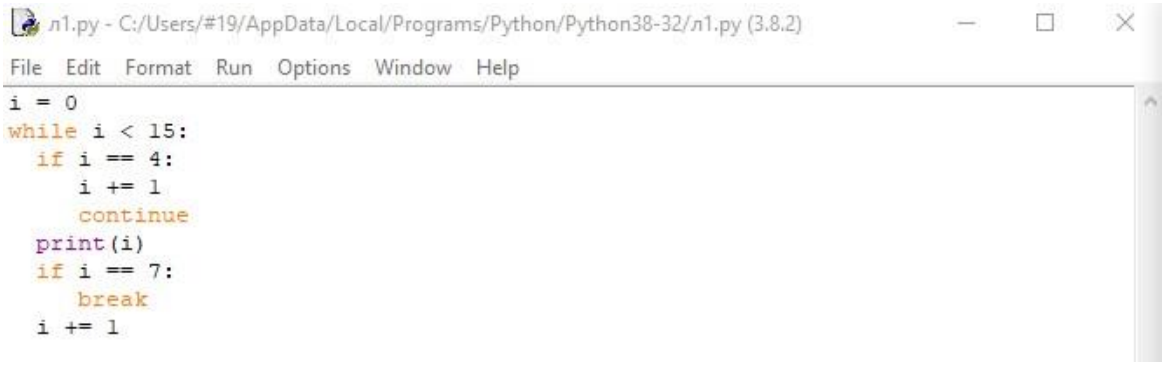
Continue-өзі тиесілі циклдың орындалып жатқан айналымын үзіп, келесі айналымына өткізіп жібереді, циклды тұтастай тоқтатпайды. Бұл оператордың қажеттілігі циклдың өзінен

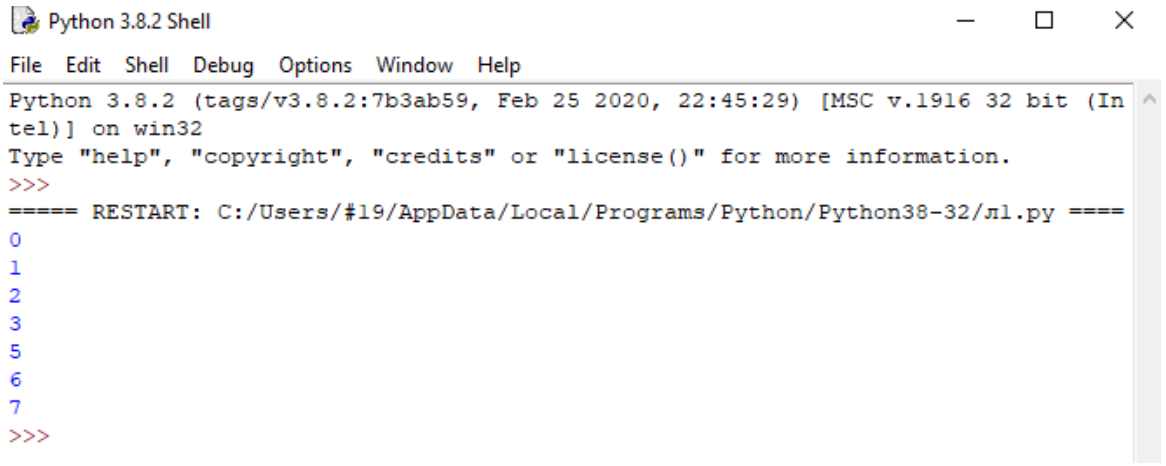
тұтас шығып кетпей, бір айналымын ғана өткізіп жіберу керек болған тұстарда туындайды.

continue қай тұста орындалса, сол тұстан бастап циклдың соңына дейінгі операторлар орындалмай, цикл қайтадан басынан басталады.

Оның жоғарыда айтып өткен **break** операторынан айырмашылығы - ол цикл жұмысын тоқтатпайды, тек келесі цикл жүрісіне - итерацияға ауыстырады.

Мысал:





4. Практикалық жұмыс

№1. Берілген сан негізінде өзгеше пирамида тұрғызу керек, яғни пирамиданың биіктігі енгізілген санға тең болады. Программасын құр.

Программасы:

```
n=int(input("Санды енгіз  "))
i=1
while i<=n:
    print(str(i)*i)
    i+=1  # i+=1 жазуы  i=i+1 жазуымен бірдей
```

Нәтижесі:

```
Санды енгіз  8
1
22
333
4444
55555
666666
7777777
88888888
```

№2. Берілген санға кері санды табатын программа жаз. Мысалы берілгенсан 4582 болса, нәтижесінде 2854 шығуы керек.

1-әдіс.Программасы:

```
n1 = int(input("Бутін санды енгіз: "))
n2 = 0

while n1 > 0:
        digit = n1 % 10; # қалдық-санның соңғы цифрын табамыз
        n1 = n1 // 10; # Бутин болігін аламыз
        n2 = n2 * 10 #  разрядын улкейтеміз
        n2 = n2 + digit # келесі цифрды қосамыз

print('Кері сан:',n2)
```

Нәтижесі:

```
Бутін санды енгіз: 4578
Кері сан: 8754
```

2-әдіс.Программасы:

```
a = input()
b = ''
for i in a:
    b = i + b
print(b)
```

Нәтижесі:

```
Бутін санды енгіз: 4578
Кері сан: 8754
```

№3. Берілген сандарға байланысты Евклид алгоритмінің қайталану бөлігінің неше рет орындалатынын табу керек.

Евклид алгоритміне алдындағы ереже бойынша санаушы қосамыз.Программасы:

Нәтижесі:

```
m = int(input('m = '))
n = int(input('n = '))
k = 0   # алғашқыда санаушы 0 ге тең
while m != n:
    if m > n:
       m = m - n
    else:
       n = n - m
    k  =  k + 1 # әрбір қайталанған сайын санаушының мәні 1 ге өсіп отырады
print( 'ЕҮОБ = ', m)
print( ' k = ', k)
```

```
m = 81
n = 27
ЕҮОБ =  27
 k =  2
```

№4. Программаға енгізілген сандардың қосындысын табу керек. Енгізілген0 саны сандардың соңының белгісі.

Программасы:

```
s = 0 # қосынды s айнымалысы 0 тең
while True: # шексіз цикл
   a = int(input('сан = '))
   if a == 0: # егер енгізілге сан 0 болса
       break # циклдан шығу
   s = s + a # қосындыға кезекті санды қосу
print( 'қосынды = ', s)
```

Нәтижесі:

```
сан = 5
сан = 5
сан = 4
сан = 6
сан = 8
сан = 0
қосынды =  28
```

```
p = 1 # көбейтінді p айнымалысы 1 тең
while True: # Шексіз цикл
    a = int(input('сан = '))
    if a == 0: # егер енгізілге сан 0 болса
        break # циклдан шығу
    p = p * a # көбейтіндіге кезекті санды көбейту
print( 'көбейтінді = ', p)
```

№5. Программаға енгізілген сандардың ішінен тек оң сандардың қосындысын ғана табу керек. Сонымен қоса енгізілген барлық сандардың санын және оң сандардың санын табу керек. Енгізілген 0 саны сандардың соңының белгісі.

Программасы:

```
s = 0 # қосынды s айнымалысы 0 тең
n = 0 # барлық сандардың саны n айнымалысы 0 тең
m = 0 # барлық оң сандардың саны m айнымалысы 0 тең
while True: # шексіз цикл
   a = int(input('сан = ')) # кезекті санды енгізу
   if a == 0: # егер енгізілген сан 0 болса
       break # циклдан шығу
   n = n + 1 # барлық сандарды санаушыны 1 ге өсіру
   if a < 0: # егер енгізілген сан теріс болса
       continue # циклды басынан жалғастыру
   s = s + a # қосындыға кезекті санды қосу
   m = m + 1 # оң сандардың санаушысын 1 ге өсіру
print( 'Барлығы ', n, ' сан')
print( 'оң сандар ', m, ' сан ')
print( 'қосынды = ', s)
```

Нәтижесі:

```
сан = 5
сан = -5
сан = 4
сан = -4
сан = -3
сан = 4
сан = 6
сан = 0
Барлығы  7  сан
оң сандар  4  сан
қосынды =  19
```

№6. Программаға енгізілген сандардың көбейтіндісін табу керек. Енгізілген 0 саны, сандарды енгізу аяқталғанын білдіреді.

Программасы:

Нәтижесі:

```
сан = 3
сан = 4
сан = -2
сан = 5
сан = 0
көбейтінді =  -120
```

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Цикл дегеніміз не және ол не үшін қолданылады?
2. Python тілінде циклдің қандай түрлері бар?
3. for циклының негізгі қызметі қандай?
4. for циклы қандай жағдайда қолданылады? Мысал келтіріңіз.
5. while циклының негізгі қызметі қандай?
6. while циклы қандай жағдайда тиімді?
7. Шексіз цикл дегеніміз не және оны қалай тоқтатуға болады?
8. break операторын қолдану ерекшелігі қандай?
9. continue операторының қызметін түсіндіріңіз.
10. Циклмен бірге қолданылатын else блогының ерекшелігі неде?
11. range() функциясы қандай мақсатта қолданылады?
12. Ішкі цикл (nested loop) дегеніміз не? Мысал келтіріңіз.
13. Циклдердің орындалу ретін қалай бақылауға болады?
14. for және while циклдарының айырмашылығы неде?
15. Циклді қолданудың практикалық мысалдарын келтіріңіз.

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахъ,2020.,-119б

Дәріс №7

Дәріс тақырыбы: RANGE түріндегі шамалар

Дәріс мақсаты: RANGE түріндегі шамалармен жұмыс жасау. RANGE түріндегі әр тұрлі шамалармен практикалық тапсырмалар орындау.

Кілт сөздер: `range()`, Итерация жасау үшін жиі `for` циклымен бірге пайдаланылады, range`(stop)`,range`(start, stop)`,range`(start, stop, step)`,`len(range(...))` – элементтер санын анықтау

Дәріс жоспары:

1. RANGE түріндегі шамалар
2. Қималар
3. Практикалық жұмыс.

1. Range түріндегі шамалар арифметикалық прогрессия құрайтын *бүтінсандар* тобынан тұрады.

Ескерту: Бұл топты тізім немесе жиын деп атауға болмайды, себебіPython тілінде ***тізім*** *және* ***жиын*** *деп аталатын деректердің түрлері бар.*

range түріндегі шамалар **range()** функциясы арқылы анықталады.айнымалы = **range(басы, соңы, қадам)**

айнымалы басы, басы + қадам, ... әрі қарай осы сияқты арифметикалықпрогрессия құрайтын сандардан тұрады. Соңғы мән топқа кірмейді.

range() функциясында 1, 2 немесе 3 аргумент жазуға болады.

range(k, n, m) 3 аргумент болса, мұндағы **k –** бастапқы мән**, n** -соңы**, m –** қадам

range(k, n) 2 аргумент болса, мұндағы **k –** бастапқы мән**, n –**соңы, қадам 1тең деп есептеледі.

range(n) 1 аргумент болса, мұндағы**, n** - соңы**, –** бастапқы мән 0**,** қадам 1тең деп есептеледі.

range(1, 10, 2) нәтиже [1, 3, 5, 7, 9]

range(1, 5) нәтиже [1, 2, 3, 4]

range(5) нәтиже [0, 1, 2, 3, 4]

>>> **a** = range(1, 9, 2)

>>> type(a)

<class 'range'>

a айнымалысы 1, 3, 5, 7 сандар тобынан тұрады. Функцияда көрсетілген 9саны алынбайды, себебі топтағы сандар 9-дан кіші болуы керек.

range түріндегі сандар тік жақша арқылы нөмірленеді.

Шамалардың нөмірі немесе индексі 0 ден басталады. a[0] = 1,

a[1] = 3, a[2] = 5, a[3] = 7

range айнымалысы өлшемін (неше мәннен тұратынын) **len(айнымалы)** функциясын қолдану керек.

>>> len(a)4

range() функциясында қадам теріс сан болуы мүмкін. range(0, -5, -1)) # [0, -1, -2, -3, -4]

range айнымалысының индексі теріс сан да болуы мүмкін. Соңғы элементтің индексі -1 тең деп есептеледі, соңғының алдындағысы -2, т.с.с

a = range(1, 9, 2)

a[0] = 1, a[1] = 3, a[2] = 5, a[3] = 7

a[-1] = 7, a[-2] = 5, a[-3] = 3, a[-4] = 1

Егер функция аргументтері арифметикалық прогрессияға сәйкес болмаса,нәтиже бос мән болады.

range(6, 4, 2)) # []

range(4, 8, -2)) # []

range(0)) # []

Санның **range** тобына кіретіні **in**, кірмейтіні **not in** амалдары арқылы анықталады.

>>> a = range(1, 9, 2) # [1, 3, 5, 7]

>>> 5 in a True

>>> 4 in a False

>>> 6 not in a True

2. Қималар

Бірнеше деректерден тұратын шамаларға **қима амалы** қолданылады. айнымалы[i : j : K] – топтың элементінен **i** бастап **j** элементіне дейінгі **k** қадамымен алынған қимасы. j - элемент қимаға кірмейді.

Егер тиісті аргументтер көрсетілмесе, онда i = 0, j = элементтер санына, K =1 деп есептеледі.

t = range (10) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

r = t[1: 9 : 2] # [1, 3, 5, 7] 1 ден 9 элементке дейінгі 2 қадамменалынған қима, 9-шы элемент қимаға кірмейді.

b = t[: 7] # [0, 1, 2, 3, 4, 5, 6]

b = t[5 :] # [5, 6, 7, 8, 9]

b = t[: 7 : 2] # [0, 2, 4, 6]

b = t[: : 4] # [0, 4, 8]

b = t[: : -1] # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

b = t[: : -2] # [9, 7, 5, 3, 1]

b = t[: :] # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] - b, t – ның өзіне тең

b = t[1: 4 :-1] # [] - бос мән

for операторы

for операторы шамалар тобындағы (range, тізім, жиын, сөздік) деректерді бір - бірлеп қарастыру үшін қолданылады.

for айнымалы **in** шамалар_тобы: оператор

for операторы шамалар тобындағы деректер біткенше қайталанады. Әрбір орындалған сайын айнымалының мәні топтағы кезекті шамаға тең болады. Осындай қайталануды **итерациялық цикл** деп те атайды.

Мысалы:

```
a = range(1, 10, 2)
for p in a:   # немесе бірден жазуға болады: for p in range(1, 10, 2):
    print(p, end = ' ')
```

Нәтижесі:

```
1 3 5 7 9
```

3. Практикалық жұмыс.

№1. range(305, 405, 15) бүтін сандар тобындағы цифрларының қосындысы10-нан үлкен сандарды және олардың орнын (индексін) табу керек.

Программасы:

```
k=range(305, 405, 15)
print (list(k))
print('sany =', len(k))
for n in k:
    n1 = n
    s = 0
    while n1 != 0:
      m = n1 % 10
      n1 //= 10
      s += m
    if s > 10:
       j = k.index(n)
       print('k[', j, '] =', n, ' tsifrlar qosyndysy =', s)
```

Нәтижесі:

```
[305, 320, 335, 350, 365, 380, 395]
sany = 7
k[ 2 ] = 335  tsifrlar qosyndysy = 11
k[ 4 ] = 365  tsifrlar qosyndysy = 14
k[ 5 ] = 380  tsifrlar qosyndysy = 11
k[ 6 ] = 395  tsifrlar qosyndysy = 17
```

№2. Сөз беріледі. Әріптерін бағана бойынша шығару программасын жаз.

Len() функциясын қолданамыз. Бұл берілген сөз неше әріптен тұратынынанықтайды, мысалы информатика сөзі 11 әріптен тұрады.

1-әдіс. Программасы:

```
soz=input("Созді енгіз:")
for i in range(0,len(soz)):
    print(soz[i])
```

2-әдіс. Программасы:

```
soz=input("Созді енгіз:")
for i in soz:
   print(i)
```

Нәтижесі бірдей болады:

```
Созді енгіз:информатика
арипті енгіз:т
и
н
ф
о
р
м
а
т
и
к
а
```

№3. Сөз беріледі. Енгізілген әріп осы сөздің ішінде бар ма екенін анықтаупрограммасын жаз.

1-әдіс.Программасы:

```
soz=input("Созді енгіз:")
arip=input("Аріпті енгіз:")
for i in range(0,len(soz)):
   if soz[i]==arip:
       print("Аріп бар")
```

2-әдіс. Программасы:

```
soz=input("Созді енгіз:")
arip=input("Аріпті енгіз:")
for i in soz:
   if i==arip:
       print("Аріп бар")
```

Нәтижесі бірдей:

```
Созді енгіз:педагог
Аріпті енгіз:о
Аріп бар
```

№4. Берілген сан бойынша көбейту кестесін шығаратын программа құр.
Программасы:

```
n=int(input("Санды енгіз:"))
for i in range(1,11):
    print(n,"*",i,"=",n*i)
```

Нәтижесі:

```
Санды енгіз:5
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

Тақырып бойынша қарастырылатын сұрақтар:

1. **Range** функциясының негізгі мақсаты қандай?
2. **Range()** функциясы қанша параметр қабылдай алады?
3. **Range(n)** түріндегі жазылуда n мәні нені білдіреді?
4. **Range(start, stop)** жазылуындағы *start* және *stop* параметрлерінің рөлі қандай?
5. **Range(start, stop, step)** түріндегі жазылуда *step* параметрі не үшін қолданылады?
6. Теріс қадам (**step < 0**) мәні берілгенде **range()** қалай жұмыс істейді?
7. **Range** объектісін **list()** функциясының көмегімен тізімге қалай айналдыруға болады?
8. **Range** функциясының нәтижесі қандай типті мән болып табылады?
9. **Range(0, 10, 2)** функциясы қандай тізім құрайды?
10. Неліктен **range()** үлкен сандармен жұмыс істегенде тиімді деп есептеледі?

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахъ,2020.,-119б

Дәріс №8

Дәріс тақырыбы: Тізімдер

Дәрістің мақсаты: *Python бағдарламалау тіліндегі* тізімдер *туралы жалпы деректер беру,* тіл мүмкіндіктерін қарастыру.

Кілт сөздер: list ,append(), insert() , extend() , remove() , pop(), clear() , index() , count(), sort() , reverse() , len() , in , slicing (list[start:stop:step])

Дәріс жоспары:

1. Тізімдер - өзгермелі тізбектер
1. Тізім әдістері мен функциялары
2. Практикалық жұмыс

1. Тізімдер - өзгермелі тізбектер

Егер көп деректерді бір жерге сақтау қажет болса, деректер құрылымдарын қолдану керек. Олардың бірі - **тізімдер**. Тізімдер сандардың, жолдардың, басқа тізімдердің немесе олардың барлығының комбинациясынан тұра алады. Жалпы, **тізім** дегеніміз - **белгілі бір ретпен берілген элементтердің жиынтығы.** Тізімдегі әр элементтің өз индексі бар (1-ші элемент 0-ден басталады). Тізімдегіэлементтерді кез келген уақытта өзгертуге, жоюға немесе жаңа элементтерді енгізуге болады.

Python тіліндегі тізім (list) түріндегі шама кез келген деректердің үтірмен бөлінген тізімінен тұрады. Тізім квадрат жақшаға алынып жазылады. Мысалы:

```
d = [15, 6.15, 'мектеп', True, range(3)]
type(d)
<class 'list'>
```

Тізім түріндегі деректер тік жақша арқылы нөмірленеді. Шамалардың нөмірі немесе индексі 0 ден басталады.

```
d[0] = 15, d[1] = 6.15, d[2] = 'мектеп', d[3] = True, d[4] = range(3)u = []  # бос тізім
```

Тізім айнымалысының өлшемін (нешемәннен тұратынын) анықтау үшін **len(айнымалы)** функциясын қолдану керек.

```
len(d)          # нәтиже 5
len(u)          # 0 ге тең
```

Тізім элементтері әртүрлі деректер болу мүмкін.

type(d[0]) -<class 'int'>, type(d[1]) - <class 'float'>, type(d[2]) - <class 'str'>,type(d[3]) - <class 'bool'>, type(d[4])- <class 'range'>.

Тізім элементінің мәні мен түрін өзгертуге болады.

d[3] = -2 # *type(d[3]) - <class 'int'>* d[3] ‘элементінің мәні де, түрі деөзгерді.

Практикалық есептерде тізім бір түрдегі элементтерден тұрады.t1 = [23, 656, -20, 67, -45] # бүтін сандар тізімі

t2 = [4.15, 5.93, 6.45, 9.3, 10.0, 11.6] # нақты сандар тізімі

t3 = ["Болат ", "Кітап", "Принтер ", "Марат"] # жолдардан тұратын тізімТізімдерге **біріктіру** және **қайталау амалдары** қолданылады.

```
d = [15, -5, 84]
h = [32, 15]
p = d + h + [-8, 63]          #  [15, -5, 84, 32, 15, -8, 63]
w = 3*h + [1]*4               #  [32, 15, 32, 15, 32, 15, 1, 1, 1, 1]
```

Range сандарын тізімге айналдыру

1) түрлендіруші функциясы арқылы

Программасы:

```
d = range(2, 10, 3)      #   range түріндегі шама
p = list(d) # [2, 5, 8] тізім түріндегі шама
print(p, end=' ')
```

Нәтижесі:

```
[2, 5, 8]
```

немесе бірден түрлендіруге болады:

```
p = list(range(2, 10, 3)) # бірден түрлендіруге болады.
print(p, end=' ')
```

Нәтижесі:

```
[2, 5, 8]
```

2) тізім құрушы генератор арқылы [өрнек for айнымалы in деректер_тобы]

```
a = [i for i in range(3, 12, 2)]        # [3, 5, 7, 9, 11]
print(a, end=' ')
```

Нәтижесі:

```
[3, 5, 7, 9, 11]
```

Тізім қималары

айнымалы[i : j : K] – тізімнің **i** элементінен бастап **j** элементіне дейінгі **k** қадамымен алынған қимасы. j - элемент қимаға кірмейді.

Егер тиісті аргументтер көрсетілмесе, онда i = 0, j = элементтер санына, K =1 деп есептеледі.

Қималар арқылы тізім элементтеріу өзгертуA = [1, 2, **3, 4**, 5]

A[2:4] = [7, 8, 9] # [1, 2, **7, 8, 9**, 5]

A = [1, 2, 3, 4, 5, 6, 7]

A[: : -2] = [10, 20, 30, 40] # нәтиже [40, 2, 30, 4, 20, 6, 10], себебі

A[: : -2] = [7, 5, 3, 1]

2. Тізім әдістері мен функциялары

Тізім элементтерін өңдеу үшін тізімдерде бірқатар әдістер бар:

- append(item) - тізімнің соңына элементті қосады;
- insert(индекс, элемент) - элементті тізімге индекс бойынша қосады;
- remove(item) - элементті жояды. Элементтің біріншісі ғана жойылады. Егер элемент табылмаса, қате болады;
- clear() - тізімді тазалау (барлық элементтерді алып тастау);
- index(дерек) - деректің индексін қайтарады. Егер дерек жоқ болса, ValueError қателігі болады;
- pop([index]): индексі көрсетілген дерек тізімнен өшіріледі және функция мәні сол дерек болады (қайтарады). Егер индекс жазылмаса, онда соңғы дерек өшіріледі;
- count(item): тізімдегі элементтің қайталану санын қайтарады;
- reverse() - тізімдегі барлық элементтерді керісінше орналастырады;
- len(list): тізімнің ұзындығын қайтарады;
- sort(list): сұрыпталған тізім қайтарады;

- min(list): тізімнің ең кіші элементін қайтарады;
- max(list): тізімнің ең үлкен элементін қайтарады;
- copy() – көшірмесін алу.

Мысалдар.

```
d = [21, 7, 52, 7]
d.append(10)          нәтиже [21, 7, 52, 7, 10]
d.insert(2, 40)           [21, 7, 40, 52, 7, 10]
d.remove(8)               [21, 40, 52, 7, 10]
p = d.copy()          #
d = d.clear()         []
d = [21, 90, -5, 7]
maxd = max(d)           # maxd =90
mind = min(d)           # mind = -5
```

Тізім генераторлары.

Тізім құрушы құрылымдарды **тізім генераторлары** деп атайды. Жазылу

[тізім құрушы құрылым]

Тізім құрушы құрылым 3 бөлімнен тұрады:

1. Деректі түрлендіру өрнегі.
2. Қандай деректі алу керек.
3. Деректі қайдан алу керек.Мысалдар

қарастырайық:

1) a = [i for i in range(1, 6)] # [1, 2, 3, 4,5]

2) a = [i for i in range(20, 120) if i % 30 == 0 or i % 31 == 0] # Нәтиже a=[30, 31, 60, 62, 90, 93]

3) a = [2, -2, 3, -3, 6]
b = [i**2 for i in a] # b = [4, 4, 9, 9, 36]

4) c = [x**2 for x in range(1, 24, 2) if x % 3 != 0] # Нәтиже c = [1, 25, 49, 121, 169, 289, 361, 529] 5) tizim1 = [x for x in range(1, 60) if x % 2 == 0 if x % 3 == 0] # [6, 12, 18, 24, 30, 36, 42, 48, 54]

6) tizim2 = [2*x * y for x in [2, 3] for y in [3, 5, 6, 7]] # [12, 20, 24, 28, 18, 30,36, 42]

7) t0 = [7, 3, 6, 4]
t1 = [3, 2, 4]
t2 = [i + j for i in t0 for j in t1 if i < 5] # [6, 5, 7, 7, 6, 8]

3. Практикалық жұмыс

№1.Тізім элементтерін енгізіп, оны экранға шығару.

Программасы:

```
a = [0] * int(input('неше сан:')) # тізім өлшемі (ұзындығы)
# программа орындалғанда анықталады
for i in range(len(a)):
    a[i] = int(input('сан жаз ' ))
# Тізімдегі мәндерді жазудың бірінші жолы
for i in range(len(a)):
    print(a[i], end=' ')
print()
# Тізімдегі мәндерді жазудың екінші жолы
for elem in a:
    print(elem, end=' ')
```

Нәтижесі:

```
неше сан:6
сан жаз 4
сан жаз 3
сан жаз -4
сан жаз -3
сан жаз 8
сан жаз 15
4 3 -4 -3 8 15
4 3 -4 -3 8 15
```

№2. Элементтер саны белгілі range сандарын тізім құрушы генераторарқылы тізімге айналдыру.

Программасы:

```
a = [input('сан жаз: ')
for i in range(int(input('неше сан: ')))]
print(a, end=' ')
```

Нәтижесі:

```
неше сан: 6
сан жаз: 78
сан жаз: 45
сан жаз: -12
сан жаз: 65
сан жаз: 4
сан жаз: 34
['78', '45', '-12', '65', '4', '34']
```

№3. Тізім элементтерін біртіндеп енгізу.

Программасы:

```
a = []
for i in range(int(input('неше сан =' ))):
    print('a[',i,'] =', end=' ')
    a.append(int(input()))
print(a)
```

Нәтижесі:

```
неше сан =5
a[ 0 ] = 4
a[ 1 ] = 45
a[ 2 ] = -8
a[ 3 ] = 56
a[ 4 ] = 24
[4, 45, -8, 56, 24]
```

№4. Тізімдегі теріс сандардың ішіндегі ең үлкенін табу керек.
Программасы:

```
import random
n=int(input('n='))
b=[]
c=-20   #       c=int(input('c =')) d=50        #
d=int(input('d ='))   # [c, d] аралығынан кездейсоқ n нақты мәндерді аламыз
for i in range(n):
    t=int(100*random.uniform(c, d))/100
    b.append(t)
    print(b)
for i in range(n):
    if b[i] < 0:
       bmax = b[i]
       k = i
    break
for i in range(k+1, n):
    if b[i] < 0 and b[i] > bmax:
        bmax = b[i]
print('bmax =', bmax)
```

Нәтижесі:

```
d =24
[-5.36]
[-5.36, 3.15]
[-5.36, 3.15, -16.9]
[-5.36, 3.15, -16.9, -17.54]
bmax = -5.36
```

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Python тілінде **тізім (list)** дегеніміз не?
2. Тізімді құрудың қандай тәсілдерін білесіз?
3. Тізімдерде қандай деректер түрлері сақталуы мүмкін?
4. Тізімнің ұзындығын қалай анықтаймыз?
5. Тізімнің элементтеріне индекстеу арқылы қалай жүгінеміз?
6. Тізімнен белгілі бір бөлікті алу үшін **слайсинг (slicing)** қалай қолданылады?
7. `append()` және `insert()` әдістерінің айырмашылығы қандай?
8. Тізімге бірнеше элементті бірден қосу үшін қандай әдіс қолданылады?
9. `remove()` және `pop()` әдістерінің айырмашылығы неде?
10. Тізімді толық тазалау үшін қандай әдіс пайдаланылады?
11. `index()` және `count()` әдістерінің қызметі қандай?
12. Тізімді өсу немесе кему ретімен сұрыптау үшін қандай әдістер қолданылады?
13. `reverse()` әдісінің қызметі қандай?
14. `in` және `not in` операторлары тізімдерде қалай жұмыс істейді?
15. Тізім элементтерін **for циклі** арқылы қалай шығарамыз?
16. Көпөлшемді тізімдер дегеніміз не және оларды қалай қолдануға болады?
17. Тізімдерді біріктіру үшін қандай амалдар немесе әдістер қолданылады?

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс №9

Дәріс тақырыбы: Массивтер

Дәріс мақсаты: Python бағдарламалау тілінде қолданылатын массивтер және тізімдер туралы мәліметпен таныстыру. Оларды қолдану мүмкіндіктерін қарастыру.Практикалық тапсырмалар орындау.

Кілт сөздер: `list` ,`tuple` ,`array` ,`numpy.array` – NumPy ,`import array` , `from array import array`

Дәріс жоспары:

1. Массив
2. Python-да екі өлшемді массивтер.
3. Кездейсоқ сандарды алу функциялары
4. Практикалық жұмыс.
5. Сұрыптау.

1. Массив – бір атаумен біріктірілген, бір типтегі деректерден тұратын айнымалылардың жиынтығы. Әрбір алынған жеке айнымалы **массив элементі** деп аталады. Массив элементтері индекстер арқылы белгіленеді, индекстер тік жақшаның ішінде жазылады және олар нөлден бастап нөмірленеді.

Бір өлшемді массив – бір атауға біріктірілген бірнеше бір типтес элементтерден тұратын жиын. Әрбір элементтің өзінің реттік нөмірі болады. **Реттік нөмір** – массив элементінің индексі. Массив элементтері бірінен соң бірі ретпен орналасады. Массивті жариялаған кезде массивтің атауын, массив элементтерінің типін көрсетеміз. Массивтің өлшемі көрсетілмейді, ол кез келген уақытта жаңа элементтерді қосу арқылы кеңейтілуі мүмкін. Массивті Python-да пайдалану үшін array стандартты модулін импорттау қажет.

Python тілінде арнайы «массив» деген тип жоқ, элементтер жиынымен жұмыс жасау үшін тізімдер қолданылады. Мысалы b=[2,15,45,7,85,12,32,56]

Print(b) – баспаға шығару.

n=int(input()) # тізімдегі элементтер саны b=[]

#бос тізім құрып аламыз

for i= in range(1, n+1): # i цикл параметрі, элементтер саны 1-ден n+1-ге дейін

a=int(input()) # тізімге элементтерді енгіземіз b.append(a) #

append функциясы тізімге элемент қосадыprint(b) # экранға

шығару

Есептерде деректерді кесте түрінде сақтау жиі кездеседі. Мұндай деректер **матрица** немесе **екі өлшемді массив** деп аталады. Python программалау тіліндекөпөлшемді массивтер жоқ. Алайда кез келген кесте тізімдер тізіміменұсынылуы мүмкін. Яғни тізімнің әрбір элементі тізім болып табылады.

Бұл кесте 2x3 өлшемдегі екіөлшемді массив түрінде ұсынылуы мүмкін.

Мұндай массивте 6 элемент болады.

5	9	2
2	4	8

Массив элементтеріне сәйкес келетін индекстерді қарастырайық. Жолдар мен бағандардың индекстері 0-ден басталып, тік жақшаға жазылады.

a [0][0]	a [0][1]	a [0][2]
a [1][0]	a [1][1]	a [1][2]

2. Python-да екі өлшемді массивтер. Python-да екі өлшемді массивтермен жұмыс істеу үшін кіріктірілген тізімдерді пайдалануға болады. Мысалы, кестедегі массивтің программасын a = [[45, 69, 52], [12, 64, 78]] түрінде жазуға болады.

a тізімі әрқайсысы үш саннан тұратын және белгілі бір массив мерзіміне сәйкес келетін екі кіріктірілген тізімнен тұрады. Екіөлшемді массивтің барлық жолымен жұмыс істегенде кіріктірілген тізімнің индексін көрсету керек.

Екіөлшемді массивтің барлық элементін кіріктірілген циклдер арқылы көрсетуге болады. Сыртқы цикл жол индекстерінің, ішкі цикл баған индекстерінің мәнін көрсетеді.

3. Кездейсоқ сандарды алу функциялары

import random # random модулін қосу керек(random-аудармасы кездейсоқ)

random.uniform(a, b) # [a, b] аралығында жататын кездейсоқ нақты сандыбереді.

random.randint(a, b) # [a, b] аралығында жататын кездейсоқ бүтін сандыбереді.

Мысалы,

Import random

a=[random. Randint(5,50) for i in range(3)] # (5,50) аралығынан кез келгенкездейсоқ 3 санды таңдап алады, яғни массив үш элементтен тұрады.

print(a)

```
import random
a=[ random. randint(5,50) for i in range(3)]
print(a)
```

Нәтижесі:

```
[16, 35, 27]
```

4. Практикалық жұмыс.

№1. Бір өлшемді массив берілген. Массивтегі жұп элементтерді жаңамассивке енгізіңіз.

Программасы:

```
n=int(input('n=', )) # тізімдегі элементтер саны
b=[]  #бос тізім құрып аламыз
for i in range(1, n+1): # i цикл параметрі, элементтер саны 1-ден n+1-ге дейін
    a=int(input()) # тізімге элементтерді енгіземіз
    b.append(a) # append функциясы тізімге элемент қосады
print(b)   # экранға шығару
d=[]    #жұп элементтерді енгізу үшін бос тізім құрып аламыз
for i in b:    # b бір өлшемді массивтен іздейміз
  if i%2==0:  #  жұптылыққа тексереміз
     d.append(i)  # шарт орындалса, d бос тізімге жұп элементтерді қосамыз
print(d)  # экранға шығару
```

Нәтижесі:

```
n=5
2
4
5
7
8
[2, 4, 5, 7, 8]
[2, 4, 8]
```

№2. Бір өлшемді массив берілген. Массивтің 5 цифрымен аяқталатынэлементтерінен жаңа

массив құрыңыз.

Программасы:

```
import random
a=[random.randint(-50,50) for i in range(25)] # кездейсоқ сандар генераторы
print(a)
b=[] # бос тізім құрып аламыз
for i in a:
   if i%10==5: #5 цифрына аяқталатын элементтерді тексеру
      b.append(i)
print(b)
```

Нәтижесі:

```
[46, -15, -6, 38, 12, 8, 28, 9, 12, -49, -50, 31, 44, -49, -4, 33, -29, -13, -28
, 30, 31, 16, -13, 12, -37]
[-15]
```

№3. Массивті кездейсоқ сандармен толтыру және шығару.

Массивтің индексін басынан аяғына дейін сұрыптап, оның әр ұяшығынакездейсоқ құрылған санды жазады.

from random аудармасы-от случайных, из произвольных-кездейсоқ, еркін

Программасы:

```
from random import random
n = int(input('n=', ))          # тізімдегі элементтер саны
a = []           #бос тізім құрып аламыз
for i in range(n):
    t = int(random()*100)
    a.append(t)
print(a)
```

Нәтижесі:

```
n=7
[60, 94, 73, 23, 4, 1, 15]
```

№4. Массивті пернетақтадан енгізілген сандармен толтыру және шығару.Программасы:

Нәтижесі:

```
n = int(input('n=', ))          # тізімдегі элементтер саны
b = []           #бос тізім құрып аламыз
print("sandy engiz")
for i in range(n):
    t = int(input())
    b.append(t)
print(b)
n=4
Sandy engiz
15
25
35
40
[15, 25, 35, 40]
```

№5. Екі массивтің қосындысын табу. Массивтің мазмұнын экранға шығару.

Программасы:

```
n = int(input('n=', ))          # тізімдегі элементтер саны
a = []            #бос тізім құрып аламыз
b = []
c = []
print("sandy engiz")
for i in range(n):
    t = int(input())
    a.append(t)
print(a)
print("sandy engiz")
for i in range(n):
    k = int(input())
    b.append(k)
print(b)
for i in range(n):
    p = a[i] + b[i]
    c.append(p)
print('a[i] + b[i]=',c)
```

Нәтижесі:

```
n=6
sandy engiz
4
8
6
7
9
12
[4, 8, 6, 7, 9, 12]
sandy engiz
14
15
12
23
25
31
[14, 15, 12, 23, 25, 31]
a[i] + b[i]= [18, 23, 18, 30, 34, 43]
```

№6. -5-тен 5-ке дейінгі кездейсоқ сандарды екі массивке бөліп орналастыру керек: біреуінде тек оң, екіншісінде - тек теріс сандар. Нөлге тең сандар ескерілмейді. Барлық құрылған кездейсоқ сандарды және екі массивтің элементтерін экранға шығарыңыз.

Программасы:

```
from random import random
a = []
b = []
for i in range(20):
    n = int(random() * 11) - 5#Кездейсоқ сан жасау
    print(n, end=', ')#оны экранға шығару
    if n > 0:
        a.append(n)
    elif n < 0 :
        b.append(n)
print()
print(a)
print(b)
```

Нәтижесі:

```
5, 1, -1, -5, -4, -5, -2, -5, -4, 2, 2, -2, -1, -5, -5, 5, 2, 2, -1, -5,
[5, 1, 2, 2, 5, 2, 2]
[-1, -5, -4, -5, -2, -5, -4, -2, -1, -5, -5, -1, -5]
```

№7. Массивтің жұп элементтерінің индекстерін анықтаңыз.

Программасы:

```
from random import random
n = 10
a= [0]*n
index = []
for i in range(n):
    a[i] = int(random() * 10) + 10
    if a[i] % 2 == 0:
        index.append(i)
print(a)
print('Жұп элементтер индексі: ', index)
```

Нәтижесі:

```
[15, 11, 11, 17, 18, 12, 18, 14, 14, 14]
Жұп элементтер индексі:  [4, 5, 6, 7, 8, 9]
```

№8. Массивтегі ең кіші және ең үлкен элементтердің орнын ауыстыру.1-әдіс.
Программасы:

```
from random import random
n=int(input('n=', ))
arr = [0]*n
for i in range(n):
    arr[i] = int(random()*100)
    print(arr[i],end=' ')
print()
mn = min(arr)
mx = max(arr)
imn = arr.index(mn)
imx = arr.index(mx)
print('arr[%d]=%d arr[%d]=%d' % (imn+1, mn, imx+1, mx))
arr[imn],arr[imx] = arr[imx],arr[imn]
for i in range(n):
    print(arr[i],end=' ')
print()
```

Нәтижесі:

```
n=6
85 58 50 30 31 58
arr[4]=30 arr[1]=85
30 58 50 85 31 58
```

2-әдіс. Программасы:

```
from random import random
n=int(input('n=', ))
arr = [0]*n
for i in range(n):
    arr[i] = int(random()*100)
    print(arr[i],end=' ')
print()
mn = 0
mx = 0
for i in range(n):
   if arr[i] < arr[mn]:
        mn = i
   elif arr[i] > arr[mx]:
    mx = i
print('arr[%d]=%d arr[%d]=%d' % (mn+1, arr[mn], mx+1, arr[mx]))
b = arr[mn]
arr[mn] = arr[mx]
arr[mx] = b
for i in range(n):
    print(arr[i],end=' ')
print()
```

Нәтижесі:

```
n=5
0 83 36 45 36
arr[1]=0 arr[2]=83
83 0 36 45 36
```

№9. Массивте максималды теріс элементті табыңыз. Оның мәнін жәнемассивтегі орнын көрсету.

Программасы:

```
from random import random
n = int(input('n=',  ))
arr = []
for i in range(n):
        arr.append(int(random() * 100) - 50)
print(arr)

i = 0
index = -1
while i < n:
        if arr[i] < 0 and index == -1:
                index = i
        elif arr[i] < 0 and arr[i] > arr[index]:
                index = i
        i += 1
print('index=',index+1,': arr[',index+1,']=', arr[index])
```

Нәтижесі:

```
n=6
[3, 47, -47, 31, 45, -12]
index= 6 : arr[ 6 ]= -12
```

№10. Массивтің жұп оң элементтерінің қосындысын табу.

Программасы:

```
from random import random
n = int(input('n=',  ))
arr = [0] * n
for i in range(n):
    arr[i] = int(random() * 10) - 5
print(arr)
s = 0
for i in range(n):
    if arr[i] >= 0 and arr[i]%2 == 0:
        s += arr[i]
print(s)
```

Нәтижесі:

```
n=8
[-5, 3, -2, -2, -3, 2, 4, -3]
6
```

№11. Массив элементтерінің қосындысы мен көбейтіндісін есептеу. Массивтің өзін, оның элементтерінің қосынды мен көбейтіндісін шығару.

Программасы:

```
n=int(input('n=', ))
a = [0]*n
s = 0
m = 1
for i in range(n):
    a[i] = float(input())
    s += a[i]
    m *= a[i]
print(a)
print(s)
print('%.4f'%m)
```

Нәтижесі:

```
n=6
4
8
2
7
9
10
[4.0, 8.0, 2.0, 7.0, 9.0, 10.0]
40.0
40320.0000
```

№12. -5-тен 4-ке дейінгі диапазонда кездейсоқ құрылған 20 бүтін сандардың арасында қанша оң, теріс және нөлдік мәндер бар екенін анықтау. Массив элементтерін және есептелген сандарды шығару.

Программасы:

```
from random import random
neg = pos = zero = 0
a = []
for i in range(20):
    n = int(random() * 10) - 5
    a.append(n)
    print(n, end=' ')
    if n > 0:
        pos += 1
    elif n < 0:
        neg += 1
    else:
        zero += 1
print("\nоң сан: ", pos)
print("терiс сан  : ", neg)
print("нөл: ", zero)
```

Нәтижесі:

```
-3 -5 4 2 3 3 1 -4 3 0 4 0 -3 0 1 -1 0 -2 1 -3
оң сан:  9
терiс сан  :  7
нөл:  4
```

№13. Сызықтық массивтің оң элементтерінің арифметикалық ортасын табу.

Массивті толтыру үшін жалған кездейсоқ сандар генераторын қолданамыз.Сандарды -5-тен 4-ке дейінгі диапазонда құру керек делік. Барлық мәндер 10, - 5-ке ауысады. Осылайша, стандартты функцияның көмегімен біз 0-ден 10-ға дейінгі сандарды жасаймыз және олардан 5-ті аламыз.

Программасы:

```
from random import random
N = 20
a = []
for i in range(N):
    n = int(random() * 10) - 5
    a.append(n)
print(a)

s = q = 0
for i in range(N):
    if a[i] > 0:
        s += a[i]
        q += 1
print("%5.2f" % (s/q))
```

Нәтижесі:

```
[-2, -4, 0, -5, -1, 4, 1, -4, -2, -2, 4, -3, -1, 0, -3, -2, -4, -5, -5, 2]
 2.75
```

№14. Массивтегі ең үлкен элементті және оның реттік нөмірін табу.Программасы:

Нәтижесі:

```
from random import random
n = 10
arr = [0] * n
index = 0
for i in range(n):
    arr[i] = random() * 100
    print("%.2f" % arr[i], end='; ')
    if arr[i] > arr[index]:
        index = i
print("\narr[%d] = %.2f" % (index, arr[index]))
```

```
33.34; 0.30; 68.83; 91.20; 39.13; 69.38; 86.21; 95.45; 30.17; 57.43;
arr[7] = 95.45
```

Python-да массив элементтерін жою немесе кірістіру үшін кіріктірілген тізім функциялары мен әдістерін қолдануға болады.

Әдіс	Мақсаты
list.append(x)	list тізімнің соңына x элементін қосу
list.extend(L)	list тізімнің соңына L тізімінің элементтерін қосу
list.insert(i, x)	x мәні бар i индексіне элементті орнына қою
list.remove(x)	x мәні бар элементті жою
list.pop(i)	i индексі бар элементті жою
list.copy()	list тізімін көшіру
list.clear()	list тізімін тазарту (барлық элементтерді жою)

№15. Сандық массив берілген. 2 индексіндегі элементті жою керек.
Программасы:

```
san=[2,4,6,8,10]
san.pop(2)
print(san)
```

Нәтижесі:

```
[2, 4, 8, 10]
```

remove() әдісі жақшада көрсетілген мәні бар элементті жоюға мүмкіндік береді.

№16. Массив берілген. '8' мәніне ие элементті жою керек.
Программасы:

```
san=[2,4,6,8,10]
san.remove(8)
print(san)
```

Нәтижесі:

```
[2, 4, 6, 10]
```

Ескерту: егер массивтегі бірнеше элементтердің мәндері **remove()** әдісінде көрсетілген мәнмен сәйкес келсе, онда көрсетілген мәні бар бірінші элемент жойылады.
clear() әдісін қолдана отырып, бүкіл тізімді тазартуға болады.

№17. Массив берілген. Барлық элементтерді жою, тазарту керек.
Программасы:

```
san=[2,4,6,8,10]
san.clear()
print(san)
```

Нәтижесі: массивтің барлық элементтері жойылады.

```
[]
```

№18. Массивтің теріс элементтерін жою.
Программасы:

```
from random import random
a = []
for i in range(10):
    n = int(random()*10) - 5
    a.append(n)
print(a)
i = 0
m = 10
while i < m:
    if a[i] < 0:
        del a[i]
        m -= 1
    else:
        i += 1
print(a)
```

Нәтижесі:

```
[-4, 1, -5, 3, 4, 3, -1, 1, 2, -3]
[1, 3, 4, 3, 1, 2]
```

Элементтерді кірістіру

insert() әдісі белгілі бір позицияға массивке жаңа элемент қосуға мүмкіндік береді. Барлық кейінгі элементтер оңға жылжытылады. Массив элементтерінің саны 1-ге көбейтіледі. **insert()** әдісін шақырған кезде екі аргумент көрсетіледі:

- жаңа элементтің индексі;
- жаңа элементтің мәні.

№19. Сандық массив берілген. Үшінші орынға «11» санын кірістіру керек.

Программасы:

```
san=[3,6,9,12,15,18,20]
san.insert(3,11)
print(san)
```

Нәтижесі:

```
[3, 6, 9, 11, 12, 15, 18, 20]
```

№20. Массив берілген. '22' мәніне ие элементті кірістіру керек.Программасы:

Нәтижесі:

```
san=[3,6,9,12,15,18,20]
san.append(22)
print(san)
```

```
[3, 6, 9, 12, 15, 18, 20, 22]
```

extend() әдісінің көмегімен тізімнің соңына бірнеше элементтер кірістіругеболады.

№21. san2 массивінің сандарын san1 сандық массивіне кірістіру керек.

Программасы:

```
san1=[1,3,5,7]
san2=[12,24,36]
san1.extend(san2)
print(san1)
```

Нәтижесі:

```
[1, 3, 5, 7, 12, 24, 36]
```

copy() әдісі тізімді көшіру үшін қолданылады.

№22. san1 сандық массиві берілген. Барлық элементтерді san2 жаңа массивіне көшіру керек.

Программасы:

```
san1=[1,3,5,7]
san2=[12,24,36]
san2=san1.copy()
print(san1)
print(san1)
```

Нәтижесі:

```
[1, 3, 5, 7]
[1, 3, 5, 7]
```

Екі өлшемді массивтермен жұмыс істеу кезінде кейде жаңа элементтерді кірістіру немесе кейбір элементтерді жою керек. Екі өлшемді массив элементтерін жою немесе кірістіру үшін кіріктірілген тізім функциялары мен әдістерін қолдануға болады.

Берілген индекстегі элементті массивтен жою үшін **pop(i)** әдісін қолдануға болады.

№23. Екі өлшемді nxmмассиві берілген. Бірінші жолды жою керек.

Программасы:

```
n=5
m=4
a=[[1,5,8,18],
   [5,12,7,21],
   [58,12,4,5],
   [85,45,62,35],
   [25,65,4,8]]
a.pop(1)
for i in range(n-1)
    print(a[i])
print()
```

Нәтижесі:

```
[1, 5, 8, 18]
[58, 12, 4, 5]
[85, 45, 62, 35]
[25, 65, 4, 8]
```

Python-дағы **del** операторын көрсетілген индекстегі элементті жою үшін пайдалануға болады.

№24. Екі өлшемді массив берілген. Қолданушы енгізетін индекстерге сайжол мен бағанның қиылысындағы элементті жою керек.

Программасы:

```
a=[[1,5,8,18],
   [5,12,7,21],
   [58,12,4,5],
   [85,45,62,35],
   [25,65,4,8]]
print('Жолды енгіз')
b=int(input())
print('Бағанды енгіз')
c=int(input())
del a[b][c]
for i in range(5):
    print(a[i])
print()
```

Нәтижесі:

```
Жолды енгіз
2
Бағанды енгіз
3
[1, 5, 8, 18]
[5, 12, 7, 21]
[58, 12, 4]
[85, 45, 62, 35]
[25, 65, 4, 8]
```

insert() әдісі белгілі бір позициядағы массивке жаңа элемент қосуға мүмкіндік береді. **insert()** әдісін қолданған кезде жаңа элементтің индексі мен

мәні көрсетіледі. Бұл әдісті қолдану арқылы массивке жаңа жолдар немесебағандар қосуға болады.

№25. Екі өлшемді nxm массиві берілген. 3-тен тұратын екінші жолды қосукерек.

Программасы:

```
n=5
m=4
a=[[1,5,8,18],
   [5,12,7,21],
   [58,12,4,5],
   [85,45,62,35],
   [25,65,4,8]]
a.insert(1,[3,3,3,3])
for i in range(n+1):
    print(a[i])
print()
```

Нәтижесі:

```
[1, 5, 8, 18]
[3, 3, 3, 3]
[5, 12, 7, 21]
[58, 12, 4, 5]
[85, 45, 62, 35]
[25, 65, 4, 8]
```

6. Сұрыптау.

Программалардағы массивтер деректерді сақтау, іздеу және өңдеу қызметін атқарады. Массивтер арқылы әртүрлі операцияларды жүзеге асыруға болады. Кейбір әрекеттерді жеңілдету үшін массив элементтерін белгілі бір принцип бойынша реттеу керек. Мұны сұрыптау арқылы жасай аласың.

Сұрыптау – массив элементтерін өсу немесе кему ретімен жүйелеу.

Өсу ретімен сұрыпталғаннан кейін сандық деректер ең төменгі мәнненжоғары мәнге дейін, мәтіндік мәліметтер алфавит бойынша сұрыпталады.

Кему ретімен сұрыпталғаннан кейін сандық деректер ең үлкен мәннен ең төменгі мәнге дейін, мәтіндік деректер кері алфавиттік тәртіппен орналасады.

Сұрыптау үшін әртүрлі алгоритмді қолдануға болады. Python-да әдістер мен кіріктірілген тізім функцияларын қолдана отырып, сандық және мәтіндік массивтерді сұрыптауға болады.

Аталуы	Мақсаты
sort()	Тізімді сұрыптау әдісі
reverse()	тізім элементтерін кері бағытта орналастыруға арналған әдіс
sorted()	тізімді өсу ретімен сұрыптауға арналған функция

№26. Облыстардың халық саны туралы ақпарат берілген. Қала аттарыналфавит бойынша сұрыпта.

Программасы:

```
a=[['Aktobe',895100],['Atyrau',290700],['Mangystau',719500]]
a.sort()
for i in range(3):
    print (a[i])
```

Нәтижесі:

```
['Aktobe', 895100]
['Atyrau', 290700]
['Mangystau', 719500]
```

reverse() әдісі элементтерді кері бағытта орналастыруға мүмкіндік береді.

№27. Түстер массиві берілген. Түс атауларын кері алфавиттік ретпенорналастыр.

Программасы:

```
alfavit = ['b', 'g', 'o', 'd', 'r']
alfavit.reverse()
print (alfavit )
```

Нәтижесі:

```
['r', 'd', 'o', 'g', 'b']
```

sorted() функциясы тізімді өзгертпейді, бірақ жаңа тізім жасайды. Осылайша, реттелген тізім негізінде жаңа тізім шығады.

№28. Кездейсоқ 10 бүтін саннан тұратын массив жасалады. Элементтеріөсу ретімен орналасатын жаңа массив құра.

Программасы:

```
from random import randint
A = [randint(10,50) for i in range(10)]
B=sorted(A)
print (A)
print (B)
```

Нәтижесі:

```
[40, 28, 37, 29, 30, 46, 32, 47, 41, 39]
[28, 29, 30, 32, 37, 39, 40, 41, 46, 47]
```

Көпіршікті сұрыптау массив арқылы бірнеше рет өтуді қамтиды. Әр өтуде кезекті элемент келесі элементпен салыстырылады. Егер келесі элемент үлкен болса (өсу ретімен сұрыптағанда), онда массив элементтері ауыстырылады.

Осылайша, массив бірінші рет өсу ретімен сұрыпталғанда, массивтің соңғы мәні ең үлкен болады. Келесі салыстыруда соңғы элементтің алдындағы мән қалған сандардың үлкені болады. Соңғы және соңғының алдындағы сандарды салыстырудың мағынасы жоқ. Демек, әр өту кезіндегі қаралған массив элементтерінің саны 1-ге азаяды. Өту саны массив элементтерінің санынан бірге кем болады, өйткені жұптық салыстыру арқылы жүргізіледі.

Мысалы, «5 1 4 2 8» массиві берілген және оларды өсуі бойынша сұрыптайық, ол үшін көпіршік әдісін қолданамыз. Қарамен белгіленген элементтер, мына кезеңде салыстырылып отырылған элементтер.

Бірінші жол:

(**5 1** 4 2 8) (**1 5** 4 2 8), Мұнда алгоритм бастапқы екі элементті салыстырып, орындарын ауыстыруда, себебі 5 > 1

(1 **5 4** 2 8) (1 **4 5** 2 8), Орындарын ауыстырады, себебі 5 > 4

(1 4 **5 2** 8) (1 4 **2 5** 8), Орындарын ауыстырады, себебі 5 > 2

(1 4 2 **5 8**) (1 4 2 **5 8**), Енді әрбір элемент өз орнында тұрған себепті (8 > 5), алгоритм олардың орнын ауыстырмайды.

Екінші жол:

(**1 4** 2 5 8) (**1 4** 2 5 8)

(1 **4 2** 5 8) (1 **2 4** 5 8), Орындарын ауыстырады, себебі 4 > 2

(1 2 **4 5** 8) (1 2 **4 5** 8)

Енді алгоритм толықтай сұрыпталды, бірақ программаның оған көзіжеткен жоқ. Сол себепті программа тағы бір толық жол өткізеді.

Үшінші жол:

(**1 2** 4 5 8) (**1 2** 4 5 8)

(1 **2 4** 5 8) (1 **2 4** 5 8)

№29. Кездейсоқ сандармен толтырылған массивті өсу бойынша "көпіршік"әдісімен сұрыптау. Бастапқы және сұрыпталған түрдегі массивті шығару.

Программасы:

```
from random import random
a = [0]*10
for i in range(10):
    a[i] = int(random()*100)
print(a)

for i in range(9):
    for j in range(9-i):
        if a[j] > a[j+1]:
            a[j], a[j+1] = a[j+1], a[j]
print(a)
```

Нәтижесі:

```
[34, 33, 99, 97, 86, 65, 93, 92, 68, 21]
[21, 33, 34, 65, 68, 86, 92, 93, 97, 99]
```

Таңдау арқылы сұрыптау

Таңдау арқылы сұрыптауды өсу ретімен орындаудың бірінші қадамында массивтің минималды элементі табылады да, ол бірінші тұрған элементпен ауыстырылады. Массив екінші қадамды екінші элементтен бастапқарастырады. Ең кіші элементті тауып, екінші элементпен орын алмастырады. Әр элементтің ретін табу үшін осындай қадамдар қайталанады. Әрбір жаңа қадам массивтің сұрыпталмаған бөлігінен минимум мәнді тауып, оны ағымдағы элементпен ауыстырады. Әрекеттер **n-1** рет қайталанады.

Әрекеттердің қайталануы **итерация** деп аталады.

Кему ретімен сұрыптау үшін әрбір қадамда массивтің максималды элементін тауып отырады.

№30. 0-ден 100-ге дейінгі кездейсоқ 10 саннан тұратын массивті өсу ретібойынша таңдау арқылы сұрыптау.

Программасы:

```
from random import randint
N = 10
arr = []
for i in range(N):
    arr.append(randint(1, 99))
print(arr)
i = 0
while i < N - 1:
    m = i
    j = i + 1
    while j < N:
        if arr[j] < arr[m]:
            m = j
        j += 1
    arr[i], arr[m] = arr[m], arr[i]
    i += 1
print(arr)
```

Нәтижесі:

```
[19, 37, 1, 87, 29, 27, 10, 45, 2, 88]
[1, 2, 10, 19, 27, 29, 37, 45, 87, 88]
```

№31. Квадрат матрицаның негізгі диагоналінің оң элементтерін табу. Диагональдар матрицалар квадрат болған жағдайда ғана ерекшеленеді,
яғни жолдар саны бағандар санына тең болған жағдайда. Негізгі диагональэлементтерінің индекстері [1,1], [2,2], ... [n, n]. Бірінші индекс екіншісіне тең болғандықтан, матрицаны циклде сұрыптай отырып, бір айнымалы есептегішті қолдану жеткілікті (мысалы,i): массив атауы[i, i]. Егер диагоналдың келесі элементі нөлден үлкен болса, оны экранға шығарамыз.

Программасы:

```
from random import random
N = 10
a = []
for i in range(N):
    z = []
    for j in range(N):
        n = int(random() * 100) - 50
        z.append(n)
        print("%4d" % n, end='')
    print()
    a.append(z)
print()

for i in range(N):
    if a[i][i] > 0:
        print("%4d" % a[i][i], end='')
print()
```

Нәтижесі:

```
   7  11  11  -4  31  43 -17 -12  -6  -6
   1  -2  33  28   4 -20 -10 -30 -40 -44
  -9 -39 -33 -10  41   3 -19  12  -4  20
  46  -8 -17 -38  47 -29   0 -49 -33  15
  26 -37   0 -11   3  19  24  47  46 -45
 -13   9 -22  14  13  -7   8  23  25  49
 -12 -13 -19 -49  27  26 -39 -31 -30 -21
  15 -42  21   8  47 -48  49 -43  44 -30
   5 -26 -10  -2   0 -12 -31   1 -33  13
  11  -2  40   2 -20  43  19 -40 -50  36

   7   3  36
```

№32. 0-ден 999-ға дейінгі сандардың матрицасын құрып, оны экранғашығару. Ондағы екі таңбалы сандардың санын есептеу.

Матрицаны толтыратын циклге дейін екі таңбалы сандар есептегішін енгізу керек. Матрицаның әр ұяшығын толтырған кезде санды тексеру керек. Егер ол 10-дан 99-ға дейін болса, онда ол екі таңбалы болады және есептегішті бірге арттыру керек.

Программасы:

```
from random import random
M = 7
N = 5
count = 0
a = []
for i in range(N):
    b = []
    for j in range(M):
        n = int(random()*1000)
        if 9 < n < 100:
            count += 1
        b.append(n)
    a.append(b)

for i in a:
    print(i)
print("Екі таңбалы сандардың саны: %d" % count)
```

Нәтижесі:

```
[870, 542, 437, 306, 475, 598, 245]
[979, 59, 476, 297, 167, 102, 602]
[569, 34, 310, 695, 299, 387, 433]
[73, 236, 662, 79, 683, 949, 454]
[786, 512, 34, 513, 248, 461, 384]
Екі таңбалы сандардың саны: 5
```

№33. Екі матрицаның элементтерін салыстыру нәтижелері бойыншаүшінші матрицаны

толтыру.

Екі матрицаның элементтерін салыстырып, үлкенін үшінші матрицаға жазукерек.

Программасы:

```
M = 4
N = 3
a = []
for i in range(N):
    z = []
    for j in range(M):
        z.append(int(input()))
    a.append(z)
print("Бірінші матрица:")
for i in a:
    print(i)
b = []
for i in range(N):
    z = []
    for j in range(M):
        z.append(int(input()))
    b.append(z)
print("Екінші матрица:")
for i in b:
    print(i)

c = b[0:]
for i in range(N):
    for j in range(M):
        if a[i][j] > b[i][j]:
            c[i][j] = a[i][j]
print("Нәтиже:")
for i in c:
    print(i)
```

Нәтижесі:

```
14
15
24
26
23
51
45
41
85
54
56
84
Бірінші матрица:
[14, 15, 24, 26]
[23, 51, 45, 41]
[85, 54, 56, 84]
12
36
39
35
56
58
95
65
45
58
75
12
Екінші матрица:
[12, 36, 39, 35]
[56, 58, 95, 65]
[45, 58, 75, 12]
Нәтиже:
[14, 36, 39, 35]
[56, 58, 95, 65]
[85, 58, 75, 84]
```

№34. Квадрат матрицаның негізгі және бүйір диагональдарының мәндерінауыстыру.

Программасы:

```
from random import random
N = 10
a = []
for i in range(N):
    z = []
    for j in range(N):
        n = int(random() * 100)
        z.append(n)
        print("%4d" % n, end='')
    print()
    a.append(z)
print()

for i in range(N):
    for j in range(N):
        if i==j:
            b = a[i][j]
            a[i][j] = a[i][N-1-j]
            a[i][N-1-j] = b

for i in range(N):
    for j in range(N):
        print("%4d" % a[i][j], end='')
    print()
```

Нәтижесі:

```
  36  15  57  32  76  66  70  92  15   8
  35  61  58  11  29  17  46  29  24  25
  73  48  62  64  58  70   5  39   0  68
  86  42  43  80  41  36  93  78  76  53
  76  17  22  42  90  44  48  27  83  31
  94  35  74  22  80  64   0  38   2  32
  66  58  83  59  61  18  17   1   6  87
  42  32   2  81  96  30  50  67  35   9
  79  77  21  45  60  87  57  80  36  53
  69  42  11   0  44  10  25  49  68   5
```

```
   8  15  57  32  76  66  70  92  15  36
  35  24  58  11  29  17  46  29  61  25
  73  48  39  64  58  70   5  62   0  68
  86  42  43  93  41  36  80  78  76  53
  76  17  22  42  44  90  48  27  83  31
  94  35  74  22  64  80   0  38   2  32
  66  58  83  17  61  18  59   1   6  87
  42  32  67  81  96  30  50   2  35   9
  79  36  21  45  60  87  57  80  77  53
   5  42  11   0  44  10  25  49  68  69
```

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Екі өлшемді массив (2D массив) дегеніміз не?
2. Python тілінде екі өлшемді массивті қалай құруға болады?
3. Екі өлшемді массивтің элементтеріне қалай қатынауға болады?
4. matrix[row][col] жазылуындағы *row* және *col* нені білдіреді?
5. Екі өлшемді массивтен бір жолды немесе бір бағанды қалай алуға болады?
6. Екі өлшемді массивтің өлшемін (қатар, баған) қалай анықтаймыз?
7. Екі өлшемді массивтің барлық элементтерін шығару үшін қандай циклдер қолданылады?
8. NumPy кітапханасы арқылы екі өлшемді массивті қалай құруға болады?
9. np.zeros((m, n)) және np.ones((m, n)) функциялары қандай массивтер құрайды?
10. reshape() әдісінің қызметі қандай?
11. Массивті транспонирлеу үшін қандай әдіс қолданылады?
12. sum(axis=0) және sum(axis=1) дегеніміз не? Айырмашылығы қандай?
13. Екі өлшемді массивтегі ең үлкен және ең кіші элементті қалай табамыз?
14. Кәдімгі тізімдер мен NumPy массивтерінің айырмашылығы неде?
15. Python-да екі өлшемді массивтерді қай салаларда қолдануға болады?

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс №10

Дәріс тақырыбы:Функцияны анықтау және қолдану

Дәріс мақсаты: *Python бағдарламалау тілінде* функцияны қолдану. Функцияның негізгі мүмкіндіктері. Ішкі программалармен жұмыс жасау.

Кілт сөздер: `def` – функцияны анықтау (define),`return` – функциядан мәнді қайтару, `pass` – бос функция денесін белгілеу, `lambda` – анонимді (бір жолдық) функция құру, `yield` – генератор функциясында мәнді шығару, `global` – функция ішінен жаһандық айнымалыға қатынау, `nonlocal` – ішкі функциядан сыртқы (локалды) айнымалыға қатынау

Дәріс жоспары:

1. Ішкі программалар

2. Функция

3. lambda-функциялар

4. Практикалық жұмыс.

1.Ішкі программалар

Есепті шешу алгоритмі бүкіл есепті жеке ішкі тапсырмаларға бөлу арқылы жасалады. Әдетте ішкі тапсырмалар ішкі программалар түрінде жүзеге асырылады. **Ішкі программалар** - бұл программаның жеке функционалды тәуелсіз бөлігі. Ішкі программалар екі түрге бөлінеді - **процедуралар мен функциялар**.

Python-да ішкі программаларды С немесе Паскальда сияқты функциялар мен процедураларға ресми бөлу жоқ және процедураны бос мәнді қайтаратын функция деп санауға болады. Мұнда негізінен қолданылатын жалғыз термин - функция.

Python тілінде модульдерге біріктірілген көптеген стандартты функциялар бар, мысалы, math модулінің функциялары.

Python-да процедураны немесе функцияны анықтау үшін **def**-ді көрсету керек, оның атын және оның барлық аргументтерін үтірмен бөліп, жақша ішіне жазу керек, қос нүкте символынан кейін процедуралар командаларының жиынтығын енгізу керек.

Процедураның функциядан айырмашылығы, Python-дағы функция орындалғаннан кейін міндетті түрде қандайда бір мәнді қайтаруы керек, ал процедура тек командаларды орындайды.

Мысалы: Таңбаны шығару процедурасы «*»:

Программасы:

```
def printline(symbol,n):
 print(symbol * n)
printline('*', 10) #Процедураны шақыру
```

Нәтижесі:

```
**********
```

Мысалы: Факториалды есептеу функциясы:

Программасы:

```
def factorial(n):
 prod = 1
 i = 2
 while i<=5:
   prod *= i
   i +=1
 return prod
print(factorial(5))#Функцияны  шақыру
```

Нәтижесі:

Ауқымды айнымалылар – программа басында кез келген ішкі программаларды хабарлағанға дейін хабарланған деректер типтері, тұрақтылар мен айнымалылар. Бұл нысандар барлық программада, сонымен қатар оның барлық ішкі программаларында көрініп тұрады. Ауқымды айнымалылар программаның барлық жұмысы кезінде бар болады.

Жергілікті айнымалылар белгілі бір ішкі программаның ішінде хабарланады және тек осы ішкі программаға ғана көрінеді. Олар ішкі программа шақырғанша орындалмайды.

2. Функция

Программалау тілдерінде жиі қолданылатын есептеулерді функция түріндеөрнектеуге болады. Функция анықтамасында аты, аргументтері және орындалатын операторлар көрсетіледі.

def аты(аргументтер тізімі): оператор

Анықталған функцияны программада аты мен аргументтерінің нақты мәнін көрсету арқылы қолданылады.

аты(нақты аргументтер тізімі)

Мұны функцияны шақыру деп те атайды. Нақты аргументтердің мәндері, функция анықтамасындағы сәйкес аргументтерге меншіктеледі.

Мысалы,

```
# функция анықтамасы:
def fun(a, b):
print(a + b)
# функцияны қолдану:
1) x = 6
 y = 8
 fun(x, y)                    # a = x, b = y ,2)
t = -5.6
 p = 7.9
 fun(t, p)                    #    a = t, b = p,3)
d = [12, 3, 64]
 c = [2, 12]
 fun(c, d)                     #    [2, 12, 12, 3, 64]
 4) fun(4, 15)                # a = 4, b = 15
 5) fun(x + 2, y - 1)          #     a = x + 2, b = y - 1
```

Функция есептеу нәтижесін **return өрнек** операторы арқылы қайтарады.

```
# функция анықтамасы:
def summa (x, y):
return x + y
 # функцияны қолдану:
 1) print(summa (10, 4))          # нәтиже 142)
 a = 6
 b = 8
 d = 10 + summa(a, b)
 print(d) # нәтиже 24
 3) print(summa('Kosyndy', 'orneK')) # нәтиже KosyndyorneK
```

Функция анықтамасындағы аргументтерінің аттары, функцияда қолданылатын деректерді ғана көрсетеді. Ол атаулардың программадағыайнымалылардың атымен ешқандай байланысы жоқ.

def summa(x, y): return x + y функциясын def summa(a, b): return a + bдеп те анықтауға болады.

Функция аргументтері міндетті немесе міндетті емес болуы, орны немесеаты

бойынша көрсетілуі мүмкін. Функцияның аргументтері болмауы да мүмкін.# fun1 функциясының анықтамасында, c – міндетті емес аргумент, яғни үшінші аргумент жазылмаса, ол 2 тең деп есептелінеді.def fun1(a,
b, c = 2): return a + b + c
функцияны қолдану
fun1(1, 2) # a = 1, b = 2, c = 2 (үшінші аргумент жазылмаған)fun1(1, 2,
3) # a = 1, b = 2, c = 3 (орны бойынша)
fun1(b = 2, a = 3) # a = 3, b = 2, c = 2 (аты бойынша)
fun1(c = 3, a = 6) # a = 3, c = 6, b – мәні берілмеген, сондықтан қатешығады.

3. lambda-функциялар

lambda-функция дегеніміз, бір өрнекті есептеуге арналған, аты жоқ функция. lambda-функцияның аргументтері бірнешеу болуы мүмкін.

Мысал.

>>> (lambda x: x**2) (4) 16

lambda-функцияны бір айнымалыға меншіктеп, осы айнымалыны функцияның аты ретінде қолдануға болады.

1) sqrt = lambda x: x**0.5
sqrt(36) 6.0
2) func = lambda x, y: x + y
func(3, 5)
8
3) func('a', 'b') 'ab'
(lambda x, y: x + y)(3, 5) 8 (lambda x,
y: x + y)('a', 'b') 'ab'

lambda функцияларда return операторы болмайды.

4. Практикалық жұмыс.

№1.Тізімдегі сандардың қосындысын есептеу функциясы.

Программасы:

```
def sum(a):
    s = 0
    for k in a:
        s += k
    return s
t1 = [3, 27, 12, -6, -6]
t2 = [3.12, 4.52, 5.65, 10.2, 12.7]
print(sum(t1))
print(sum(t2))
```

Нәтижесі:

```
30
36.19
```

№2.Квадрат теңдеуді шешу функциясы.

Программасы:

```
def kvtendeu(a, b, c):
    d = b**2 - 4*a*c
    if d < 0:
       print('жауабы жоқ')
    elif d == 0:
        x = -b/a
        print('x = ',x)
    else:
      x1 = (-b - d) / (2 * a)
      x2 = (-b + d) / (2 * a)
      print('x1 = ', x1, ' x2 = ', x2)
kvtendeu(2, 4, 2)
kvtendeu(4, 2, 6)
kvtendeu(1, 6, 6)
```

Нәтижесі:

```
x =  -2.0
жауабы жоқ
x1 =  -9.0  x2 =  3.0
```

№3. Берілген санның неше орынды сан екенін анықтау.1-әдіс.
Программасы:

```
def digits(n):
    i = 0
    while n > 0:
        n = n//10
        i += 1
    return i

num = abs(int(input('Санды енгіз: ')))
print('Разрядтар саны:', digits(num))
```

```
Санды енгіз: 25741
Разрядтар саны: 5
```

2-әдіс. Программасы:

```
print('Разрядтар саны:', len(str(abs(int(input('Санды енгіз: '))))))
```

Нәтижесі:

```
Санды енгіз: 457812
Разрядтар саны: 6
```

№4. Берілген аралықта y=f (x) функциясының мәндерін есептеу.
Төмендегі функцияның мәндерін -10-дан 10-ға дейінгі аралықта , 1қадаммен есептеу.

$$y = \begin{cases} x^2, \text{ егер } -5 <= x <= 5; \\ 2*|x|-1, \text{ егер } x < -5; \\ 2x, \text{ егер } x > 5. \end{cases}$$

Функцияның мәнін есептеу x мәнін қабылдайтын және функцияның алынған мәнін (y) қайтаратын бағдарламалық функция түрінде жасалады.
Программасы:

```
def func(x):
    if -5<=x<=5:
        return x*x
    elif x < -5:
        return 2*abs(x)-1
    else:
        return 2*x

for i in range(-10,11):
    print(func(i), end=' ')
print()
```

Нәтижесі:

```
19 17 15 13 11 25 16 9 4 1 0 1 4 9 16 25 12 14 16 18 20
```

№5. Summa және summa2 функциялары бірдей орындалады. Тексеріп көр.Программасы:
Нәтижесі:

```
def summa(a,b): return a+b
summa2 = lambda a,b: a+b
print(summa(20, 30))
print(summa2(20, 30))
```

```
50
50
```

№6. lambda функция анықталған орнында бірден қолданылады.

```
fun = lambda *a: a
d = fun(6, 5, 11, 8)
print(d)
print(type(d))
```

Нәтижесі:

```
(6, 5, 11, 8)
<class 'tuple'>
```

№7. lambda функциялар тізімін қолдану.

Программасы:

```
L = [lambda x: x**2, lambda x: x**3, lambda x: x**4]
for f in L:
 print(f(3))     # Нәтиже 9,27,81
print(L[0](4))          # Нәтиже 16
```

Нәтижесі:

```
9
27
81
16
```

Осы мысалды дербес функциялар түрінде анықтауға болады, бірдейнәтиже береді.

Программасы:

```
def f1(x): return x ** 2
def f2(x): return x ** 3
def f3(x): return x ** 4
L = [f1, f2, f3]        # функциялар тізімі
for f in L:
   print(f(3))  # Нәтиже 9, 27, 81
print(L[0](4))          # Нәтиже 16
```

Нәтижесі:

```
9
27
81
16
```

№8. lambda-функцияларда шартты өрнекті қолдану:

Программасы:

```
fun = (lambda x, y: x if x < y else y)
d=fun(7,10)
print(d)
```

Нәтиже:

```
7
```

№9. lambda – өрнектерді фунция ішінде қолдану.

Программасы:

```
def fun(x):
  return (lambda y: x + y)
d = fun(6)
print(d)
print(d(8))
```

Нәтиже:

```
<function fun.<locals>.<lambda> at 0x03A23F58>
14
```

Осы тапсырманы басқаша анықтауға болады:

```
fun = (lambda x: (lambda y: x + y))
d= fun(15)
print(d(3))
```

Нәтиже:

```
18
```

Немесе бұл тапсырманы Python Shell немесе **терминалда** тура орындауғаболады:

```
>>> ((lambda x: (lambda y: x + y))(20))(4)
24
```

Өзін-өзі шақыратын функцияны **рекурсиялық функция** деп атайды.

№10. Натурал санның факториалын есептеу. n! = 1*2*….*(n-1)*n. Яғни n! = (n-1)!*n.

Программасы:

```
def fact(num):
    if num == 0:
       return 1
    else:
       return num * fact(num - 1)
a = fact(7)
print(a)
```

Нәтижесі:

```
5040
```

№11. Натурал n саны берілген. Есептеу керек.1-әдіс.

$$\left(1+\frac{1}{1^2}\right)\left(1+\frac{1}{2^2}\right)\ldots\left(1+\frac{1}{n^2}\right)$$

Рекурсиямен есептеу

Программасы:

```
n = int(input('n ='))
def f( n):
n =5     == 0:
f =3.069eturn 1
    return (1 + 1. / (n*n)) * f(n - 1)
print('f =%.3f' % f(n))
```

Нәтижесі:

2-әдіс. Рекурсиясыз есептеу.

Программасы:

```
n = int(input('n ='))
def f(n):
    a = 1
    for i in range(1, n+1):
        a= a* (1 + 1. / (i*i))
    return a
print('f =%.4f' % f(n))
```

Нәтижесі:

```
n =5
f =3.0694
```

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Функция дегеніміз не және оның негізгі мақсаты қандай?
2. Python тілінде функцияны анықтау үшін қандай кілттік сөз қолданылады?
3. Функцияның тақырыбы (header) мен денесі (body) қандай бөліктерден тұрады?
4. Функциядағы параметрлер мен аргументтердің айырмашылығы қандай?
5. Функциядан мәнді қайтару үшін қандай оператор қолданылады?
6. Әдепкі (default) параметр дегеніміз не? Мысал келтіріңіз.
7. *args және **kwargs параметрлерінің қызметі қандай?
8. Функцияның ішінде анықталған айнымалылардың қолжетімділік аймағы қандай?
9. Жаһандық (global) және жергілікті (local) айнымалылардың айырмашылығы неде?
10. Функция ішінде басқа функцияны қолдануға бола ма?
11. Анонимді (lambda) функцияның ерекшелігі неде?
12. Функцияға бірнеше мәнді қайтаруға бола ма? Қалай жүзеге асады?
13. Рекурсиялық функция дегеніміз не?
14. docstring не үшін қолданылады?
15. Кірістірілген (built-in) функциялар мен пайдаланушы анықтаған функциялардың айырмашылығы қандай?

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс №11

Дәріс тақырыбы: Жоғарғы дәрежелі функциялар map(), filter(), reduce()map()

Дәріс мақсаты: Жоғарғы дәрежелі функциялар map(), filter(), reduce()map() функцияларымен танысу. Практикалық тапсырмаларды орындау барысында функциялардың мүмкіндіктерін қарастыру

Кілт сөздер: map(function, iterable) , function , iterable , lambda , list(map(...)) , tuple(map(...)) , **итератор (iterator)** – map() , map(func, list1, list2, ...), (higher-order function)

Дәріс жоспары:

1. map() функциясы
2. filter() функциясы
3. reduce() функциясы
4. Практикалық жұмыс.

1. map() функциясы функциясының екі аргументі болады, функция және деректер тобы.

map(функция, деректер тобы)

map() функциясы деректер тобының әрбір мүшесіне бірінші аргументтекөрсетілген функцияны қолданады.

Мысалы:

```
1) tizim1 = ['2', '3', '7']
 tizim2 = list(map(int, tizim1)) # int() функциясы аргументті бүтін санға
түрлендіреді
 print (tizim2)              # [2, 3, 7]
```

2) map әрбір элементті квадраттайды:

```
Kvadrat = map(lambda x: x * x, [1, 3, 5])
print(Kvadrat)              # [1, 9, 25]
```

map() функциясы тізімдегі сандарға lambda x: x * x функциясын қолданады.

2. filter() функциясы

filter(функция, деректер тобы)

filter функциясы деректерді сүзгілеу үшін қолданылады. Бірінші аргументте көрсетілген функция деректер тобының әрбір мүшесіне қолданылып, нәтижесі True (ақиқат) немесе False

(жалған) болады. True мәні болған элементтер нәтижеге қосылады. Керісінше, False мәнді элементтер filter функциясының нәтижесіне кірмейді.

3. reduce() функциясы

reduce (функция, деректер тобы, алғашқы мәні)

reduce() функциясы деректер тобының әрбір мүшесіне бірінші аргументте көрсетілген функцияны қолданады және тек қана бір нәтиже береді.

Аргумент ретіндегі функцияның екі аргументі болады.

Бірінші аргумент нәтижені жинаушы болады, оның мәні келесі функция орындалған сайын сақталып отырады. Бірінші аргумент алғашқы мәнді қабыл- дайды. Егер алғашқы мән жазылмаса, онда топтағы бірінші мәнге тең болады.

Екінші аргумент топтағы деректерді орындалған сайын біртіндеп қабылдап отырады.

Бұл функция **functools** модулінде орналасқан. Сондықтан **reduce()** функциясын қолдану үшін оны модульден қосу керек.from

functools import reduce

4. Практикалық жұмыс.

№1. map() функциясы деректер тобына Kosu функциясын қолданады.Программасы:

Нәтижесі:

```
def kosu(n):
    return n + n
tizim = (3, 6, 7)
result = map(kosu, tizim)
print(type(result))
print(list(result))
<class 'map'>
[6, 12, 14]
```

№2. map() функциясын тізімдерге қолдану.

Программасы:

```
a = [2, 3, 4]
b = [18, 13, 11]
c = [-2, -5,  10]
result1=list(map(lambda x, y : x + y, a, b))
result2=list(map(lambda x, y, z : x + y + z, a, b, c))
result3=list(map(lambda x, y, z : 2.5*x + 2*y - z, a, b, c))
print(list(result1))
print(type(result1))
print(list(result2))
print(type(result2))
print(list(result3))
print(type(result3))
```

Нәтижесі:

```
<class 'list'>
[18, 11, 25]
<class 'list'>
[43.0, 38.5, 22.0]
<class 'list'>
```

№3.map() функциясы бірнеше тізімге қолданылады.

Программасы:

```
t1 = [2, 6, 3]
t2 = [3, 5, 9]
t3 = list(map(lambda x, y: x + y, t1, t2))
print (t3)
```

Нәтижесі:

```
[5, 11, 12]
```

№4.Тізімдердің ұзындықтары әртүрлі болса, онда функцияның орындалуы **қысқа тізіммен** шектеледі.

Программасы:

```
t1 = [2, 6, 3]
t2 = [3, 5]
t3 = list(map(lambda x, y: x * y, t1, t2))
print (t3)
```

```
[6, 30]
```

Нәтижесі:

№5. Деректер тобынан оң сандарды сүзіп алу.
Программасы:

Нәтижесі:

```
[7, 3, 15, 17]
```

Сүзгілеу нәтижесін 1 рет қана алу керек. Екінші рет алу бос нәтиже береді.

№6. Шартты қанағаттандыратын сандарды сүзіп алу.Программасы:

Нәтижесі:

```
def fun5(k):
   return k < 7
num = [2, 5, 3, 9, 8, 13,6]
print(list(filter(fun5, num))) # [2, 5, 3, 6]
[2, 5, 3, 6]
```

№7. Берілген аралықтағы теріс сандарды сүзіп алу.

```
print(list( filter((lambda x: x < 0), range(-5, 5))))
```

Нәтижесі:

```
[-5, -4, -3, -2, -1]
```

№8. Деректер тобынан шартты қанағаттандыратын сандарды сүзіп алу.1-әдіс.
Программасы:

```
def numbs(x):
   if -4 <= x <= 20:
      return 1
   else:
      return 0
s = [28, 15, -10, 18, -3]
for i in filter(numbs, s):
      print(i, end = ' ')
```

Нәтижесі:

```
15 18 -3
```

2-әдіс. Программасы:

```
print(list(filter(lambda x: -4 <= x <= 20, [28, 15, -10, 18, -3])))
```

Нәтижесі:

```
[15, 18, -3]
```

№9. Тізімдегі барлық сандардың қосындысын табу.Программасы:
Нәтижесі:

```
from functools import reduce
tizim = [23, 18, 48, 32]
sum = reduce(lambda s, x: s + x, tizim)
print (sum) # 121
121
```

№10. Тізімдегі барлық сандардың көбейтіндісін табу.
Программасы:

```
from functools import reduce
numbers = [8, 9, 10, 11]
s=reduce(lambda res, x: res * x, numbers, 1)     # 7920
print (s)
```

Нәтижесі:

```
7920
```

№11. Тізімдегі ең үлкен санды табу.

Программасы:

```
from functools import reduce
f = lambda a, b: a if (a > b) else b
s=reduce(f, [16, 23, 56, 11, 90, 85])
print(s)
```

Нәтижесі:

```
90
```

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. `map()` функциясының негізгі мақсаты қандай?
2. `map()` функциясының синтаксисі қалай жазылады?
3. `map()` функциясы неше аргумент қабылдайды және олардың қызметі қандай?
4. `map()` функциясы қандай типтегі мәнді қайтарады?
5. Неліктен `map()` нәтижесін көбіне `list()` немесе `tuple()` арқылы түрлендіреді?
6. `map()` функциясына бірнеше итерируемді объект беруге бола ма? Мысал келтіріңіз.
7. `map()` функциясында анонимді (`lambda`) функцияны қолданудың артықшылығы қандай?
8. `map()` функциясы мен `for` циклінің айырмашылығы неде?
9. `map()` функциясы қандай жағдайларда тиімді қолданылады?
10. `map()` функциясын `list comprehension` әдісімен салыстырып, айырмашылығын түсіндіріңіз.

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.
2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.
3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022
4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**
5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.
6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.
2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с
3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]
Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс №12

Дәріс тақырыбы: PYTHON тілінде файлдарды оқу және жазу

Дәріс мақсаты: PYTHON тілінде файлдардармен жұмыс жасау. Файлмен жұмыс жасауда қолданылатын командалармен жұмыс жасау.

Кілт сөздер: open(),close() ,with open(...) as f: ,'r' – оқу (read),'w' – жазу (write, файлдың мазмұнын өшіреді),'a' – қосып жазу (append),'b' – екілік (binary) режим,'x' – жаңа файл құру (егер файл бар болса, қате береді),'r+' – оқу және жазу (файлды өшірмей),write() – бір жол немесе мәтін жазу,writelines() – бірнеше жолды жазу,read() – файлдың толық мазмұнын оқу,readline() – файлдан бір жол оқу

Дәріс жоспары:

1. *Файл*
2. Файлды ашу түрлері (access_mode):
3. Практикалық жұмыс.

Файл – сыртқы есте сақтау құрылғыларында (магниттік немесе flash жадыларда) жазылған және мәліметтерді өңдеу, тасымалдау кездерінде біртұтас күйде қарастырылатын мәліметтер жиыны.

Файлдармен жұмыс істеу үшін оларды, алдымен, ашу керек, яғни файл туралы мәліметті – атын, адресін программаға белгілі ету қажет.

файл_айнымалысы = open(файл аты [, access_mode][, buffering])

file_name – жолымен (орналасу орнымен) немесе жолсыз жазылған файл аты

access_mode – файлды ашу түрі, файл қандай жұмыс үшін ашылу керек екенін көрсетеді (оқу, жазу және жалғастырып жазу т.с.с.).

buffering – буффер көлемі, 0 – ге тең болса буффер қолданылмайды, теріссан болса буфер көлемін операциялық жүйе белгілейді.

2. Файлды ашу түрлері (access_mode):

r – оқу үшін.

w - жазу үшін. Жаңа файл құрылады. Егер тиісті орында ондай файл барболса, ол қайтадан құрылады, яғни бұрынғы файл өшіріледі.

a – жалғастырып жазу үшін.

b – екілік файл белгісі, егер **b** көрсетілмесе файл мәтіндік есептеледі.

a+, w+ - оқу және жазу үшін.

Файл ашу белгілері жазылмаса, онда екілік файлды оқу үшін ашылды депесептеледі (rb).

file айнымалысының қасиеттері

file.closed – True егер файл жабылса, әйтпесе False

file.mode – файлдың ашылу түрі

file.name – файлдың аты **Файлдан оқу**

read(сан) әдісі файлдан саны көрсетілген байттарды оқиды. Нәтижесіоқылған деректер. Егер байт саны жазылмаса бүкіл файлды оқиды. read() әдісі файл соңы болғанда қолданылса, бос нәтиже береді. Бұл әдіс жол соңы белгісін дерек соңына қоспайды.

Файлды оқу үшін мысалы, Matyn.txt файлы мен Питонның p1.py файлын бір папкада сақтаған дұрыс. Және де мәтін кириллицада жазылған болса, онда оқыған кезде басқа белгілер шығып кетпес үшін encoding=’utf-8’ командасын қолданған дұрыс.

read () әдісі ақпаратты файлдың соңына дейін оқиды. Matyn.txt файлында жазылған ақпаратты толық оқып тұр.

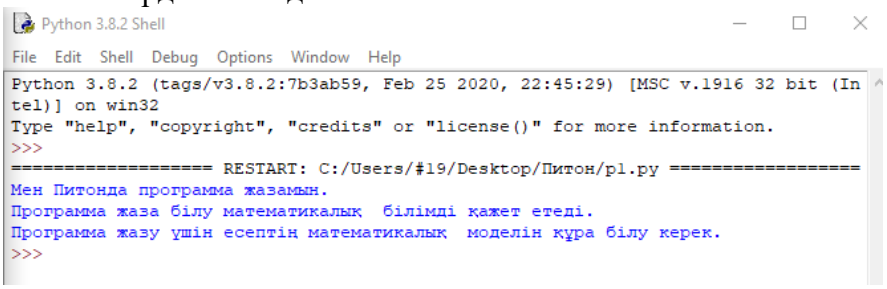
1-әдіс. Программасы:

```
p = open("Matyn.txt",encoding="utf-8",)
print ( p.read())
```

2-әдіс. Программасы:

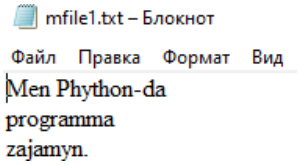
```
p = open("Matyn.txt", encoding="utf-8")
name=p.read()
print(name)
```

Нәтиже бірдей шығады:



file.readline() әдісі файлдан бір жолды оқиды. Нәтижесі оқылған жолғатең. Бұл әдіс жол соңы белгісін дерек соңына қосады. Жол соңы белгісі соңғы жолда болмайды. Егер әдіс нәтижесі бос дерек болса, бұл соңының белгісі. Егер нәтиже жол соңы белгісі болса (\n), онда ол кезекті бос жол болғаны.

Мысалы, readline()әдісі арқылы жолма жол оқуға болады. mfile1.txtфайлында мынадай мәтін жазылған:



Бұл жерде мәтін ағылшын шрифтімен жазылғандықтан encoding=’utf-8’командасын қолдану қажеті жоқ.

Программасы:

```
p = open("mfile1.txt", encoding="utf-8")
print(p.readline())
print(p.readline())
print(p.readline())
```

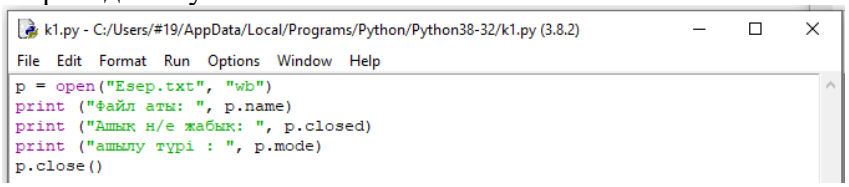
Нәтижесі:

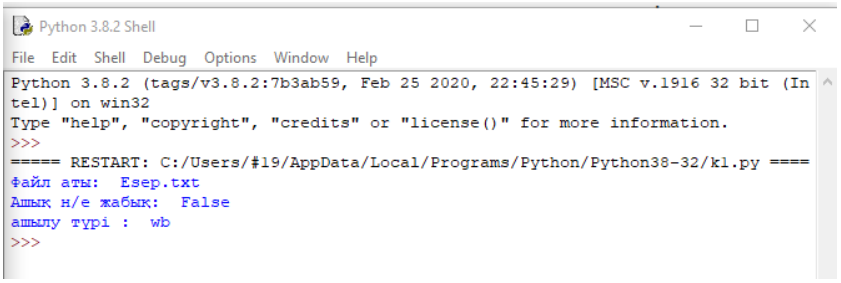
```
Men Phython-da

programma

zajamyn.
```

файлды ашу





Файлды жабу

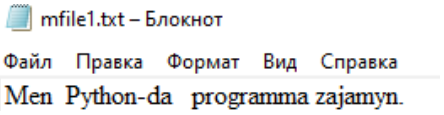
Файлмен жұмыс (оқу/жазу) аяқталған соң, файл жабылу керек. Файлclose() әдісімен жабылады. файл_айнымалысы.close();

Егер файл келесі түрде ашылса, ол міндетті түрде жабылады.with
open(файл_аты, ‘w’) as файл_айнымалысы:
файлмен жұмыс операторлары

Файлға жазу

Файлға деректі жазу үшін **write() әдісі** қолданылады. Файлға бірнеше жолды жазу үшін writelines() әдісі қолданылады.Бұл әдіс жол соңы белгілерінфайлға жазбайды.

Мысалы, mfile1.txt файлына төмендегідей ақпарат жазылған.



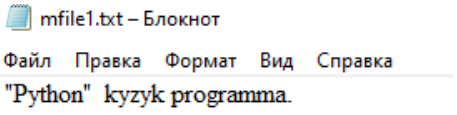
Осы файлға жаңа деректер жазып, файлды жауып және файлды қайтаданоқуға ашып, файлды жабу керек.

```
b = """ "Python"  kyzyk programma."""        # Қандайда мәтін жаздық.
fail = open("mfile1.txt", "w")          # деректі жазу үшін mfile1.txt файлын ашу
fail.write(b)             # файлға b-дағы мәтінді жазу.
fail.close()              # файлды жабу.
fail2 = open("mfile1.txt", "r")          # mfile1.txt файлын оқуға ашу.
p1 = fail2.readline()     # fail2-дегіні жол бойынша оқу.
fail2.close()      #файлды жабу.
print(p1, end= ' ')         # p1-дегі ақпаратты шығару.
```

Нәтижесі:

```
"Python"  kyzyk programma.
```

Бұл файл **w** - жазу үшін ашылғандықтан, жаңа файл құрылады, яғнибұрынғы файл өшіріледі. Ол нәтижеде көрінбейді, файлды ашып қарауға болады, мысалы mfile1.txt файлындағы ақпарат өзгерді:



Файл көрсеткіші.

Файлмен жұмыс кезінде деректі оқу немесе жазу орнын анықтаушы файлкөрсеткіші болады.

Файл көрсеткішінің әдістері.

Файл көрсеткішінің мәнін *tell()* әдісі арқылы білуге болады.

Файл көрсеткішінің орнын *seek(offset[, from])* әдісі арқылы белгілеугеболады.

offset аргументі файл көрсеткішінің неше байтқа ауысу керек екенін анықтайды.

from аргументі қай орыннан бастап есептелінетінін анықтайды. Егер from аргументі 0 болса, файлдың басынан бастап есептеледі. Егер бұл аргумент 1-ге тең болса, файлдың соңынан бастап есептеледі.

Мысалы, төмендегі программа арқылы mfile2.txt файлы ашылады, fail KorsetKishi tell() adisi arKyly anyKtalady ақпараты жазылады, бұл бізге көрінбейді, файлды ашып қарауға болады. Программасы:

```
with open('mfile 2.txt', 'w+') as f: #  mfile2.txt   файлы оқу және жазу үшін аш
   f.write("fail korsetkishi")
   f.tell()      # Файл көрсеткішінің мәні-16
   f.write(' tell() adisi')
   f.tell()   # Файл көрсеткішінің мәні-29
   f.write(' arkyly anyktalady')
   f.tell()    # Файл көрсеткішінің мәні-47
   f.seek(0)   #Файл    көрсеткішінің   орны
   d = f.read()
print(d)
```

Нәтижесі:

```
 fail korsetkishi tell() adisi arkyly anyktalady
```

3. Практикалық жұмыс.

№1. Файлды жол бойынша оқу.

Программасы:

```
fun = open("mfile1.txt", "r")
for a in fun:
  print(a, end = ' ')
fun.close()
```

Нәтижесі:

```
"Python"  kyzyk programma.
programma
identifikator
transliator
interpretator
```

№2. file.readlines() әдісі файлдан барлық жолды оқиды.

Программасы:

```
f = open('mfile1.txt')
for k in f.readlines():
  print (k)
```

Нәтижесі:

```
Phython

programmasyn

bilemin.
```

№3. Мәтіндік файл құру және оған мәліметтер жазу.

Мәтіндік файл жасау, оған қолданушы енгізетін деректерді жол-жолға жазу. Кірістің соңы бос жол болсын.

Ең алдымен, мәтіндік файл жасап, оны жазуға ашу керек. Файл жазбаға ашылғаннан кейін сіз шексіз циклды бастай аласыз, онда жол пернетақтадан оқылып, содан кейін файлға жазылады. Цикл денесінде бос жолды тексеруболуы керек: егер бос жол енгізілсе, цикл үзілуі керек.

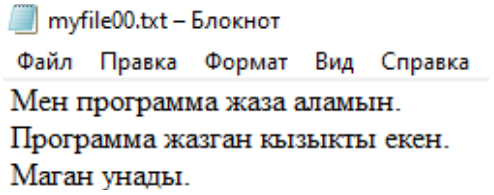
Программасы:

```
fname = input('Файл: ')
f = open(fname,'w')
while True:
    s = input()
    if s == '': break
    f.write(s+'\n')
f.close()
```

Нәтижесі:

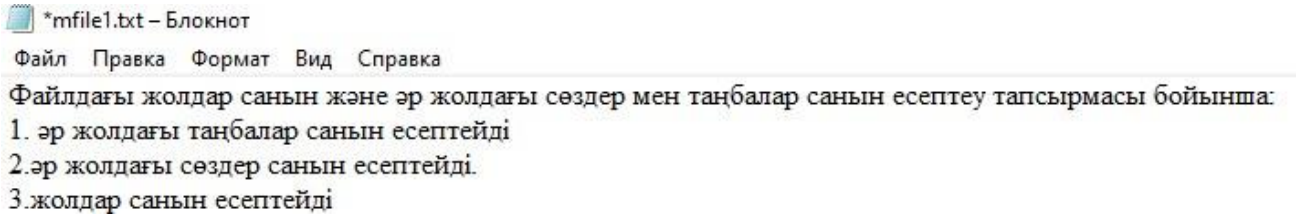
```
Файл: C:\Users\#19\Desktop\myfile00.txt
Мен программа жаза аламын.
Программа жазған қызықты екен.
Маған унады.
```

Программа бойынша алдымен жазуға ашылған файлдың қайда орналасқа- нын (адресін) көрсету керек. Пернетақтадан мәтін енгіземіз. Енгізілген мәтін жазуға ашып қойған файлға жазылады.



№4. Файлдағы жолдар санын және әр жолдағы сөздер мен таңбалар санын есептеу.

Файлға жазылған мәтін:



Программасы:

```
f = open("mfile1.txt",encoding="utf-8",)
line = 0
for i in f:
    line += 1

    flag = 0
    word = 0
    for j in i:
        if j != ' ' and flag == 0:
            word += 1
            flag = 1
        elif j == ' ':
            flag = 0

    print(i,len(i),'-таңбалар саны',word,'-сөздер')

print(line,'-жолдар саны')
f.close()
```

Нәтижесі:

```
Файлдағы жолдар санын және әр жолдағы сөздер мен таңбалар санын есептеу тапсырмасы бойынша:
 92 -таңбалар саны 13 -сөздер
1. жолдар санын есептейді
 26 -таңбалар саны 4 -сөздер
2.әр жолдағы сөздер санын есептейді.
 37 -таңбалар саны 5 -сөздер
3.әр жолдағы таңбалар санын есептейді. 38 -таңбалар саны 5 -сөздер
4 -жолдар саны
```

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Python тілінде файлмен жұмыс жасау үшін қандай функция қолданылады?
2. `open()` функциясының негізгі параметрлері қандай?
3. Файлды ашу режимдері қандай және олардың айырмашылығы неде? (`r`, `w`, `a`, `b`, `x`, `r+`)
4. Файлды ашқаннан кейін неге оны жабу қажет?
5. Файлды жабу үшін қандай әдіс қолданылады?
6. Файлды автоматты түрде жабу үшін қандай құрылым пайдаланылады?
7. Файлдан барлық мәтінді оқу үшін қандай әдіс қолданылады?
8. Файлдан бір жолды оқу үшін қандай әдіс пайдаланылады?
9. Файлдағы барлық жолдарды тізім түрінде оқу үшін қандай әдіс қолданылады?
10. Файлға бір жол мәтін жазу үшін қандай әдіс қолданылады?
11. Файлға бірнеше жолды жазу үшін қандай әдіс қолданылады?
12. `seek()` және `tell()` әдістерінің қызметі қандай?
13. Файлды оқу немесе жазу кезінде `encoding="utf-8"` параметрін көрсету не үшін қажет?
14. Егер файлды `'w'` режимінде ашсақ, бұрынғы деректерге не болады?
15. `'a'` режимі `'w'` режимінен несімен ерекшеленеді?

Негізгі әдебиеттер:

1. 1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.
2. 2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.
3. 3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022
4. 4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с.
5. 5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.
6. 6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

7. 1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.
8. 2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с
9. 3.Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]
10. Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б
11.

Дәріс №13

Дәріс тақырыбы: PYTHON тілінде алгоритмдер

Дәріс мақсаты: PYTHON тілінде алгоритмдермен жұмыс жасау.

Кілт сөздер: Іздеу, жою. Сұрыптау, минималды іздеңіз, индекстерді анықтау.

Дәріс жоспары:

1. Алгоритм

2. Сұрыптау, минималды іздеңіз, индекстерді анықтау.
3. Тізімді толығымен қопару:
4.

1. Алгоритм. Алдыңғы тақырыптарда біз Python бағдарламашы жұмысын қамтамасыз ететін негізгі деректер түрлерін (сыныптарды) қарадық. Алгоритмдер туралы бірнеше сөз айтайық.

Алгоритм - тапсырманы орындау үшін қажет қадамдардың соңғы жиынтығы, мысалы, шайды қайнату алгоритмі немесе дүкенге бару алгоритмі. Бағдарламада және әрбір бағдарламада әрбір функция бағдарламалау тілінде жазылған нақты алгоритмді енгізу болып табылады.

Мысалы, келесі деректер жинағындағы ең кіші элементтің орнын табуымыз керек: 809, 834, 477, 478, 307, 122, 96, 102, 324, 476.

Ең алдымен, сақтау үшін тиісті деректер түрін таңдаймыз. Әлбетте, бұл тізім болады:

```
>       > > counts = [809, 834, 477, 478, 307, 122, 96, 102, 324, 476]
>       > > counts.index(min (counts)) # тапсырманы бір жолға шешу! 6
>>>
```

Тапсырманы қиындатып, сұрыпталмаған тізімдегі екі ең кіші элементтің орнын табуға тырысамыз.

Алгоритмдердің шешімі қандай болуы мүмкін?

1. Іздеу, жою, іздеу. Тізімдегі ең төменгі элементтің іздеу индексін алып тастаймыз, қайтадан минималды іздеңіз, жойылған элементті тізімге қайтарамыз.
2. Сұрыптау, минималды іздеңіз, индекстерді анықтау.
3. Тізімді толығымен қопару. Әр элементті тәртіппен салыстырамыз, ең кішкентай мәндерді аламыз, ең кіші мәндер табылғанда мәндерді жаңартамыз.

Аталған алгоритмдердің әрқайсысын қарастырайық.

1. Іздеу, жою, іздеу. Тізімдегі ең төменгі элементтің іздеу индексін алып тастаймыз, қайтадан минималды іздеңіз, жойылған элементті тізімге қайтарамыз.

Бастайық:



Тізімнен ең аз элементті алып тастаймыз. Бұл жағдайда, жаңартылған тізімдегі индекстер жылжытылады:



Жойылған (бірінші минималды) элементті тізімге қайтарамыз:



Бірінші минималды элементті алып тастағаннан кейін индекстерді ауыстыру туралы ұмытпаңыз: екінші минималды элементтің индексі бірінші минималды элементтің индексіне тең, сондықтан біз екінші минималды элементтің индексін 1-ге көбейтеміз.

Бұл алгоритмді іске асыратын функция:

```
def find_two_smallest(L):
smallest = min(L)
min1 = l. index(smallest)
L. remove (smallest) # бірінші минималды элементті жоямыз
next_smallest = min(L)
min2 = l. index(next_smallest)
L. insert (min1, smallest)# бірінші минималдыны кері қайтарамыз if min1 < =
min2: # ығысудын кесірінен екінші минималдының
индексін тексереміз
min2 += 1 # min2 = min2 + 1
return (min1, min2) # кортежді қайтару
```

2. Сұрыптау, минималды іздеңіз, индекстерді анықтау.

Екінші алгоритмді енгізу интуитивті түсінікті, сондықтан функцияның бастапқы кодын ғана береміз:

```
def find_two_smallest(L):
temp_list = sorted (L) # сұрыпталған тізімнің көшірмесін қайтарамыз smallest =
temp_list[0]
next_smallest = temp_list[1] min1 =
l. index(smallest) min2 = l.
index(next_smallest) return (min1, min2)
```

3. **Тізімді толығымен қопару**:

Әр элементті тәртіппен салыстырамыз, ең кішкентай мәндерді аламыз, ең кіші мәндер табылғанда мәндерді жаңартамыз.

Үшінші алгоритм - жоғарыда аталғандардың ең күрделісі, сондықтан біз оны толығырақ қарастырамыз.

Адамға қарағанда,ол толық тізімді бірден қамтуы мүмкін және ол қандай элементтің минималды екенін бірден айта алады, компьютерде ондай ақыл жоқ. Машина элементтерді бір-бірлеп сканерлейді, элементтерді жүйелі түрде салыстырады және салыстырады.

Бірінші қадамда тізімнің алғашқы екі элементін қараймыз:



809 және 834-ті салыстырыңыз және олардың минималды min1 және min2 мәндерін орнатыңыз, мұнда бірінші минималды және екінші минималды элементтердің көрсеткіштері сақталады. Содан кейін 2 индексінен бастап тізімнің соңына дейін элементтерді қайта қопарамыз:



809 бірінші минималды, ал 834 тізімнің алғашқы екі табылған мәнінің екінші минималды элементі болып табылатынын анықтады.

Төмендегі тізім элементін қараңыз (477):



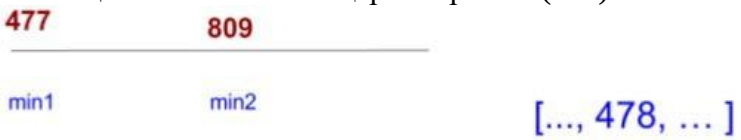
477 элементі бәрінен аз болды (шартты түрде бұл " бірінші нұсқа»):



Сондықтан min1 және min2 айнымалыларының мазмұнын жаңартамыз, өйткені біз жаңа ең кіші элементті таптық:



Тізімнің келесі элементін қарастырамыз (478):



Ол екі ең төменгі элементтің арасында болды (шартты түрде "екінші" деп атаймыз):



Ең төменгі элементтерді қайта жаңартамыз (енді тек min2 жаңартылды):



Осылай тізімнің соңына жеткенше болады:



Ұсынылған алгоритмді іске асыратын функцияның бастапқы коды: def find_two_smallest(L):

if L [0] < L[1]:
min1, min2 = 0, 1 # Бастапқы мәндерді орнатамыз else:
min1, min2 = 1, 0

for i in range(2, len(L)):
if L[i] < L[min1]: # " бірінші нұсқа» min2 = min1
min1 = i
elif L [i] < L[min2]: # " екінші нұсқа» min2 = i
return (min1, min2)

11.1 жаттығу

Ең үлкен және ең үлкен арасындағы айырмашылықты қайтаратын функцияны жазыңыз.Бүтін кездейсоқ сандардың тізімінен ең аз мәндермен.

11.2 жаттығу

1000 кездейсоқ тізімнің бүтін саны үшін бағдарламаны жазыңыз оның арасында қанша теріс элементтер орналасқанын анықтайды максималды және минималды элементтер.

11.3 жаттығу

Тізімнің барлық элементтерінің орташа мәніне ең жақын элементті табу.

11.4 жаттығу

Саннан тұратын тізім берілген. Тізімдегі қарапайым сандардың сомасын табу.

11.5 жаттығу

Бүтін сандардың тізімі берілген. Онда кем дегенде бір жұп көрші екенін анықтау тақ сандар. Оң жауап болған жағдайда бірінші элементтердің нөмірін анықтау мұндай жұп.

11.6 жаттығу

Бүтін сандардың тізімі берілген. Жұп элементтердің санын және санын анықтау 5 цифрмен аяқталатын элементтердің саны.

11.7 жаттығу

Бүтін сандардың тізімі берілген. Элементтердің пайыздық құрамын анықтау, тізімнің барлық элементтерінің орташа арифметикалық мәнінен асатын.

11.8 жаттығу

Бүтін сандардың тізімі берілген. Тізімдегі учаскелердің санын анықтау элементтер біркелкі өседі (әрбір келесі Сан алдыңғы саннан артық).

11.9 жаттығу

20 элементтен тұратын тізім берілген. Бес көрші элементті табу, мәндер сомасы ең көп.

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Алгоритм ұғымы
2. Бағдарлама жасау қадамдары
3. Алгоритм ұғымы
4. Сұрыптау
5. Индексті анықтау

Ұсынылатын әдебиеттер:

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с.

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.-832с

3. Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704

стр.[pic 1]
4. Бекенова А.С., Python бағдарламалау тілі., Оқу құралы.,

Дәріс №14

Дәріс тақырыбы: Python- да обьектіге бағытталған бағдарламалау

Дәріс мақсаты: Python- да обьектіге бағытталған бағдарламалауды пайдаланып студенттерді қарапайым есептерді алгоритмдеу дағдыларына қалыптастыру.

Кілт сөздер: Объект (Object), Класс (Class)**,** Экземпляр (Instance)**,** Атрибут (Attribute), Әдіс (Method)**,** Конструктор (**__init__**)**,** Инкапсуляция (Encapsulation)**,** Мұрагерлік (Inheritance)**,** Көптүрлілік (Polymorphism)**,** Абстракция (Abstraction)**, self** кілттік сөзі**, super()** функциясы**,** Деректер мүшелері (Data members)

Дәріс жоспары:

1. Python объектілі-бағдарланған бағдарламалаудың негізгі ұғымдар мен түсініктері
2. Python мұрасы
3. Python мұрагерлік иерархиясы

1. **Python объектілі-бағдарланған бағдарламалаудың негізгі ұғымдар мен түсініктері**

Бұған дейін біз Python объектілі-бағдарланған бағдарламалау тілі болып табылатының айттық, бірақ бұл мағынасын толық қарастырған жоқпыз. Бұл сұраққа қайта оралайық, мысалдан бастайық.

Кейбір адамның мекен-жайын сипаттау үшін жол айнымалылары бар деп болжаймыз:

```
addr_name = 'Ivan Ivanov' # адамның аты addr_line1
= '1122 Main Street'
addr_line2 = ''
addr_city = 'Panama City Beach' addr_state
= 'FL'
addr_zip = '32407' # индекс
```

Адам туралы барлық ақпаратты экранға шығаратын функцияны жазамыз:

```
def printAddress(name, line1, line2, city, state, zip):
print (name)
if len (line1) > 0:
print(line1)
if len (line2) > 0:
print(line2)
print (city+", "+state+" +zip)
# Функцияны шақыру, аргументтерді жіберу:
printAddress(addr_name, addr_line1, addr_line2, addr_city, addr_state,
addr_zip)
```

Бағдарлама жұмысының нәтижесінде:

```
>>>
======== RESTART: C:/Python35-32/addr.py ========
Ivan Ivanov
1122 Main Street
Panama City Beach, FL 32407
>>>
```

Бастапқы шарттар өзгергенін және мекен-жайдағы адамның екінші индексі бар дейік. Неге жоқ? Жаңа айнымалыны құрайық:

```
# индексті қамтитын айнымалыны қосыңыз
addr_zip2 = "678900"
```

Жаңа мәліметтерді ескере отырып, printAddress() функциясын өзгертіңіз:

```
def printAddress(name, line1, line2, city, state, zip, zip2): # zip2
параметрін қосты
print (name)
if len (line1) > 0:
print(line1)
if len (line2) > 0:
print(line2)
# Zip2 айнымалы экранға шығаруын қосты print
(city+","+ state +" + zip + zip2)
# Addr_zip2 жаңа аргументін қосты:
printAddress (addr_name, addr_line1, addr_line2, addr_city, addr_state, addr_zip,
addr_zip2)
```

PrintAddress () функциясы жаңа шарттарда дұрыс жұмыс істейтін етіп жаңа индексті бірнеше рет қосу керек болды. Қарастырылған тәсілдің қандай да бір кемшілігі бар? Көп айнымалылар! Өндегіңіз келетін адам туралы көбірек ақпарат алу үшін, бізге қажет айнымалылар қажет. Әрине, сіз бәрін тізімге енгізе аласыз (тізім элементтері содан кейін жолдар болады), бірақ Python-да объектілерге бағытталған әртүрлі деректер жиынтығымен жұмыс істеу үшін әмбебап тәсіл бар.

Адамның мекен-жайы туралы барлық ақпаратты қамтитын аты-жөні Address бар деректер құрылымын (сыныпты) жасаңыз:

```
class Address (): # сыныптын атауын бағдарламашы таңдайды name = ""
# сынып өрістері
line1 = ""
line2 = "" city = ""
state = "" zip = ""
```

Сынып мекенжайды сақтауға арналған үлгісін орнатады. Address класының объектісін (данасын) жасау арқылы үлгіні нақты мекенжайға айналдыра аласыз:

```
homeAddress = Address()
```

Енді объектінің өрістерін нақты мәндермен толтыра аламыз:

```
# homedress нысанның name өрісін толтырыңыз:

homeAddress.name = " Ivan Ivanov"
homeAddress.line1 = " 701 N. C Street" homeAddress.line2 =
"Carver Science Building" homeAddress.city = "Indianola"
homeAddress.state = "IA"
homeAddress.zip = " 50125"
```

Сол адамның қала сыртындағы үйі туралы ақпаратты қамтитын Address класының тағы бір нысанын жасаймыз:

```
# айнымалы Address класы нысанының мекенжайын қамтиды: vacationHomeAddress =
Address()
```

Vacationhomeaddress айнымалысында орналасқан нысанның өрісін анықтаймыз:

```
vacationHomeAddress.name = " Ivan Ivanov"
vacationHomeAddress.line1 = "1122 Main Street"
vacationHomeAddress.line2 = ""
vacationHomeAddress.city = "Panama City Beach"
vacationHomeAddress.state = " FL"
vacationHomeAddress.zip = " 32407"
```

Негізгі және қала сыртындағы мекен-жайлар үшін қала туралы ақпаратты экранға шығарамыз (объектілердің аттарын көрсету арқылы):

```
print ("негізгі мекенжайы" + homeAddress.city)
print ("қала сыртындағы үй мекенжайы" + vacationHomeAddress.city)
```

Объект туралы алынған білімді ескере отырып, printAddress()функциясының бастапқы мәтінін өзгертеміз:

```
def printAddress (address): # нысан функциясына жібереміз
print(address.name) # нысанның өрісін экранға шығарамыз if
len(address.line1) > 0:
print(address.line1)
if len(address.line2) > 0:
print(address.line2)
print(address.қала + ", " + address.state + " " + address.zip)
```

Егер homeAddress және vacationHomeAddress объектілері бұрын құрылған болса, printAddress () функциясын аргумент ретінде тапсыру арқылы олар туралы ақпаратты көрсете аламыз:

```
printAddress(homeAddress)

printAddress(vacationHomeAddress)
```

Бағдарламаны орындау нәтижесінде:

```
>>>
========= RESTART: C:/Python35-32/addr2.py ==========
Ivan Ivanov
701 N. C Street
Carver Science Building
Indianola, IA 50125 Ivan
Ivanov
1122 Main Street
Panama City Beach, FL 32407
>>>
```

Класстар мен нысандардың мүмкіндіктері бір атаумен айнымалы бірлестіктермен шектелмейді, яғни объектінің күйін сақтау. Сыныптар сонымен қатар, өзіңіздегі функцияларды (әдістер) сынып өрістерімен жұмыс істеу үшін анықтауға мүмкіндік береді, яғни объектінің мінез- құлқына әсер ету үшін.

Dog сыныбын құрайық:

```
class Dog():
age = 0 # иттің жасы
name = "" # иттің аты
weight = 0 # иттің салмағы
# Кез келген әдістің бірінші дәлелі әрқашан self болып табылады, яғни нысанның өзі
```

```
    def bark (self): # сынып ішіндегі функция әдіс деп аталады #
self.name- ағымдағы иттің объектінің аттын сілтемесі print(self.name, "
Гав дейді")
    # Dog сыныптың myDog объектісін жасау:
    myDog = Dog()

    # MyDog нысан өрістеріне мән беру:
    myDog.name = "Spot" # құрылған иттің атын ойлап табамыз myDog.weight = 20 #
иттің салмағын көрсетеміз
    myDog.age = 1 # иттің жасы
    # Bark() нысаны myDog әдісін шақырыңыз, яғни итке дауыс беруді сұраймыз:

    myDog.bark()
    # MyDog.bark () деп аталатын толық нысаны болады: Dog.bark (myDog),
    # яғни толық нысаны бірінші аргумент ретінде объектіні қажет етеді – self Бағдарлама
жұмысының нәтижесі:
    ========== RESTART: C:/Python35-32/ndog.py ========
    Spot гав дейді
    >>>
```

Бұл мысал нақты өмірге жақын объектілер құрылғанда бағдарламалауда объектілі-бағытталған тәсілді көрсетеді. Нысандар арасында әдістерді шақыру арқылы өзара іс-қимыл жасалады. Объектінің өрістері (ауыспалы) оның жай-күйін белгілейді, ал әдісті шақыру объектінің реакциясына және/немесе оның жай-күйінің өзгеруіне (объект ішіндегі айнымалылардың өзгеруіне) әкеледі.

14.1 жаттығу

Cat класын жасау. Name (ат), color (түс) және weight (салмақ) атрибуттарын анықтаңыз. Meow деп аталатын әдісті қосыңыз. Cat сыныптары нысандарын жасаңыз, атрибуттар орнатыңыз, meow әдісін шақырыңыз.

Алдыңғы мысалда Dog класындағы myDog нысанын жасау және оған атау беру (my Dog.name= "Spot") біраз уақыт өтті. Мүмкін, программист атын көрсетуді ұмытпайды, содан кейін ит атаусыз болады – біз оған жол бере алмаймыз! Мұндай қатені болдырмау үшін арнайы әдіс (конструктор) мүмкіндік береді, ол берілген сынып объектісін құру кезінде бірден туындайды.

Алдымен конструктор жұмысын жалпы түрде қарастырайық:

```
    class Dog():
    name=""
    # Конструктор осы түрді жасау кезінде шақырылады;
    # Python-ның арнайы әдісі, сондықтан екі төменгі асты сызуы def _ init
_ (self):
    print ("Жаңа ит дүниеге келді!")
    # Ит жасаңыз (объект myDog сынып Dog) myDog =
Dog()
```

Бағдарламаны іске қосамыз:

```
>>>
========== RESTART: C:/Python35-32/dog1.py ==========
Жаңа ит дүниеге келді!
>>>
```

Сынып конструкторының шақыруы арқылы ит атын беру үлгісін қарастырайық:

```
class Dog():
name = ""
# Конструктор
# Осы түрді жасау кезінде шақырылады def _
_ init_(self, newName):
self.name = newName

# Итті жасайық және оның атын орнатамыз:
myDog = Dog ("Spot")
# Иттің атын шығару, оның орнатылғанына көз жеткізу
print(myDog.name)

# Келесі команда қате береді, себебі #
конструкторға аты берілмеген
# herDog = Dog()
```

Бағдарлама жұмысының нәтижесі:

```
>>>
========= RESTART: C:/Python35-32/dog2.py =========
Spot
>>>
```

Енді иттің атауы оның құрылу уақытында тағайындалады. Конструкторда self.name көрсетілді, себебі конструктор өз кезегінде өздігінен емес, белгілі бір объект ауыстырылды, яғни myDog.

Алдыңғы мысалда, иттің атына сілтеме жасау үшін, біз myDog.name өрісін көрсеттік, яғни нақты әлем тіліне аудардық, біз объектінің ішіне кіріп, сол жерден алынған ақпаратты алдық. Бұл қорқынышты болады, сондықтан біз иттің объектісі (setName және getName) атымен жұмыс істеу үшін

«адамгершілік» әдістерді ұсынамыз:

```
class Dog():
name = ""
# Конструктор осы сынып нысанын жасау кезінде шақырылады def _ _
init_(self, newName):
self.name = newName
# Кез келген сәтте әдісті шақыра және ит атын өзгерте аламыз def
setName (self, newName):
self.name = newName
# Кез келген сәтте әдісті шақыра аласыз және ит атын біле аласыз def
getName(self):
return self.name # нысанның ағымдағы атын қайтарады

# Бастапқы атымен ит жасаңыз:
myDog = Dog ("Spot")
```

```
# Иттің атын шығарамыз:
print(myDog.getName())

# Жаңа ит атын орнатамыз:
myDog.setName ("Sharik")

# Аты өзгерісін көрейік:
print(myDog.getName())
```

Барлығы жұмыс істейтінін тексереміз:

```
>>>
========== RESTART: C:/Python35-32/dog3.py ==========
Spot Sharik
>>>
```

14.2 жаттығу

1. Animal класын сипаттайтын кодты жазыңыз:

- жануардың атрибутын қосыңыз
- "Ням-ням" шығаратын, eat () әдісін қосыңыз
- getname() және setname() әдістерін қосыныз
- «Жануардың аты Гррр дейді» шығаратын, makeNoise() әдісін қосыңыз
- «жануардың аты жануар туған» шығаратын, animal класының конструкторын қосыңыз

2. Негізгі бағдарлама:

- жануарды жасаңыз, құру кезінде оның атын анықтаңыз
- getname() әдісін шақыру арқылы жануардың атын анықтаңыз
- setName() әдісіне шақыру арқылы жануардың атын өзгерту
- жануарға арналған eat() және makeNoise() шақырыңыз

14.3 жаттығу

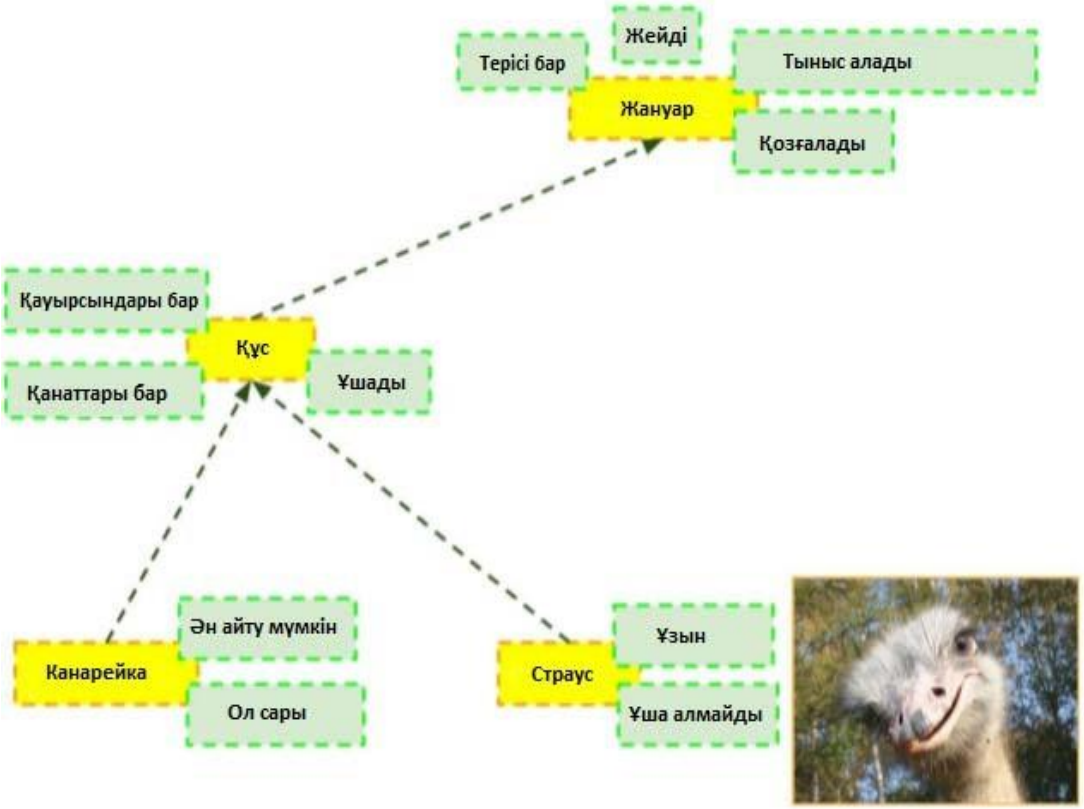
Set () және get () әдістерін қамтитын жол деректері түрімен жұмыс істеу үшін StringVar сыныпты жасаңыз. Set () әдісі жолдың мазмұнын өзгерту үшін пайдаланылады, get () жолдың мазмұнын алу үшін қолданылады. StringVar түрінің нысанын жасаңыз және оның әдістерін тексеріңіз.

14.4 жаттығу

Координаттармен (x, y) жұмыс істеуге мүмкіндік беретін Point сыныбын жасаңыз. Қажетті класс әдістерін қосыңыз.

2. Python мұрасы

Бағдарламалаудағы объектілі-бағытталған тәсіл адамның ойлауымен, оның жадымен тығыз байланысты. ООП қасиеттерін жақсы түсіну үшін адам жадынан ақпаратты сақтау және шығару моделін қарастырамыз (модельді ғалымдар Коллинз және Квиллиан ұсынған). Өз тәжірибесінде олар канарейка туралы білім ұсынылған семантикалық желіні қолданды:

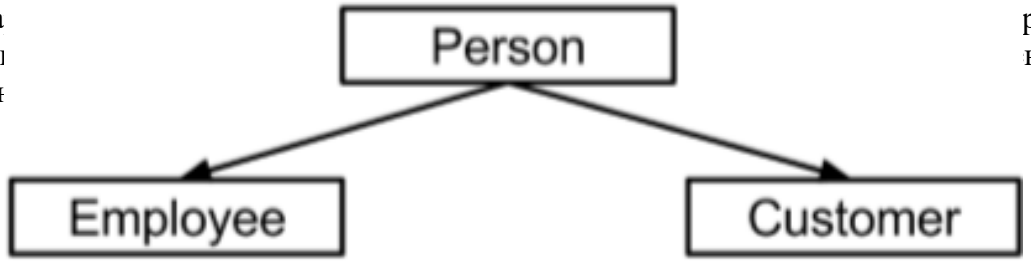


Сурет 11. Семантикалық желі

Мысалы, "канарейка-бұл ән айта алатын Сары құс"," құстардың қауырсындары мен қанаттары бар, ұша алады " және т. б. бұл желіде Білім түрлі деңгейлерде берілген: төменгі деңгейде жеке білім, ал жоғарғы деңгейде — жалпы. Мұндай тәсілде "Канарейка ұша алады" деген пікірді түсіну үшін канарейка құстардың көп бөлігіне жататыны туралы ақпаратты жаңғырту қажет және құстарда "ұша" жалпы қасиеті бар, ол канарейкаға да таралады (мұра). Зертханалық эксперименттер адамдардың " Канарейка — бұл құс?", "Канарейка ұшуға болады?"немесе" Канарейка ән айта алады?"уақыт бойынша ерекшеленеді. Сұраққа жауап " канарейка ұша ала ма?"канарейка ән айту мүмкін бе"деген сұраққа қарағанда көп уақытты талап етеді. Коллинз мен Квиллианның пікірінше, бұл ақпарат адамның ең абстрактілі деңгейде есте қалатынына байланысты. Әрбір құстың барлық қасиеттерін есте сақтаудың орнына, адамдар ерекше ерекшеліктерді ғана есте сақтайды, мысалы, сары түс және қанарда ән айту қабілеті, ал барлық басқа қасиеттер абстрактілі деңгейлерге ауыстырылады: канарейка құс сияқты ұша алады және қауырсынмен жабылған; құстар жануарлар бола отырып, дем алады және тамақтанады және т. б. шын мәнінде, "канарейка дем ала ма?"көп уақытты қажет етеді, өйткені адам өз жадында түсініктердің иерархиясы бойынша жүруі керек. Екінші жағынан, нақты қасиеттер жалпы жабуы мүмкін, бұл да ақпаратты өңдеуге аз уақытты талап етеді. Мысалы, "страус ұша алады ма" деген сұрақ "Қанаттың түйеқұйрығы бар ма?"немесе" тыныс ала ма?".

Жоғарыда аталған мұра қасиеті объектілі бағдарлы бағдарламалауда өз көрінісін тапты.

Мысалы, қызметкер (Employee) мен клиент (Customer) сыныптарының сипаттамасы бар бағда рге ие, сондықта ншілес кластарын



Сыныптар иерархиясын сипаттайтын код төменде берілген: class Person():
name = "" # кез келген адамның аты class Employee(Person):
job_title = "" # қызметкер лауазымының атауы
class Customer(Person):
email = "" # клиент поштасы
Кластар негізінде нысандарды құрайық және олардың өрістерін толтырамыз:

johnSmith = Person()
johnSmith.name = "John Smith"

janeEmployee = Employee()
janeEmployee.name = "Jane Employee" # өрісі Person сыныбынан мұра janeEmployee.job_title = "Web Developer"

bobCustomer = Customer()
bobCustomer.name = "Bob Customer" # өрісі Person класынан мұра bobCustomer.email = "send_me@spam.com"

Employee және Customer кластарының объектілерінде Person сыныбынан алынған name өрісі пайда болды.
Базалық кластың өрістерінен басқа әдістерді мұраға алады:

class Person():
name = ""
def _ _ init _ (self): # базалық сынып конструкторы print ("құрылған адам")

class Employee(Person):
job_title = ""
class Customer(Person):
email = ""
johnSmith = Person()
janeEmployee = Employee()
bobCustomer = Customer()
Бағдарлама жұмысының нәтижесі:
>>>
========= RESTART: C:\Python35-32\person.py ========= Адам құрылды

Адам құрылды Адам құрылды
>>>

Осылайша, объектілерді құру кезінде базалық класстан мұра болған конструктор шақырылады. Егер еншілес кластарда өз әдістері болса, олар орындалады:

```
class Person():
name = ""
def _ _ init _ (self): # базалық сынып конструкторы print
("құрылған адам")

class Employee(Person):
job_title = ""
def _ _ init _ (self): # еншілес кластың конструкторы print
("құрылған қызметкер")

class Customer(Person):
email = ""
def _ _ init _ (self): # еншілес кластың конструкторы print
("сатып алушы құрылды")
johnSmith = Person()
janeEmployee = Employee()
bobCustomer = Customer()
```

Бағдарлама жұмысының нәтижесі:

```
>>>
========== RESTART: C:\Python35-32\person.py ==============
Адам құрылды Қызметкер құрылды
Сатып алушы құрылды
>>>
```

Нысанды құру кезінде еншілес кластағы конструктор шақырылатынын көріп отырмыз, яғни еншілес кластағы конструктор базалық кластағы конструкторды қайта анықтады.

Кейде еншілес класты конструктордан базалық класты конструкторды шақыру қажет:

```
class Person():
name = ""
def _ init _ (self):
print ("құрылған адам")
class Employee(Person):
job_title = ""
def _ init _ (self):
Person._ init _ _ (self) # базалық сынып конструкторын шақырамыз print
("құрылған қызметкер")
class Customer(Person):
email = ""
def _ init _ (self):
Person._ init _ _ (self) # базалық сынып конструкторын шақырамыз print
("сатып алушы құрылды")

johnSmith = Person()
```

janeEmployee = Employee()
bobCustomer = Customer()

Бағдарлама жұмысының нәтижесі:

>>>
========== RESTART: C:\Python35-32\person.py ==========
Адам құрылды Адам құрылды Қызметкер құрылды Адам құрылды
Сатып алушы құрылды
>>>

14.5 жаттығу

1. Animal класын сипаттайтын кодты жазыңыз:

a) жануардың атрибутын қосыңыз.

b) "Ням-ням"шығаратын eat () әдісін қосыңыз.

c) getName () және setName () әдістерін қосыңыз.

d) "жануардың аты Гррр айтады"деп шығаратын makeNoise() әдісін қосыңыз.

e) "жануар туылды"атты Animal класының конструкторын қосыңыз.

2. Animal cat сынып үшін ата-ана болсын. Cat класының makeNoise() әдісі "жануардың аты Мяу дейді". Cat класының конструкторы "туған мысық" шығарады, сондай-ақ ата-аналық конструктор шақырады.

3. Animal Dog сыныбы үшін ата-ана болсын. Dog үшін makeNoise() әдісі "жануардың аты Гав дейді". Dog конструкторы "ит туылды", сондай-ақ ата-ана конструкторын шақырады.

4. Негізгі бағдарлама. Мысық, екі ит және бір қарапайым жануар. Әрбір жануарға атау беріңіз (әдістерді шақыру арқылы). Әрбір жануар үшін eat() және makeNoise() тудыратын код.

3. Python мұрагерлік иерархиясы

Python-да барлық құрылған сыныптар object сыныбынан мұра. Point классын құрамыз(өзіңіздің жеке деректер түрі), арнайы әдістерді анықтаймыз(object базалық сынып әдістерін қайта анықтаймыз) __init__(),
__eq__(), __str__():

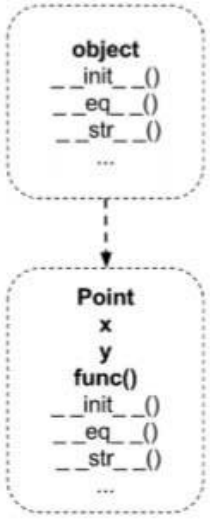
class Point:
def _ init_(self, x=0, y=0): # конструктор координаталарын орнатады self.x = x
self.y = y
def _ _ eq_(self, other): # екі нүктені салыстыру әдісі return
self.x = = other.x and self.y = = other.y
def _ _ str _ (self): # ақпаратты жол шығару әдісі return "
({0.x}, {0.y})".format(self)
def func (self): # келесі мысалда қажет
болады return abs(self.x-self.y)
a = Point () # әдепкі нысан жасаймыз x=0, y=0
print(str (a)) # мұнда Point классы ____ str_____ () әдісі шақырылады
Point толық нысаны . __ str(a)
b = Point (3, 4)

```
print(str(b))
b.x = -19
print (a.func())
print(str(b))
print (a == b, a != b) #  __eq___() әдісі пайда болады
# A == b салыстыру үшін толық пішін: Point._____ eq(a,    b)
```

Бағдарлама жұмысының нәтижесі:

```
>>>
======= RESTART: C:\Python35-32\point.py ======== (0, 0)
(3, 4)
0
(-19, 4)
False True
>>>
```

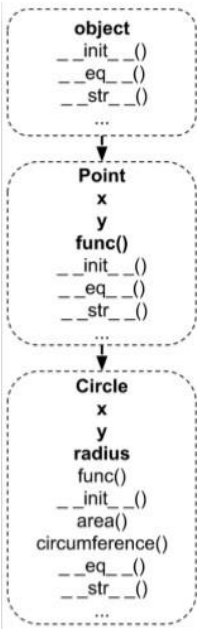
Схемалық иерархия кластарының келесі түрі бар:



Нысандардағы барлық операциялардың артында сәйкес әдістерге қоңыраулар бар екендігі анықталды. Нысандардағы әрбір стандартты операцияға өзіңіздің арнайы әдіс (қосымша әдіс __add___ (), және т.б.).

Айта кету керек, арнайы әдісті (__ne__ ()) теңсіздік үшін a =! B емес, бірақ Python салыстыруды жүзеге асыра алды, өйткені оның нәтижесін кері теңдік ретінде қабылдады (eq ___ () әдісін шақыру).

Біз Point класындағы Circle мұрасынан мұраға аламыз:



Circle сыныбының бастапқы коды:

```
import math
class Circle(Point):
def _ init _ (self, radius, x=0, y=0):
super()._ init _ _ (x, y) # базалық сынып конструкторын шақыру self.radius
= radius
def area (self): # шеңбер ауданы
return math.pi * (self.radius ** 2)
def circumference (self): # шеңбер ұзындығы return 2
* math.pi * self.radius
def _ _ eq_(self, other): # екі шеңберді салыстыру
return self.radius = = other.radius and super()._____ eq _ (other)
def _ _ str _ (self): # ақпаратты жол түрінде шығару return " ({0.radius},
{0.x}, {0.y})".format(self)
circle = Circle(2) # нысанды жасаңыз, radius=2, x=0, y=0
circle.radius = 3
circle.x = 12
a = Circle(4, 5, 6)
b = Circle(4, 5, 6)
print(str (a)) # мұнда арнайы ___str___() әдісі бар
print(str(b))
print (a = b) # мұнда арнайы ___eq___() әдісі бар
# A = b үшін әдісті шақырудың толық нысаны: Circle. _____ eq___ (a, b)
print(a = = circle)
print (a!= circle) # _eq___() әдісінің шақыру нәтижесін терістеу #
базалық кластың әдісін шақыру полиморфизм деп аталады:
print(circle.func())
```

Бағдарлама жұмысының нәтижесі:

```
>>>
======= RESTART: C:\Python35-32\circle.py ========= (4, 5, 6)
(4, 5, 6)
True False True 12
>>>
```

Арнайы әдістер кестелері:

__eq__(self, other)	self == other
__ne__(self, other)	self != other
__lt__(self, other)	self < other
__gt__(self, other)	self > other
__le__(self, other)	self <= other
__ge__(self, other)	self >= other

__add__(self, other)	self + other
__sub__(self, other)	self – other
__mul__(self, other)	self * other
__floordiv__(self, other)	self // other
__truediv__(self, other)	self / other
__mod__(self, other)	self % other
__pow__(self, other)	self ** other

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Обьектіге бағытталған бағдарламалау түсінігі
2. Класстар түсінігі
3. Мұрагерлік ұғымы

Ұсынылатын әдебиеттер:

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.-832с

3. Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

4. Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахь,2020.,-119б

Дәріс №15

Дәріс тақырыбы:Графиктік интерфейстері бар қосымшалар өңдеу

Дәріс мақсаты: Графиктік интерфейстері бар қосымшалармен жқмыс жасау

Кілт сөздер: GUI (Graphical User Interface) – графикалық интерфейс, Оқиға (Event), Оқиға өңдеуші (Event handler), Виджет (Widget), Терезе (Window), Пішін (Form), Батырма (Button), Мәтіндік өріс (Text field / Entry), Белгі (Label), Мәзір (Menu), Диалогтық терезе (Dialog box),Фрейм (Frame / Панель),Координаталар жүйесі (Geometry)

Дәріс жоспары:

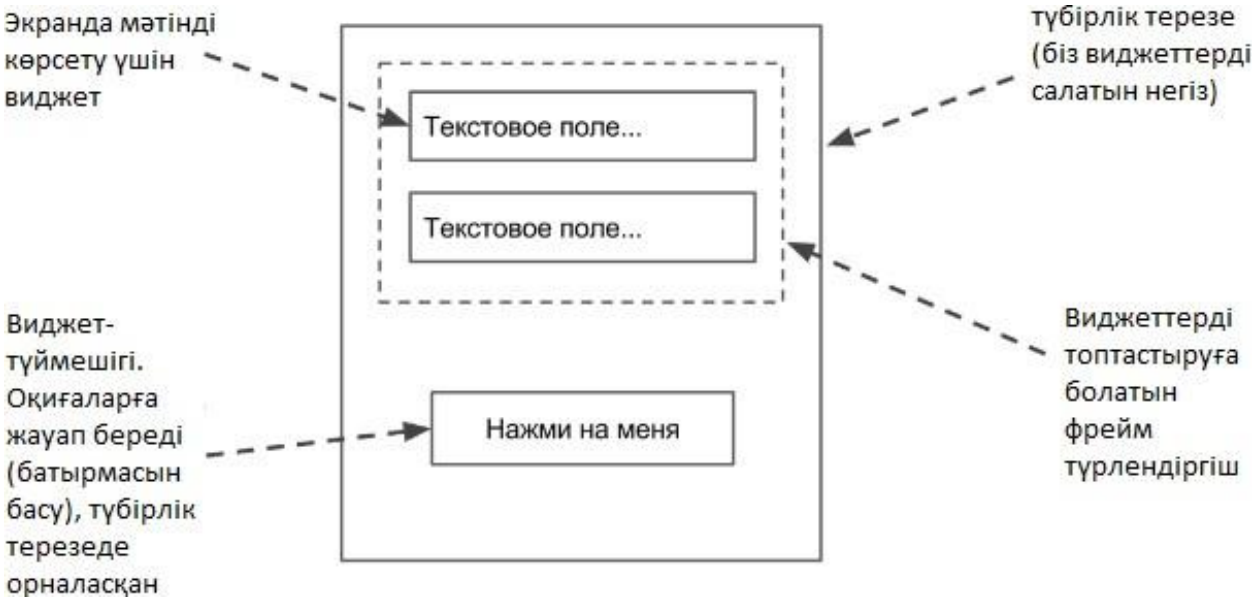
1. *Tkinter модулімен жұмыс істеу негіздері*
2. *Tkinter Модулінің үлгісінде "үлгі-вид-контроллер" үлгісі*
3. *Tkinter - мен жұмыс кезінде әдепкі параметрлерді өзгерту*

1. *Tkinter модулімен жұмыс істеу негіздері*

Python тілі сізге графикалық интерфейспен қосымшаларды жасауға мүмкіндік береді, осы мақсат үшін түрлі графикалық кітапханалар қолданылады. Стандартты (Python стандартты жинағына кіреді) графикалық кітапхананың tkinter бөлімін қарастырайық.

Ең алдымен, tkinter-мен жұмыс жасағанда, графикалық элементтердің қалғанын (виджеттер) орналастыратын негізгі (түбірлік) терезені жасау керек. Мәтінді енгізу, мәтінді шығару, ашылмалы мәзір және т.б. үшін әр түрлі виджеттер бар. Кейбір виджеттер (фреймдер) өз ішіндегі басқа виджеттерді топтастыру үшін пайдаланылады. Арнайы виджет түймешігі басылған кезде кейбір оқиғалар орын алады (оқиғалар өңделуі мүмкін).

Виджеттер жиынтығымен схемалық басты терезе 16- суретте бейнеленген:

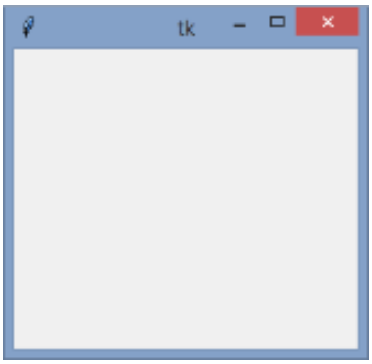


Сурет 16. Виджеттер жиынтығымен схемалық басты терезе

Жеке файлда (mytk1.py, бірақ tkinter.py атымен емес!) негізгі терезені көрсету үшін келесі қарапайым бағдарламаны орындаймыз:

```
# Графикамен жұмыс істеу әдістері бар модульді қосыңыз import
tkinter
# Негізгі (түбірлік) терезені жасаңыз,
# window айнымалысында Tk класының нысанына сілтеме жазамыз window=
tkinter.Tk()
# Түбір терезесі үшін оқиға өңдегішін орнатамыз
window.mainloop()
```

Бағдарламаны орындау нәтижесі:

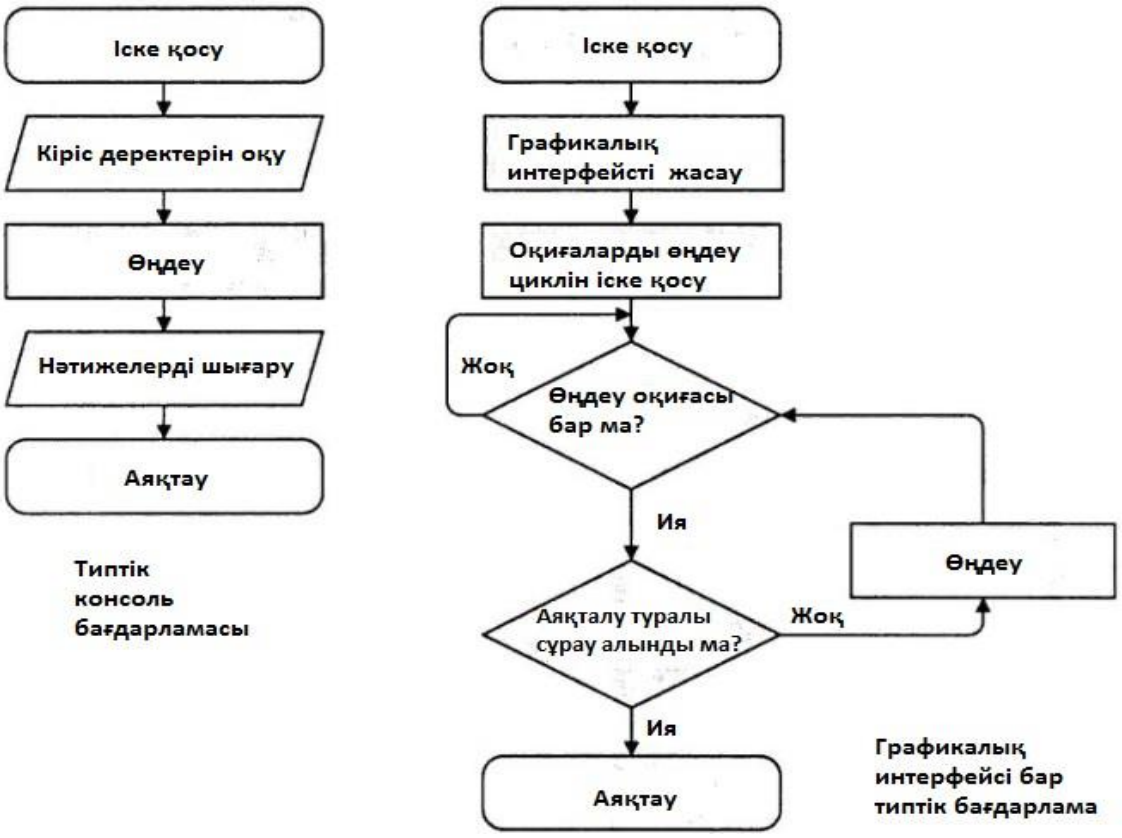


Бүктелетін, созылатын немесе жабылатын толық терезе пайда болды!

Және бұл кодтың үш жолы ғана!

Графикалық (терезе) қосымшалар оқиғадан бас тарту арқылы консольдан (терезесіз) ерекшеленеді. Біз бұрын жұмыс істеген консольдық қосымшалар үшін қандай тінтуірдің түймесі және қандай уақытта бағдарламаның пайдаланушысы басылғанын анықтау қажет емес. Терезенің қосымшаларында тышқанмен басу керек, себебі бұған байланысты, мысалы, пайдаланушы таңдайтын мәзір элементі.

Сол жағында схемада консольдік бағдарламаның жұмыс алгоритмі, оң жағында көрсетілген –графикалық интерфейсі бар бағдарламалар:



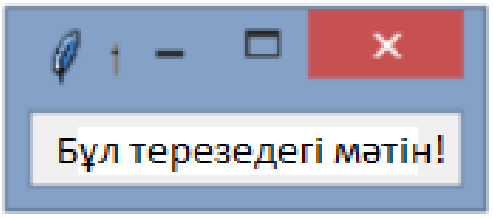
Сурет 17. консольдік бағдарламаның жұмыс алгоритмі

Келесі мысал Label виджет жасауды көрсетеді :

```
Import tkinter
window= tkinter.Tk()
# Түбірлік window таңбаша класының Label виджетін жасаңыз # text - көрсетілген мәтін үшін параметр
label= tkinter.Label(window, text = "Бұл терезедегі мәтін!") # pack менеджері арқылы виджетті көрсетеміз label.pack()
window.mainloop()
```

Бағдарлама жұмысының нәтижесі:

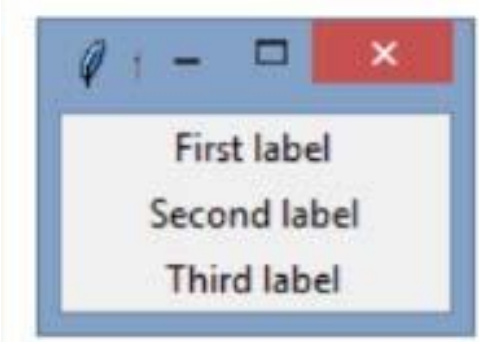
Келесі мысал фреймде виджеттердің орналасуын көрсетеді: import




```
tkinter
window= tkinter.Tk()
# Негізгі терезеде кадр жасаңыз
frame = tkinter.Frame(window) frame.pack()
# Виджеттерді жасап, frame жақтауларына орналастырамыз first = tkinter.Label(frame, text='First label')
# pack менеджері арқылы виджетті көрсетеміз
first.pack()
second = tkinter.Label(frame, text='Second label')
second.pack()
third = tkinter.Label(frame, text='Third label')
third.pack()
window.mainloop() Бағдарламаны
```

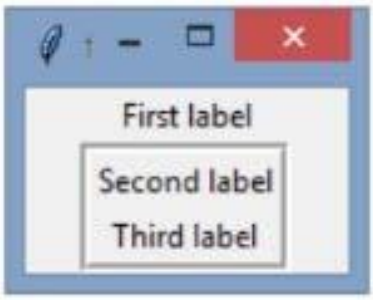
орындау мысалы:

Нысанды жасау кезінде жақтау параметрлерін өзгертуге болады:




```
import tkinter
window = tkinter.Tk()
frame = tkinter.Frame(window)
frame.pack()
# Фрейм параметрлерін өзгерте аламыз:
frame2 = tkinter.Frame(window, borderwidth=4, relief=tkinter.GROOVE) frame2.pack()
# Виджетті бірінші фреймге орналастырамыз (frame) first =
tkinter.Label(frame, text='First label')
first.pack()
# Виджеттерді екінші фреймге орналастырамыз (frame2) second
```

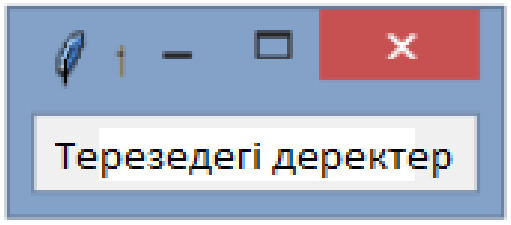
= tkinter.Label(frame2, text='Second label') second.pack()
third = tkinter.Label(frame2, text='Third label')
third.pack()
window.mainloop()



Келесі мысалда, Label виджетінде айнымалы мәнді көрсету үшін, StringVar деректер сыныбының айнымалы мәні (tkinter модулінен) пайдаланылады. Келесіде, неге tkinter өз класындағы айнымалы мәндерді пайдаланатын мысалдардан айқынырақ болады.

import tkinter
window = tkinter.Tk()
StringVar сынып нысанын жасап, оған data белгісін тағайындаңыз # (tkinter жұмыс істей алатын жол айнымалысын жасаңыз)
data = tkinter.StringVar()
StringVar класын теру әдісі айнымалының мазмұнын өзгертуге мүмкіндік береді:
data.set('Терезедегі деректер')
textvariable айнымалы деректерден жол нысанына сілтеме береміз label = tkinter.Label(window, textvariable = data)
label.pack()
window.mainloop()

Бағдарламаны орындау нәтижесі:

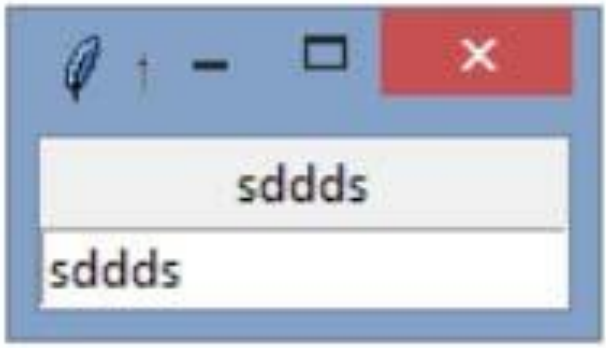


2. *Tkinter Модулінің үлгісінде "үлгі-вид-контроллер" үлгісі*

Келесі мысал деректерді енгізу үшін виджетті (Entry) қалай пайдалану керектігін көрсетеді:

import tkinter
window = tkinter.Tk()
frame = tkinter.Frame(window)
frame.pack()
var= tkinter.StringVar()
Айнымалы мазмұнды жаңарту нақты уақыт режимінде болады label = tkinter.Label (frame, textvariable=var)
label.pack()
Пайда болған өрісте мәтінді теруге тырысамыз entry = tkinter.Entry (frame, textvariable=var) entry.pack()
window.mainloop()

Бағдарламаны іске қосып, еркін мәтінді теруге тырысамыз:



Терілген мәтіннің терезеде бірден көрінетінін көреміз, себебі Label және Entry виджеттері тиісінше мәтінді көрсету және енгізу үшін StringVar

сыныбының бір және бір айнымалы мәнін пайдаланады. Терезедегі қосымшаның осындай схемасы «Model-view-controller» (Model-View- Controller немесе MVC) деп аталатын әмбебап үлгіге (үлгі) сәйкес келеді.

Жалпы айтқанда, модель (Model) деректерді сақтау тәсілі ретінде түсініледі, яғни. деректер қалай сақталады (мысалы, сынып айнымалысы) Түрі (View) деректерді көрсету үшін пайдаланылады. Контроллер (Controller) деректерді өңдеуге жауапты.

Келесі схема MVC моделінің барлық компоненттерінің байланысын көрсетеді:



Сурет 18. MVC моделінің барлық компоненттерінің байланысы

MVC-дің қызықты ерекшелігі, егер контролер деректерді өзгерткен болса (алдыңғы айнымалы мысалдағыдай var айнымалы мәнін өзгерту арқылы), өзгертілген айнымалы дисплейдің (терезені қайта жасау) туралы көрініске «сигнал жіберіледі», бұл нақты уақыттағы мәтін жаңартуларына әкеледі.

Келесі мысалда батырма басылған кездегі оқиғаларды өңдеу мүмкіндіктері көрсетілген (Button виджеті):

import tkinter

Контроллер: функция түймені басқанда шақырылады def click():

get() әдісі - ағымдағы counter мәнін қайтарады # set() әдісі - жаңа counter мәнін орнатады counter.set (counter.get () + 1)

```
window = tkinter.Tk()
# Үлгі: IntVar сынып нысанын жасаңыз counter
= tkinter.IntVar()
# Құрылған нысанды set() әдісі арқылы қалпына келтіреміз counter.set
(0)
frame = tkinter.Frame(window)
frame.pack()
# Батырманы жасап, оны басқан кезде өңдеушіні (click функциясын) көрсетеміз
button = tkinter.Button (frame, text='Click', command=click)
button.pack()
# Көрініс: нақты уақытта label виджеті жаңартылады label =
tkinter.Label (frame, textvariable=counter) label.pack()
window.mainloop()
```

Бағдарламаны орындау нәтижесі:



Екі батырмамен және екі оқиғаны өңдегішпен күрделі мысал (click_up, click_down):

```
import tkinter
window = tkinter.Tk() #
Моделі:
counter = tkinter.IntVar()
counter.set(0)
# Екі контроллер:
def click_up():
counter.set(counter.get () + 1) def
click_down(): counter.set(counter.get () -
1)
# Түрі:
frame = tkinter.Frame(window)
frame.pack()
button = tkinter.Button (frame, text='Up', command=click_up)
```

button.pack()
button = tkinter.Button (frame, text='Down', command=click_down) button.pack()
label = tkinter.Label (frame, textvariable=counter)
label.pack()
window.mainloop()

Бағдарлама жұмысының нәтижесі:



3. Tkinter - мен жұмыс кезінде әдепкі параметрлерді өзгерту

Tkinter виджеттер жасаған кезде оларды өзгертуге мүмкіндік береді: import tkinter
window = tkinter.Tk()
Түймешікті жасаңыз, шрифтті пайдаланып картежбен өзгертіңіз button = tkinter.Button(window, text= 'Hello',
font=('Courier', 14, 'bold italic'))
button.pack() window.mainloop()
Бағдарламаны орындау нәтижесі:



Келесі мысалда label виджет параметрлері өзгереді: import tkinter
window = tkinter.Tk()
Мәтін өңін, түсін өзгертіңіз:
button = tkinter.Label(window, text= 'Hello', bg= 'green', fg='white') button.pack()
window.mainloop()

Бағдарламаны орындау нәтижесі:



Орналасу менеджері (геометрия) pack-та да параметрлері бар: import tkinter

window = tkinter.Tk()

frame = tkinter.Frame(window) frame.pack()

label = tkinter.Label(frame, text='Name') # Сол жақ жиегі бойынша туралау label.pack(side= 'left')

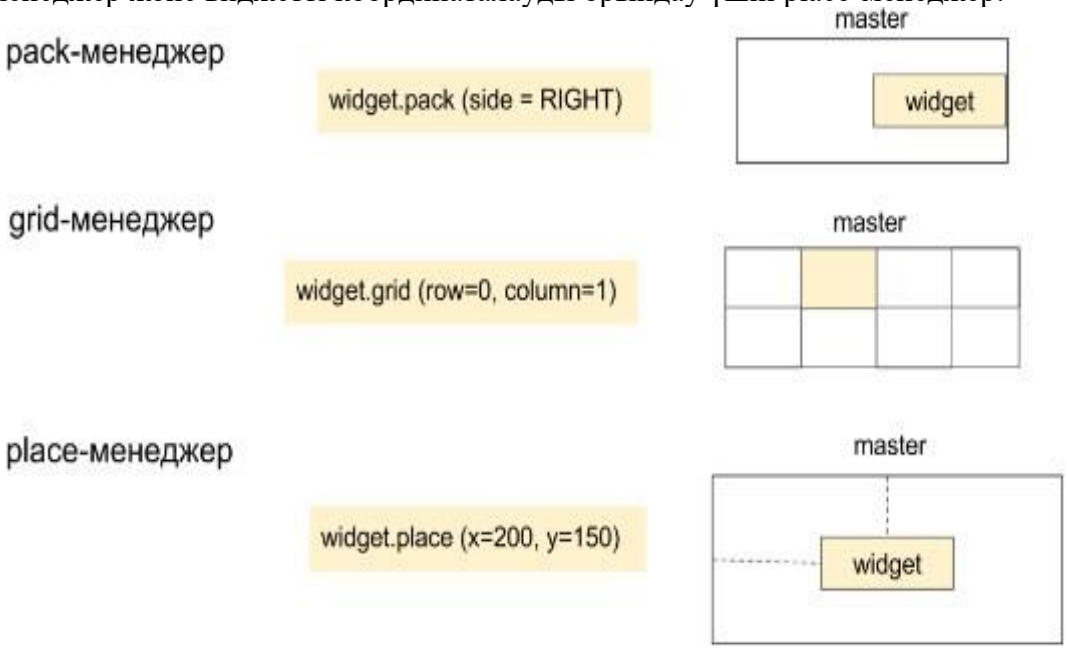
entry = tkinter.Entry(frame) entry.pack(side= 'left') window.mainloop()

Бағдарламаны орындау нәтижесі:



Анықтама үшін. Орналасу менеджерлері (геометрия)

Tkinter виджеттерді орналастыру үшін бірнеше жолы бар. Олардың ішінде: біз бұрын пайдаланылған pack-менеджер, grid-виджетті орналастыру үшін жол және баған үшін менеджер және виджетті координаталауды орындау үшін place-менеджер:



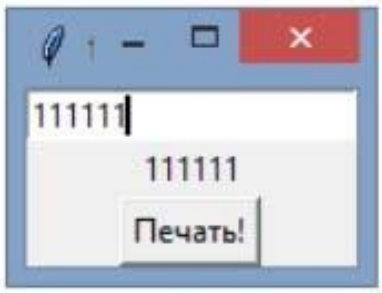
Сурет 19. place-менеджер

Келесі мысалдың ерекшелігі, ендірілген мәтін (Entry виджеті арқылы) экранда (Label виджеті арқылы) түймешік басылған кезде (Button виджеті) экранда көрсетіледі, ол бұрынғыдай емес:

```
import tkinter
# Батырманы басу кезінде пайда болады:
def click():
# Get() әдісі арқылы енгізу өрісінің жол мазмұнын аламыз # Config () көмегімен көрсетілетін мәтінді өзгерте аласыз label.config (мәтін=entry.get())

window = tkinter.Tk()
frame = tkinter.Frame(window)
frame.pack()
entry = tkinter.Entry(frame)
entry.pack()
label = tkinter.Label(frame)
label.pack()
# Click() функциясына батырманы басу өңдегішті байланыстырамыз button = tkinter.Button(frame, text= ' Печать!', command=click) button.pack()
window.mainloop()
```

Бағдарламаны орындау нәтижесі:



PyQt - бұл Python кеңейту ретінде жобаланған Python бағдарламалау тілі, үшін «Якорь» графикалық негіздемелік Qt жиынтығы.

PyQt британдық Riverbank Computing компаниясы әзірледі. Linux және басқа да UNIX-тәрізді операциялық жүйелер, Mac OS X және Windows: PyQt Qt қолдау барлық платформаларда жұмыс істейді. екі нұсқасы бар: PyQt5, Qt Qt 4. PyQt GPL лицензиясы (нұсқа 2 және 3) сәйкес және коммерциялық кеңейтеді қолдау, 5 және PyQt4 қолдайды.

PyQt толықтай дерлік Qt мүмкіндігін жүзеге асырады болып табылады. : Және бұл, оның ішінде 600-ден астам сыныптар мен 6000-ден астам функциялары мен әдістері болып табылады

- GUI виджеттер ағымдағы жиынтығы;
- Виджет стильдер;
- SQL (ODBC, MySQL, PostgreSQL, Oracle) пайдалана отырып, деректер базасына қол жеткізу;
- Крупица Виджет мәтіндік редакторы негізделген QScintilla;
- Халықаралық қолдау (i18n);
- XML талдағышы;
- қолдау SVG;
- WebKit, көрсету тетігін, HTML интеграция;
- бейне және аудио қолдау.

PyQt сонымен қатар графикалық пайдаланушы интерфейсінің дизайнерін Qt

Designer (Qt Creator) қамтиды. Pyuic бағдарламасы Qt Designer-да жасалған файлдардан Python кодын жасайды. Бұл PyQt жылдам прототипі үшін өте пайдалы құрал жасайды. Сонымен қатар, Qt Designer-ге Python-да жазылған жаңа графикалық басқару элементтерін қосуға болады.

Бұған дейін PyQt Eric әзірлеу ортасымен жіберілді PyQt. Эрик кірістірілген күйге келтірушімен жабдықталған және консольдық бағдарламаларды жасау үшін пайдаланылуы мүмкін. Енді ол жеке жоба ретінде қол жетімді

- QtCore - негізгі графикалық емес сыныптар: сигналдар мен слоттар жүйесі, Unicode үшін платформаға тәуелсіз абстракциялар, жіптер, ортақ жад, тұрақты өрнектер және т.с.с.
- QtGui - визуалды ұсыну негізінде графикалық интерфейстің (басқару элементтері) компоненттері.
- QtNetwork - желілік бағдарламалау үшін сыныптар. Мысалы, UDP және TCP арқылы клиенттер мен серверлер.
- QtOpenGL - PyQt қосымшаларында OpenGL және 3D графиктерін пайдалануға мүмкіндік беретін сыныптар.
- QtScript - қолданбаны басқару үшін Qt ішіне орнатылған JavaScript интерпретаторын пайдалануға мүмкіндік беретін сыныптар.
- QtSql - SQL көмегімен дерекқорлармен біріктіруге арналған сыныптар.
- QtSvg - SVG форматында векторлық графиканы көрсету үшін сыныптар.
- XML өңдеуді жүзеге асыратын QtXml сыныптары.
- uic - Python-GUI-ны құру үшін Qt Designer-да жасалатын өңдеу XML файлдарын енгізу

Qt Дизайнер кросс-платформаны құрастырушы болып табылады және графикалық пайдаланушы интерфейсін қалыптастырады. Бұл бағдарламада пайдаланылатын бірдей виджеттерді пайдаланып, экрандық нысандарды пайдаланып, виджеттер мен диалогтық терезелерді тез жасауға мүмкіндік береді. Qt Designer-мен жасалған нысандар толығымен функционалды және нақты уақыт режимінде де көрінуі мүмкін.

Неғұрлым күрделі мысал

Бұл мысал неғұрлым маңызды бағдарламаларды құру принциптерін көрсетеді. Ол Python 3 және Qt Designer пайдаланады. Бағдарламаның көрнекі көрінісі мен логикасын бөлуге мүмкіндік беретін объектілі- бағытталған көзқарас пайдаланылады.

test.pyw - бағдарламаны іске қосатын негізгі файл.
mainform.py - терезенің негізгі сипаттамасы.

mainform.ui - Qt Designer автоматты түрде жасалатын негізгі терезенің сипаттамасын қамтитын файл. Qt Designer QDialog негізіндегі жоба құрды және 2 элемент таңдалды: labelButton деп аталатын мәтін белгісі, pushButton деп аталатын түймешік.

Test.pyw мазмұны:

```
#! /usr/bin/env python3
#import sys
from PyQt4 import QtCore, QtGui import
mainform
def main():
app = QtGui.QApplication(sys.argv) form
= mainform.MainForm() form.show()
return app.exec()
if _____name___ ==          "
_____main____": sys.exit(main())
```

Mainform.py файлының мазмұны:

```
        #! /usr/bin/env python3
#
        from PyQt4 import QtCore, QtGui, uic class
MainForm(QtGui.QDialog):
        def _____init__________(self):
super(MainForm, self).________init___()
        uic.loadUi("mainform.ui", self)
```

15.1 жаттығу

Фаренгейт дәрежесін Цельсийге аударатын бағдарламаны жазыңыз. Бағдарламамен жұмыс істеу интерфейсі төменде берілген.





15.2 жаттығу

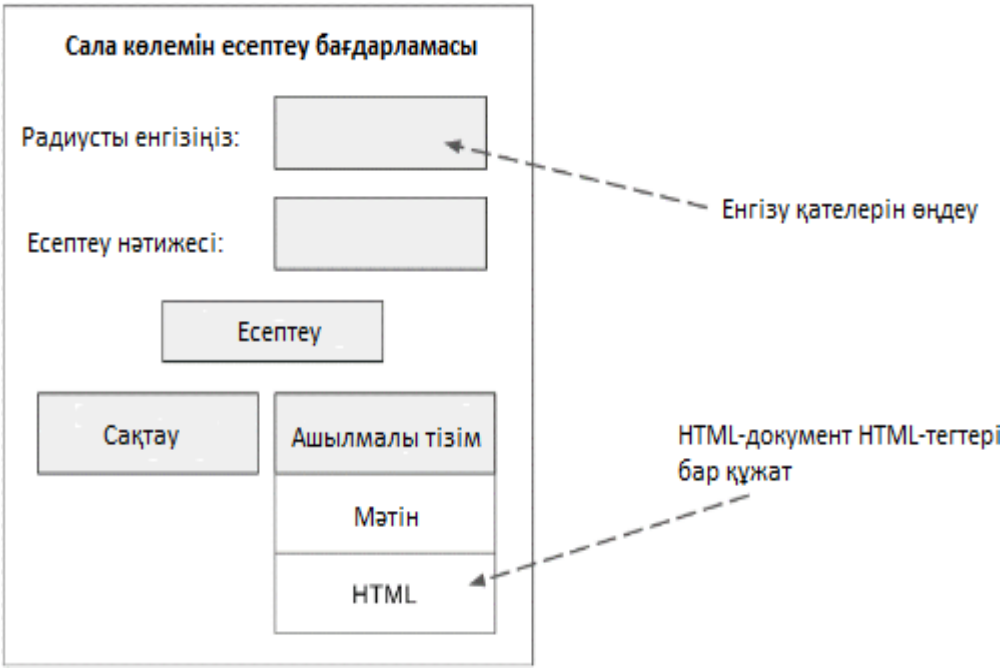
Орыс тілінде кездейсоқ сөзді көрсететін бағдарламаны жазыңыз (деректер түрі dict). Пайдаланушы оны ағылшын тілінде (немесе басқа тілде) болжауға тырысады. Сонымен қатар бағдарламаны дұрыс емес болжалды сөздердің санын шектеу.

15.3 жаттығу*

Пернетақтадан мәтінді кәдімгі файлға немесе HTML файлы түрінде сақталатын еркін мәтінді енгізуге мүмкіндік беретін бағдарламаны жазыңыз (файл түрі ашылмалы мәзір арқылы көрсетіледі).

15.4 жаттығу

Келесі графикалық интерфейспен бағдарламаны жасаңыз, ықтимал қателерді өңдеуді қамтамасыз етіңіз.



Сурет 20. Сала көлемін есептеу бағдарламасы

Студенттердің өзін-өзі тексеруге арналған сұрақтары

1. Phyton-да қай әдіс шеңбер сызады?
2. Phyton-да қай әдіс мәтінді шығарады?
3. Phyton-да функцияны қалай шақырады?
4. Phyton-да файлды ашу үшін қандай функциясы қолданылады?
5. Phyton-да қандай қызметші сөз нақты айнымалыны сипаттайды?
6. Phyton-да қай әдіс сызық сызады?
7. Phyton-да қай әдіс тіктөртбұрыш сызады?
8. Phyton-да қай әдіс эллипс сызады?
9. Phyton-да қай әдіс доға сызады?
10. Phyton-да файлды жабу үшін қандай функциясы қолданылады?
11. Phyton тілінде файлды ашу үшін қандай әдіс қолданылады?
12. Айнымалының сипатталуы :
13. Phyton-да файлды оқу және жазу үшін қандай функциясы қолданылады?
14. Phyton-да файл ашылғанда тек бос жол көрсететін қандай функциясы қолданылады?
15. Phyton-да қай әдіс көпбұрыш сызады?
16. Python-да графикалық интерфейстерді қамтамасыз етуге арналған кітапхананы атаңыз

Ұсынылатын әдебиеттер:

Негізгі әдебиеттер:

1. Ермеков Н.Т., Алгоритмдеу және бағдарламалау. Оқу құралы. Алматы: ТОО «Лантар Трейд», 2019.,-159 с.

2. Ермеков Н.Т., Таржибаева Б.Е. Введение в программирование на языке Python Алматы.Лантар Трейд .2020.-202с.

3. Халикова К.З. Идрисов С.Н. Маликова Н.Т. Python тілінде программалау негіздері: Оқу құралы. Алматы, изд. Lantar books, 2022

4. Плас, Дж.Вандер. Python для сложных задач: наука о данных и машинное обучение : учебник / Дж.Вандер Плас.- СПб: Питер, 2020.- 576с**.**

5. Доусон, М. Программируем на Python.- 3-е изд.- СПб: Питер, 2020.- 416с.

6. Асанбай Б.З. PYTHON тілінде программалау негіздері әдістемелік құрал - Ақтөбе: «Өрлеу» БАҰО» АҚ филиалының баспа бөлімі, 2021 ж.-77 б.

Қосымша әдебиет:

1. Д. Ю. Федоров. Программирование на языке высокого уровня Python. Санкт-Петербург 2019, – 85 – 90 с.

2. Прохоренок, Н. Python 3 и Pyqt 5. Разработка приложений. СПб: БХВ-Петербург, 2019.- 832с

3. Прохоренок Н. А. Python 3 и PyQt. Разработка приложений Спб: BHV, 2014 г. 704 стр.[pic 1]

4. Бекенова А.С., Python бағдарламалау тілі., Оқу құралы., Алматы. Альманахъ,2020.,-119б